

专题1：JVM面试题（史上最全、定期更新）

本文版本说明：V173

此文的格式，由 markdown 通过程序转成而来，由于很多表格，没有来的及调整，出现一个格式问题，尼恩在此给大家道歉啦。

由于社群很多小伙伴，在面试，不断的交流最新的面试难题，所以，《[尼恩Java面试宝典](#)》，后面会不断升级，迭代。

本专题，作为《尼恩Java面试宝典》专题之一，《尼恩Java面试宝典》一共41个面试专题。

《尼恩Java面试宝典》升级的规划为：

后续基本上，**每一个月，都会发布一次**，最新版本，可以扫描扫描架构师尼恩微信，发送“领电子书”获取。

尼恩的微信二维码在哪里呢？具体可以百度搜索 **疯狂创客圈 总目录**

面试问题交流说明：

如果遇到面试难题，或者职业发展问题，或者中年危机问题，都可以来 疯狂创客圈社群交流，

加入交流群，加尼恩微信即可，

尼恩的微信二维码在哪里呢？具体参见文末

升级说明：

V173升级说明（2024-09-09）：

美团面试：G1 垃圾回收 底层原理是什么？说说你的调优过程？

修改了一个大的bug，标记整理用于 老年代，之前的版本，有一句话错误的写为，用于年轻代

V107升级说明（2023-09-09）：

微博一面：JVM预热，你的方案是啥？

V87升级说明（2023-06-17）：

面试真题：你们用什么版本的jdk？聊一聊不同版本的jdk的新特性？

V81升级说明 (2023-06-17) :

美团一面: OOM后, JVM一定会退出吗? 为什么?

V13升级说明 (2022-12-06) :

- 大厂面试题: Java中的对象一定是在堆上分配的吗?
- 大厂面试题: 聊聊, 什么是即时编译?

V10升级说明 (2022-11-08) :

- 问: Java中的String对象的大小?
- 场景题: 说说JVM相关的类的加载、链接和初始化的问题?

V4升级说明 (2022-5-20) , 增加了方法区的4个连环炮:

- 问: 介绍一下方法区和永久代的区别?
- 问: 说是方法区和堆的区别?
- 问: 如何设置方法区内存的大小?
- 问: 如何解决方法区的OOM?

V3升级说明 (2022-5-16) :

升级了3个面试连环炮

JVM内存连环炮:

JVM内存包括哪些? 什么是堆内存? 什么是非堆内存? 什么是直接内存? 方法区和永久代有何区别?

对象结构连环炮:

给定一个具体的类, 请分析对象的内存占用? 怎么计算出来一个对象的内存占用? 对象头中包含哪些部分?

JVM调优连环炮:

常用的JVM启动参数有哪些? 调优命令有哪些? 设置堆内存XMX应该考虑哪些因素? 假设物理内存是8G, 设置多大堆内存比较合适? 怎样开启GC日志?



超级面试题: 一份 4000 页《尼恩Java面试宝典》, 不断迭代、不断更新 @公众号 技术自由圈

社群真题

以下真题, 来自于卷王群, 是小伙伴碰到的。

社群真题1：元空间会产生内存溢出么？在什么情况下会产生内存溢出？

具体问题：元空间会产生内存溢出么？在什么情况下会产生内存溢出？。

java8 及以后的版本使用Metaspace来代替永久代，Metaspace是方法区在HotSpot中的实现，它与永久代最大区别在于，Metaspace并不在虚拟机内存中而是使用本地内存也就是在JDK8中,classe metadata(the virtual machines internal presentation of Java class),被存储在叫做Metaspace的 native memory.

永久代 (java 8 后被元空间Metaspace取代了) 存放了以下信息：

- 虚拟机加载的类信息
- 常量池
- 静态变量
- 即时编译后的代码

出现问题原因

错误的主要原因, 是加载到内存中的 class 数量太多或者体积太大。

解决办法

增加 Metaspace 的大小

```
-XX:MaxMetaspaceSize=512m
```

代码演示

模拟Metaspace空间溢出，我们不断生成类往元空间灌，类占据的空间是会超过Metaspace指定的空间大小的

查看元空间大小

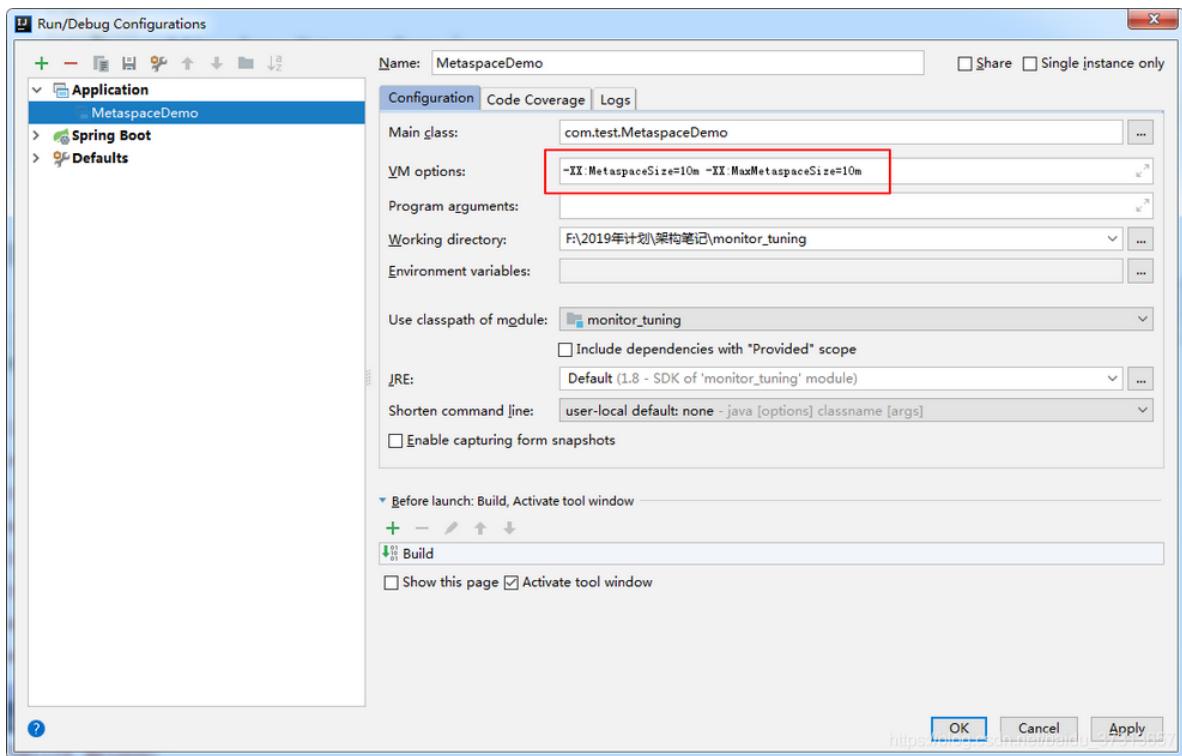
```
java -XX:+PrintFlagsInitial
```

uintx MaxTenuringThreshold	= 15	{product}
intx MaxTrivialSize	= 6	{product}
intx MaxVectorSize	= 32	{C2 product}
uintx MetaspaceSize	= 21810376	{pd product}
bool MethodFlushing	= true	{product}
uintx MinHeapDeltaBytes	= 170392	{product}
uintx MinHeapFreeRatio	= 40	{manageable}
intx MinInliningThreshold	= 250	{nproduct}

https://blog.codn.net/bsidu_37313657

设置配置 这里设置10m方便演示效果

```
-XX:MetaspaceSize=10m -XX:MaxMetaspaceSize=10m
```



编写代码

```
import org.springframework.cglib.proxy.Enhancer;
import org.springframework.cglib.proxy.MethodInterceptor;
import org.springframework.cglib.proxy.MethodProxy;

import java.lang.reflect.Method;

public class MetaspaceDemo {
    static class OOM{}
    public static void main(String[] args) {
        int i = 0; //模拟计数多少次以后发生异常
        try {
            while (true){
                i++;
                Enhancer enhancer = new Enhancer();
                enhancer.setSuperclass(OOM.class);
                enhancer.setUseCache(false);
                enhancer.setCallback(new MethodInterceptor() {
                    @Override
                    public Object intercept(Object o, Method method, Object[] objects,
                    MethodProxy methodProxy) throws Throwable {
                        return methodProxy.invokeSuper(o,args);
                    }
                });
                enhancer.create();
            }
        } catch (Throwable e) {
            System.out.println("=====多次后发生异常: "+i);
            e.printStackTrace();
        }
    }
}
```

运行结果：

```
Run: MetaspaceDemo x
"C:\Program Files\Java\jdk1.8.0_171\bin\java.exe" ...
org.springframework.cglib.core.CodeGenerationException: java.lang.OutOfMemoryError-->Metaspace
  at org.springframework.cglib.core.ReflectUtils.defineClass(ReflectUtils.java:530)
  at org.springframework.cglib.core.AbstractClassGenerator.generate(AbstractClassGenerator.java:363)
  at org.springframework.cglib.proxy.Enhancer.generate(Enhancer.java:582)
  at org.springframework.cglib.core.AbstractClassGenerator$ClassLoaderData.get(AbstractClassGenerator.java:131)
  at org.springframework.cglib.core.AbstractClassGenerator.create(AbstractClassGenerator.java:319)
  at org.springframework.cglib.proxy.Enhancer.createHelper(Enhancer.java:569)
  at org.springframework.cglib.proxy.Enhancer.create(Enhancer.java:384)
  at com.test.MetaspaceDemo.main(MetaspaceDemo.java:25)
=====多次后发生异常： 569
Caused by: java.lang.OutOfMemoryError: Metaspace
  at java.lang.ClassLoader.defineClass1(Native Method)
  at java.lang.ClassLoader.defineClass(ClassLoader.java:763) <3 internal calls>
  at org.springframework.cglib.core.ReflectUtils.defineClass(ReflectUtils.java:527)
  ... 7 more

Process finished with exit code 0
```

https://blog.csdn.net/baidu_37313657

更多面试真题，请来 疯狂创客圈社群的 微信群交流，具体请参考 [Java 高并发 发烧友社群：疯狂创客圈（总入口）](#)



转架构：通过尼恩的指导，小伙伴 转架构 后年薪从40W 涨到70W @公众号 技术自由圈

正式内容1：Java内存区域

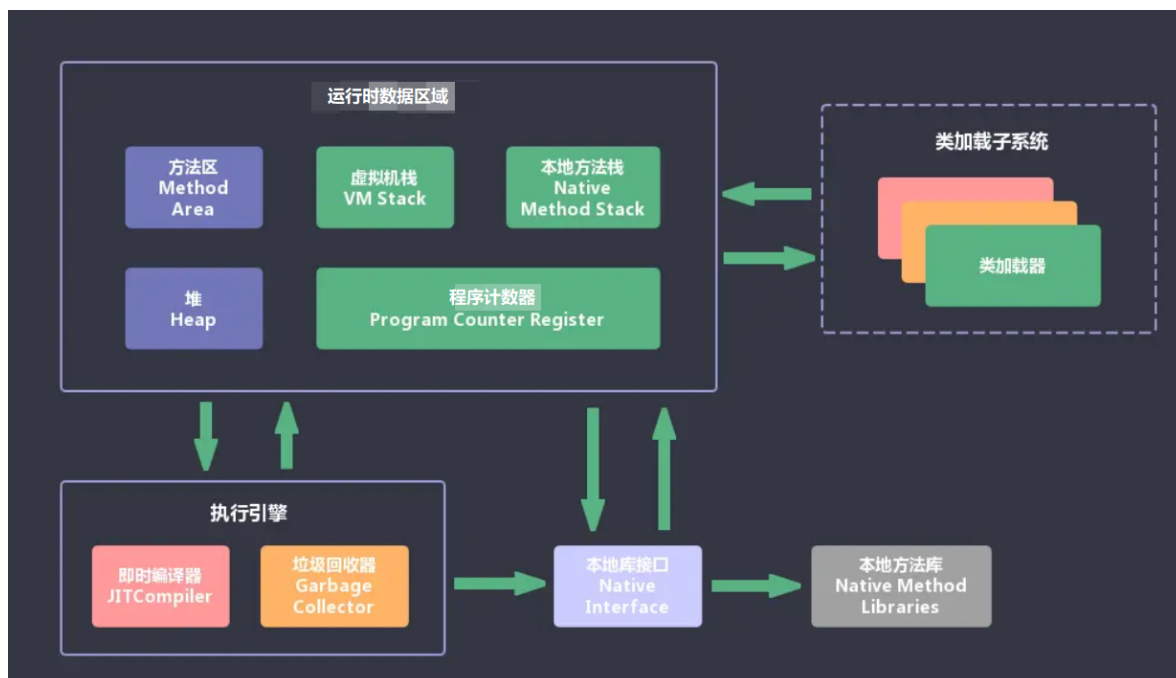
问：说一下 Java 堆空间及 GC？

当通过 Java 命令启动 Java 进程的时候，会为它分配内存。

内存的一部分用于创建堆空间，当程序中创建对象的时候，就从对空间中分配内存。

GC 是 JVM 内部的一个进程，回收无效对象的内存用于将来的分配。

问：说一下 JVM 的主要组成部分及其作用？



JVM包含两个子系统和两个组件，两个子系统为Class loader(类装载)、Execution engine(执行引擎)；两个组件为Runtime data area(运行时数据区)、Native Interface(本地接口)。

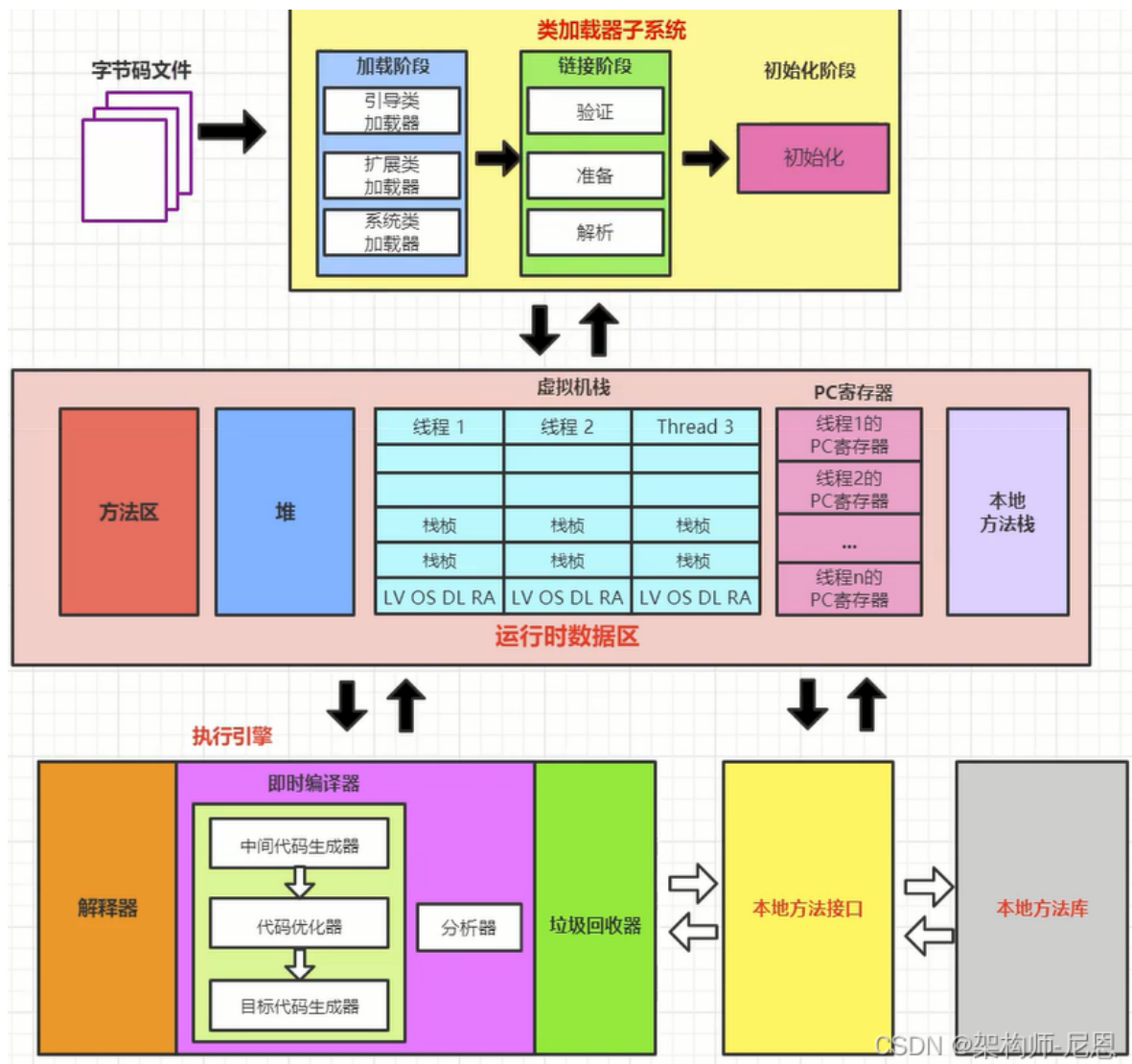
- Class loader(类装载)：根据给定的全限定名类名(如：java.lang.Object)来装载class文件到Runtime data area中的method area。
- Execution engine（执行引擎）：执行classes中的指令。
- Native Interface(本地接口)：与native libraries交互，是其它编程语言交互的接口。
- Runtime data area(运行时数据区域)：这就是我们常说的JVM的内存。

作用：

首先通过编译器把Java代码转换成字节码，类加载器（ClassLoader）再把字节码加载到内存中，将其放在运行时数据区（Runtime data area）的方法区内，

而字节码文件只是JVM的一套指令集规范，并不能直接交给底层操作系统去执行，因此需要特定的命令解析器执行引擎（Execution Engine），将字节码翻译成底层系统指令，再交由CPU去执行，

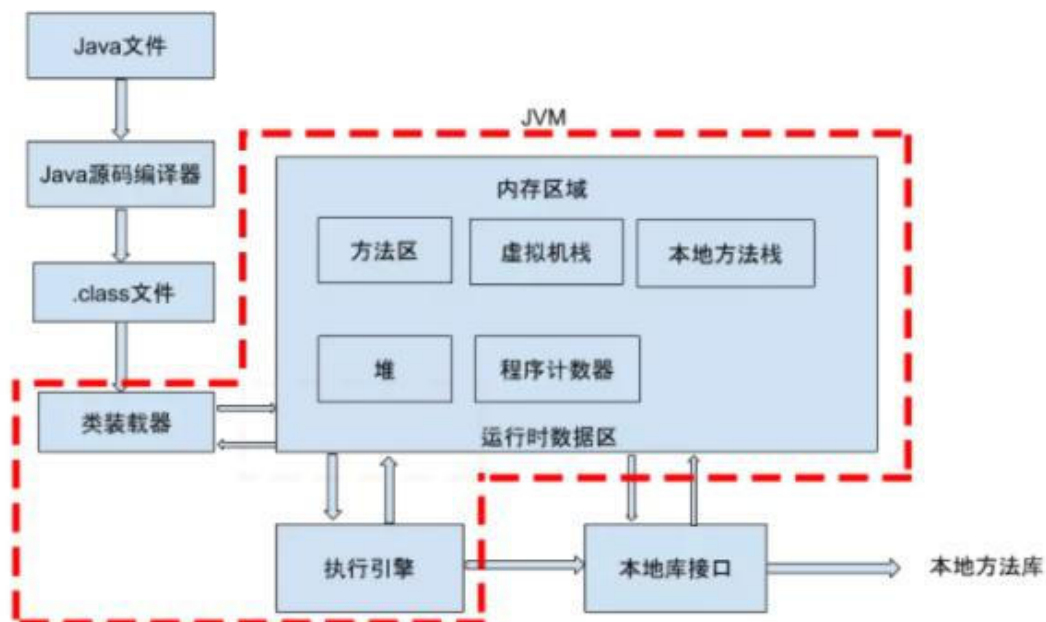
同时，Java代码中，可以调用其他语言的本地库接口（Native Interface）来，进行一些系统调用或者C函数调用。



下面是Java程序运行机制详细说明

Java程序运行机制步骤

- 首先利用IDE集成开发工具编写Java源代码，源文件的后缀为.java；
- 再利用编译器(javac命令)将源代码编译成字节码文件，字节码文件的后缀名为.class；
- 运行字节码的工作是由解释器(java命令)来完成的。



<https://blog.csdn.net/ThinkWon>

从上图可以看，java文件通过编译器变成了.class文件，接下来类加载器又将这些.class文件加载到JVM中。

其实可以一句话来解释：类的加载指的是将类的.class文件中的二进制数据读入到内存中，将其放在运行时数据区的方法区内，然后在堆区创建一个 java.lang.Class对象，用来封装类在方法区内的数据结构。



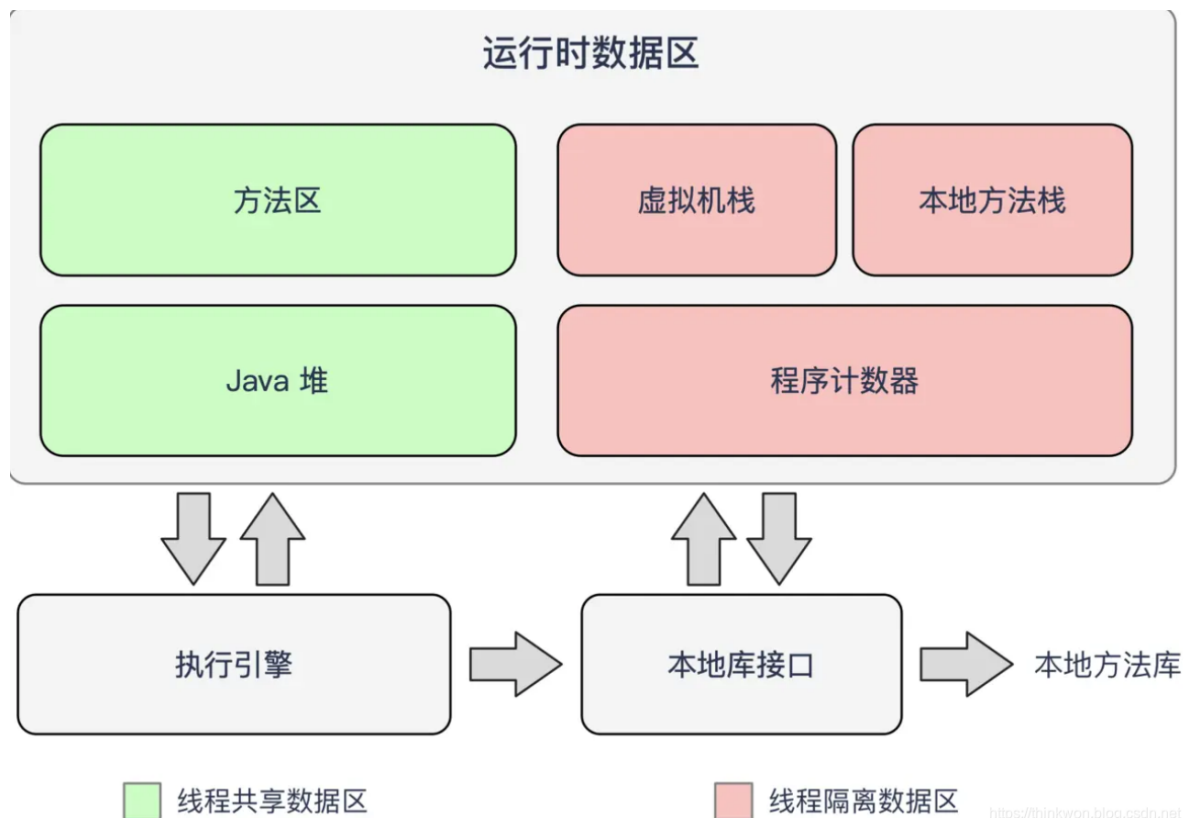
转架构：通过尼恩的指导，小伙伴 转架构 后年薪从40W 涨到70W @公众号 技术自由圈

问：说一下 JVM 运行时数据区？ 或：说一下JVM内存模型？

思路：给面试官画一下JVM内存模型图，并描述每个模块的定义，作用，以及可能会存在的问题，如栈溢出等。

Java 虚拟机在执行 Java 程序的过程中会把它所管理的内存区域划分为若干个不同的数据区域。这些区域都有各自的用途，以及创建和销毁的时间，有些区域随着虚拟机进程的启动而存在，有些区域则是依赖线程的启动和结束而建立和销毁。

Java 虚拟机所管理的内存被划分为如下几个区域：



- 程序计数器 (Program Counter Register)：当前线程所执行的字节码的行号指示器，字节码解析器的工作是通过改变这个计数器的值，来选取下一条需要执行的字节码指令，分支、循环、跳转、异常处理、线程恢复等基础功能，都需要依赖这个计数器来完成；
- Java 虚拟机栈 (Java Virtual Machine Stacks)：用于存储局部变量表、操作数栈、动态链接、方法出口等信息；
- 本地方法栈 (Native Method Stack)：与虚拟机栈的作用是一样的，只不过虚拟机栈是服务 Java 方法的，而本地方法栈是为虚拟机调用 Native 方法服务的；
- Java 堆 (Java Heap)：Java 虚拟机中内存最大的一块，是被所有线程共享的，几乎所有的对象实例都在这里分配内存；
- 方法区 (Method Area)：用于存储已被虚拟机加载的类信息、常量、静态变量、即时编译后的代码等数据。

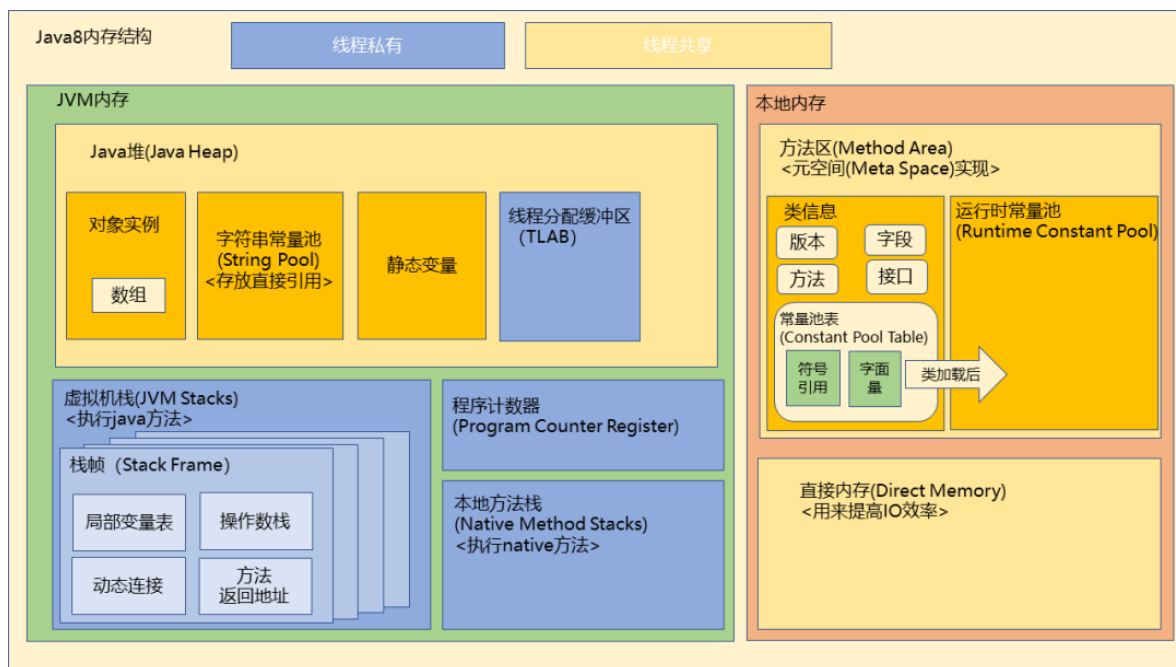
JVM内存连环炮，问：JVM内存包括哪些？

JVM总内存= 栈 + 堆 + Native

Native= 堆外内存（直接内存）+ 元空间

非堆 = 元空间

JVM8 的内存结构



程序计数器 (Program Counter Register)

程序计数器就是当前线程所执行的字节码的行号指示器，通过改变计数器的值，来选取下一行指令，通过他来实现跳转、循环、恢复线程等功能。

- 在任何时刻，一个处理器内核只能运行一个线程，多线程是通过线程轮流切换，分配时间来完成的，这就需要有有一个标志来记住每个线程执行到了哪里，这里便需要到了程序计数器。
- 程序计数器是线程私有的，每个线程都有自己的程序计数器。**

虚拟机栈 (JVM Stacks)

虚拟机栈是线程私有的，随线程生灭。虚拟机栈描述的是线程中的方法的内存模型：

每个方法被执行的时候，都会在虚拟机栈中同步创建一个栈帧（stack frame）。

每个栈帧的包含如下的内容

- 局部变量表: 局部变量表中存储着方法里的java基本数据类型（byte/boolean/char/int/long/double/float/short）以及对象的引用（注：这里的基本数据类型指的是方法内的局部变量）
 - 操作数栈
 - 动态连接
 - 方法返回地址
- 方法被执行时入栈，执行完后出栈**

虚拟机栈可能会抛出两种异常：

- 如果线程请求的栈深度大于虚拟机所规定的栈深度，则会抛出StackOverFlowError即栈溢出
- 如果虚拟机的栈容量可以动态扩展，那么当虚拟机栈申请不到内存时会抛出OutOfMemoryError即OOM内存溢出

产生StackOverFlowError的原因是：

- 无限递归循环调用（最常见）。
- 执行了大量方法，导致线程栈空间耗尽。
- 方法内声明了海量的局部变量。

本地方法栈 (Native Method Stacks)

本地方法栈与虚拟机栈的作用是相似的,都会抛出OutOfMemoryError和StackOverflowError,都是线程私有的,主要的区别在于:

- 虚拟机栈执行的是java方法
- 本地方法栈执行的是native方法

Java堆 (Java Heap)

java堆是JVM内存中最大的一块,由所有线程共享,是由垃圾收集器管理的内存区域,主要存放对象实例,当然由于java虚拟机的发展,堆中也多了许多东西,现在主要有:

对象实例

- 类初始化生成的对象
- 基本数据类型的数组也是对象实例

字符串常量池

- 字符串常量池原本存放于方法区,从jdk7开始放置于堆中。
- 字符串常量池存储的是string对象的直接引用,而不是直接存放的对象,是一张string table

静态变量

- 静态变量是有static修饰的变量, jdk7时从方法区迁移至堆中

线程分配缓冲区 (Thread Local Allocation Buffer)

- 线程私有, 但是不影响java堆的共性
- 增加线程分配缓冲区是为了提升对象分配时的效率

java堆既可以是固定大小的,也可以是可扩展的(通过参数-Xmx和-Xms设定),如果堆无法扩展或者无法分配内存时也会报OOM。

Java 7的方法区(Method Area) / Java 8 的元空间 Metaspace

它用于存储已被虚拟机加载的类型信息、常量、静态变量、即时编译器编译后的代码缓存等。

Java方法区是Java虚拟机的一部分,是用于存储类结构信息、常量、静态变量等数据的内存区域。在Java 8及之前的版本中,方法区是在Java堆之外的一块独立的区域。但是在Java 8之后,方法区已经被元数据区所取代。

在Java 8之前,方法区是位于Java虚拟机内存布局中的非堆部分。可以通过命令行参数 `-XX:MaxPermSize` 来设置方法区的最大大小。

但是在Java 8及之后的版本中,方法区已经被元数据区所取代,元数据区是与堆共享的一部分。

元数据区是存储类的元数据、方法、常量池等信息的内存区域。

在Java 8及之后的版本中,可以通过命令行参数 `-XX:MaxMetaspaceSize` 来设置元数据区的最大大小。

A. 类型信息:

对每个加载的类型(类class、接口interface、枚举enum、注解annotation),JVM必须在方法区中存储以下类型信息:

- 这个类型的**完整有效名称**(全名=包名.类名)。
- 这个类型**直接父类**的完整有效名(对于interface或是java.lang.Object,都没有父类)。
- 这个类型的修饰符(public, abstract, final的某个子集)。

- 这个类型实现接口的一个有序列表。

B. 域 (Field) 信息:

JVM必须在方法区中保存类型的所有**域的相关信息**以及**域的声明顺序**。

域的相关信息包括:

- 域名称
- 域类型
- 域修饰符 (public, private, protected, static, final, volatile, transient的某个子集)。

C. 方法 (Method) 信息:

JVM必须在方法区中保存类型的所有**方法的相关信息**以及**方法的声明顺序**。

方法的相关信息包括:

- 方法名称
- 方法的返回类型 (或void)
- 方法参数的数量和类型 (按顺序)
- 方法的修饰符 (public, private, protected, static, final, synchronized, native, abstract的一个子集)
- 方法的字节码 (bytecodes)、操作数栈、局部变量表及大小 (abstract和native方法除外)
- 异常表 (abstract和 native方法除外)
 - 每个异常处理的开始位置、结束位置、代码处理在程序计数器中的偏移地址、被捕获的异常类的常量池索引

D. 静态变量 (non-final的)

静态变量和类关联在一起, 随着类的加载而加载, 它们成为类数据在逻辑上的一部分。

类变量被类的所有实例共享, 即使没有类实例时你也可以访问它。

补充说明: 被声明为final的静态变量的处理方法则不同, 被static和final修饰的变量也称为全局变量, 每个全局常量在编译的时候就会被赋值了。

```
public class Order {  
    public static int num = 10;  
    public static final int COUNT = 20; //全局变量,在编译的时候就会被赋值  
}
```

通过javap命令反编译后的class文件:

```

#21 = NameAndType          #5:#6          // num:I
#22 = Utf8                  com/yupeng/Order
#23 = Utf8                  java/lang/Object
{
  public static int num;
    descriptor: I
    flags: ACC_PUBLIC, ACC_STATIC

  public static final int COUNT;
    descriptor: I
    flags: ACC_PUBLIC, ACC_STATIC, ACC_FINAL
    ConstantValue: int 20

  public com.yupeng.Order();
    descriptor: ()V
    flags: ACC_PUBLIC
    Code:

```

E. 运行时常量池:

理解运行时常量池，需要了解字节码文件（ClassFile）中的常量池；

方法区内部包含运行时常量池，字节码文件内部包含了常量池。

首先简单说明常量池：

一个有效的字节码文件中除了包含类的版本信息、字段、方法以及接口等描述信息外，还包含一项信息，那就是常量池（Constant Pool Table），包括各种**字面量（数量值和字符串值）和对类型、域和方法的符号引用**。常量池可以看做是一张表，虚拟机指令根据这张常量表，找到要执行的字面量、类名、方法名、参数类型等。

然后说明运行时常量池：

- 运行时常量池（Runtime Constant pool）是方法区的一部分。
- 常量池（Constant Pool Table）是class文件的一部分，用于存放**编译期生成**的各种字面量与符号引用，这部分内容将在**类加载后存放到方法区的运行时常量池**中。
- 在加载类和接口到虚拟机后，就会创建对应的运行时常量池。
- JVM为**每个**已加载的类型（类或接口）都维护一个运行时常量池，池中的数据项像数组项一样，是通过索引访问的。
- 运行时常量池中包含多种不同的常量，包括编译期就已经明确的数值字面量，也包括到运行期解析后才能够获得的方法或者字段引用。此时不再是常量池中的符号地址了，会转换为真实地址。
 - 运行时常量池，相对于Class文件中的常量池的另一重要特征是：具备**动态性**。比如String.intern()方法会动态地向池中增加内容
- 运行时常量池类似于传统编程语言中的符号表（symbol table），但是它所包含的数据比符号表要更加丰富。
- 如果构造类或接口的运行时常量池时，所需的内存空间超过了方法区所能提供的最大值，则JVM会抛出OutOfMemoryError异常。

直接内存(Direct Memory)

jdk1.4中加入了NIO(New Input/Output)类，引入了一种基于通道(channel)与缓冲区(buffer)的新IO方式，它可以使用native函数直接分配堆外内存，然后通过存储在java堆中的DirectByteBuffer对象作为这块内存的引用进行操作，这样可以在一些场景下大大提高IO性能，避免了在java堆和native堆来回复制数据

java 的 NIO 库允许 java 程序使用直接内存。直接内存是在 java 堆外的、直接向系统申请的内存空间。通常访问直接内存的速度会优于 java 堆。因此出于性能的考虑，读写频繁的场所可能会考虑使用直接内存。

由于直接内存存在 java 堆外，因此它的大小不会直接受限于 Xmx （虚拟机参数）指定的最大堆大小，但是系统内存是有限的，java 堆和直接内存的总和依然受限于操作系统能给出的最大内存。

直接内存位于本地内存，不属于JVM内存，不受GC管理，但是也会在物理内存耗尽的时候报OOM。

注意：direct buffer不受GC影响，但是direct buffer归属的JAVA对象是在堆上且能够被GC回收的，一旦它被回收，JVM将释放direct buffer的堆外空间

直接内存(Direct Memory)的特点：

- 直接内存并非 JVMs 定义的标准 Java 运行时内存。
- JDK1.4 加入了新的 NIO 机制，目的是防止 Java 堆 和 Native 堆之间往复的数据复制带来的性能损耗，此后 NIO 可以使用 Native 的方式直接在 Native 堆分配内存。
- 直接内存区域是全局共享的内存区域。
- 直接内存区域可以进行自动内存管理(GC)，但机制并不完善。
- 本机的 Native 堆(直接内存)不受 JVM 堆内存大小限制。可能出现 OutOfMemoryError 异常。

JVM内存连环炮：什么是堆内存？

以Hotspot为例，堆内存（HEAP）主要由GC模块进行分配和管理，可分为以下部分：

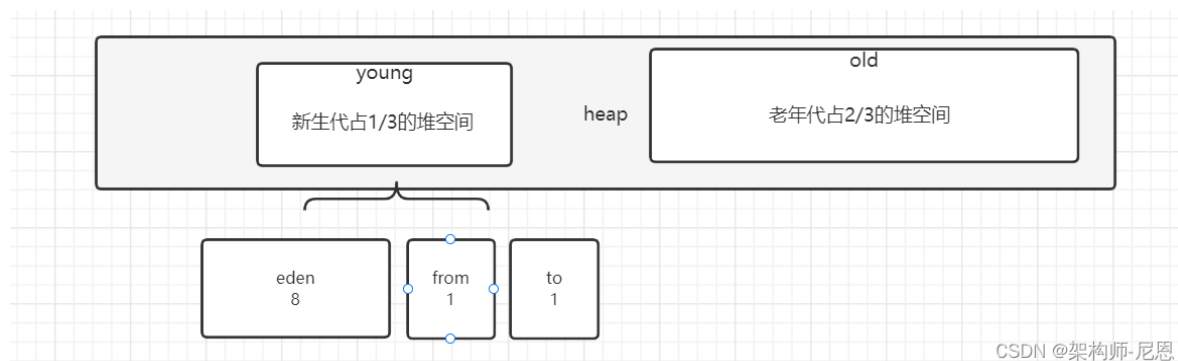
- 新生代(伊甸园区+幸存者区)
- 老年代

其中，1 伊甸园区 + 2 幸存者区，一般称为年轻代。

我们在jvm参数中只要使用-Xms，-Xmx等参数就可以设置堆的大小和最大值，理解jvm的堆还需要知道下面这个公式：

堆内内存 = 新生代+老年代

如下面的图所示：



CSDN @架构师-尼恩

在使用堆内存（on-heap memory）的时候，完全遵守JVM虚拟机的内存管理机制，采用垃圾回收器（GC）统一进行内存管理，

GC会在某些特定的时间点进行一次彻底回收，也就是Full GC，GC会对所有分配的堆内内存进行扫描，在这个过程中会对JAVA应用程序的性能造成一定影响，还可能会产生Stop The World。

常见的垃圾回收算法主要有：

- 引用计数器法 (Reference Counting)
- 标记清除法 (Mark-Sweep)
- 复制算法 (Copying)
- 标记压缩法 (Mark-Compact)
- 分代算法 (Generational Collecting)
- 分区算法 (Region)

JVM内存连环炮：什么是非堆内存？

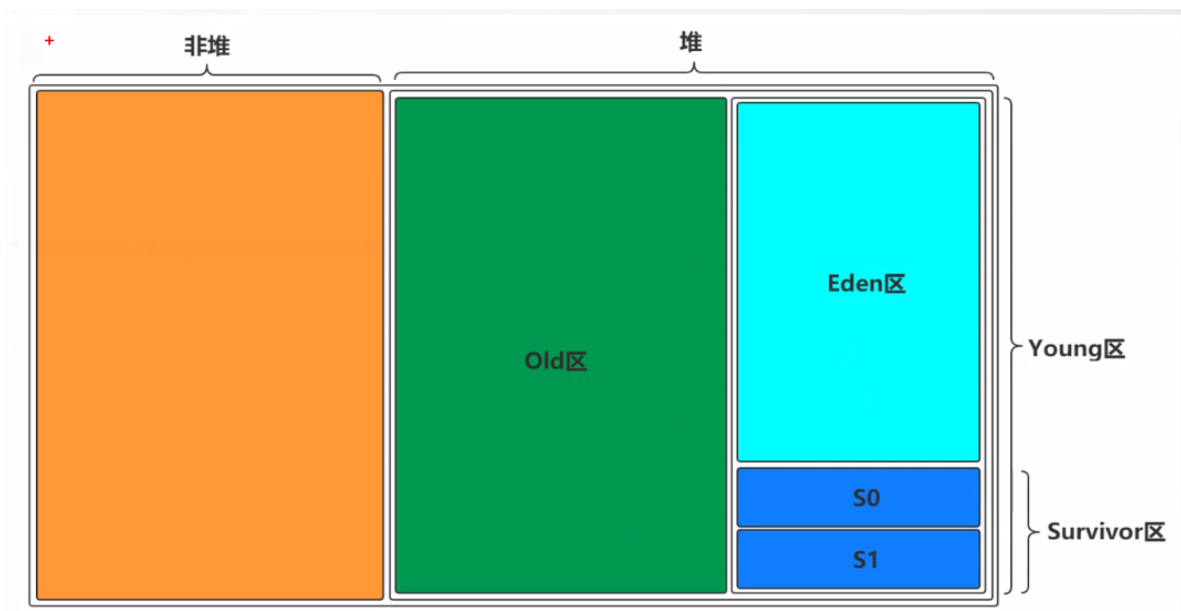
除堆内存之外，JVM的内存池还包括非堆（NON_HEAP），

非堆内存 大概包括：

- MetaSpace
- CodeCache
- Compressed Class Space

对应于JVM规范中的方法区，也就是jdk1.8的元空间

通常 Method Area又称之为Non-Heap，与Heap进行区分理解(JDK8之前的Method Area实现是Perm Space，JDK8及之后的Method Area实现叫Meta Space)



JVM内存连环炮：什么是堆外内存/直接内存（direct memory）？

堆外内存，常常又叫做直接内存。和堆内内存相对应，堆外内存就是把内存对象分配在Java虚拟机的堆以外的内存，

这些内存直接受操作系统管理（而不是虚拟机），这样做的结果就是能够在一定程度上减少垃圾回收对应用程序造成的影响。

作为JAVA开发者我们经常用java.nio.DirectByteBuffer对象进行堆外内存的管理和使用，它会在对象创建的时候就分配堆外内存。

DirectByteBuffer类是在Java Heap外分配内存，对堆外内存的申请主要是通过成员变量unsafe来操作，下面介绍构造方法。

```
DirectByteBuffer(int cap) {
    super(-1, 0, cap, cap);
    //内存是否按页分配对齐
    boolean pa = VM.isDirectMemoryPageAligned();
    //获取每页内存大小
    int ps = Bits.pageSize();
    //分配内存的大小，如果是按页对齐方式，需要再加一页内存的容量
    long size = Math.max(1L, (long)cap + (pa ? ps : 0));
    //用Bits类保存总分配内存(按页分配)的大小和实际内存的大小
    Bits.reserveMemory(size, cap);
    long base = 0;
    try {
        //在堆外内存的基地址，指定内存大小
        base = unsafe.allocateMemory(size);
    } catch (OutOfMemoryError x) {
        Bits.unreserveMemory(size, cap);
        throw x;
    }
    unsafe.setMemory(base, size, (byte) 0);
    //计算堆外内存的基地址
    if (pa && (base % ps != 0)) {
        // Round up to page boundary
        address = base + ps - (base & (ps - 1));
    } else {
        address = base;
    }
    cleaner = Cleaner.create(this, new Deallocator(base, size, cap));
    att = null;
}
```

注：在Cleaner 内部中通过一个列表，维护了一个针对每一个 directBuffer 的一个回收堆外内存的 线程对象(Runnable)，回收操作是发生在 Cleaner 的 clean() 方法中。

```
private static class Deallocator implements Runnable {
    private static Unsafe unsafe = Unsafe.getUnsafe();
    private long address;
    private long size;
    private int capacity;
    private Deallocator(long address, long size, int capacity) {
        assert (address != 0);
        this.address = address;
        this.size = size;
        this.capacity = capacity;
    }

    public void run() {
        if (address == 0) {
            // Paranoia
        }
    }
}
```



```
        return;
    }
    unsafe.freeMemory(address);
    address = 0;
    Bits.unreserveMemory(size, capacity);
}
}
```

JVM内存连环炮：介绍一下方法区和永久代的区别

方法区

方法区属于JVM规范的内容，

JVM规范中，

方法区主要用于存储类的信息、常量池、方法数据、方法代码等。

方法区 是 JVM 的规范，所有虚拟机 必须遵守的。常见的JVM 虚拟机 Hotspot、JRockit (Oracle)、J9 (IBM)

PermGen (永久代)

PermGen，就是 PermGen space，全称是 Permanent Generation space，是指内存的永久保存区域。

这块内存主要是被JVM存放Class和Meta信息的，Class 在被 Loader 时就会被放到 PermGen space 中。

绝大部分 Java 程序员应该都见过 java.lang.OutOfMemoryError: PermGen space 这个异常。

这里的 PermGen space 其实指的就是 方法区。

PermGen space 则是 HotSpot 虚拟机 基于 JVM 规范对 方法区 的一个落地实现，

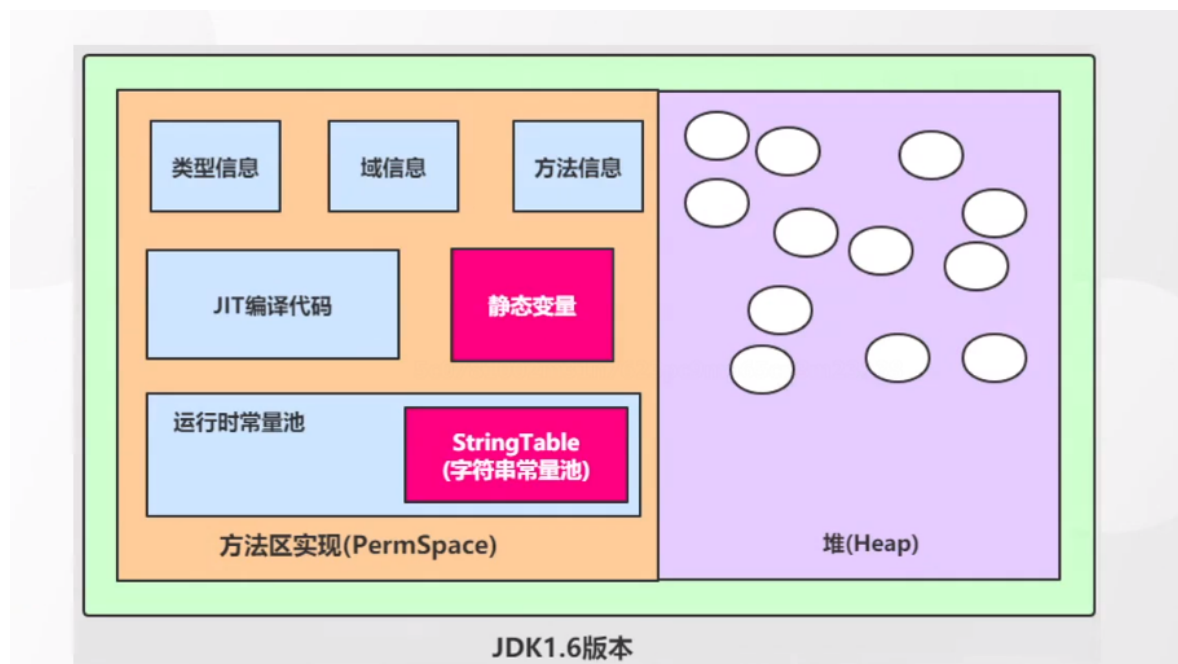
PermGen space 是 HotSpot 虚拟机有，而如 JRockit (Oracle)、J9 (IBM) 虚拟机有 方法区，但是就没有 PermGen space。

强调：只有 HotSpot 才有 PermGen space。

PermGen space 则是 HotSpot 虚拟机 基于 JVM 规范对 方法区 的一个落地实现，

PermGen space 是 JDK7及之前，HotSpot 虚拟机 对 方法区 的一个落地实现。

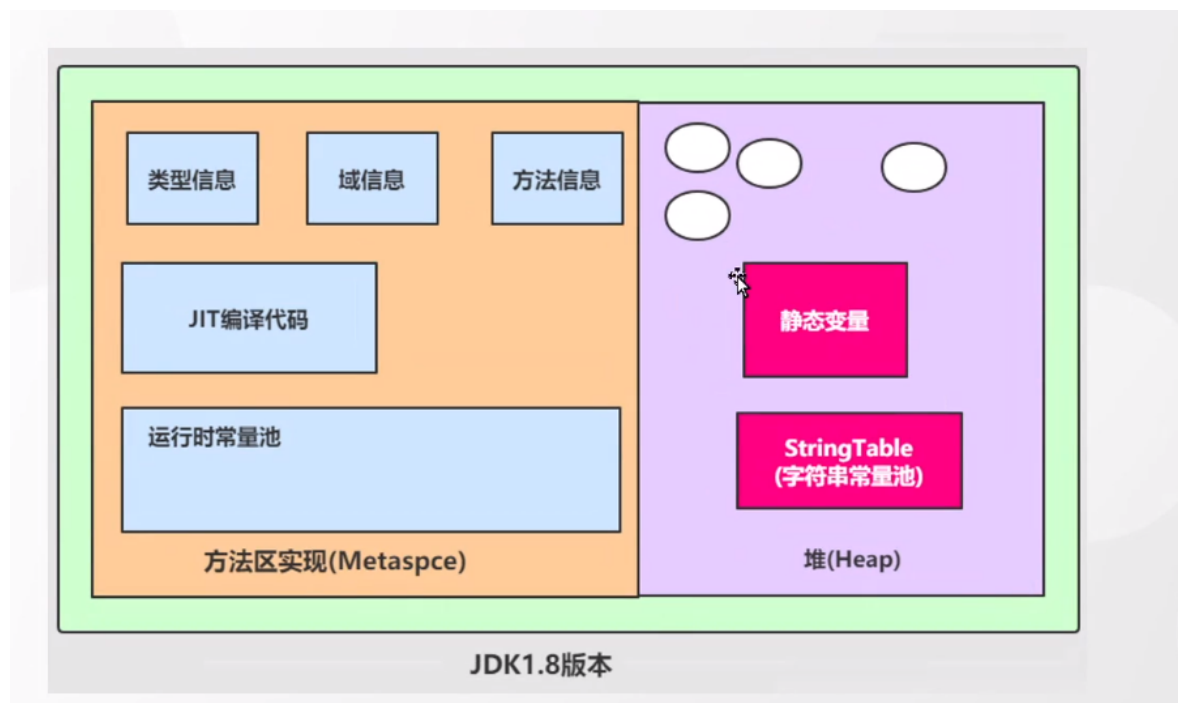
HotSpot jdk1.6 的 PermGen space 如下：



HotSpot 也有去永久代的趋势，在 JDK 1.7 中 HotSpot 已经开始了“去永久化”，把原本放在永久代的字符串常量池移出。

HotSpot 1.7 永久代主要存放常量、类信息、**静态变量**等数据，与垃圾回收关系不大，新生代和老年代是垃圾回收的主要区域。

永久代 在JDK8被移除，JDK1.8方法区 叫做 元空间：



Metaspace（元空间）是JDK8及之后，HotSpot虚拟机对方法区的新的实现。

HotSpot 方法区变迁

版本	演进细节
JDK6及之前	方法区的实现为永久代，静态变量存放在永久代中，字符串常量池（StringTable）位于运行时常量池中。
JDK7	方法区的实现为永久代，但已经逐步“去永久代”，静态变量、字符串常量池移除，保存在堆中
JDK8	方法区的实现为本地内存的元空间，字符串常量池、静态变量仍在堆中

JDK1.2 ~ JDK6

在JDK1.2 ~ JDK6 的实现中，HotSpot 使用永久代（永久代）实现方法区；

HotSpot 使用 GC 分代实现方法区带来了很大便利；

JDK7

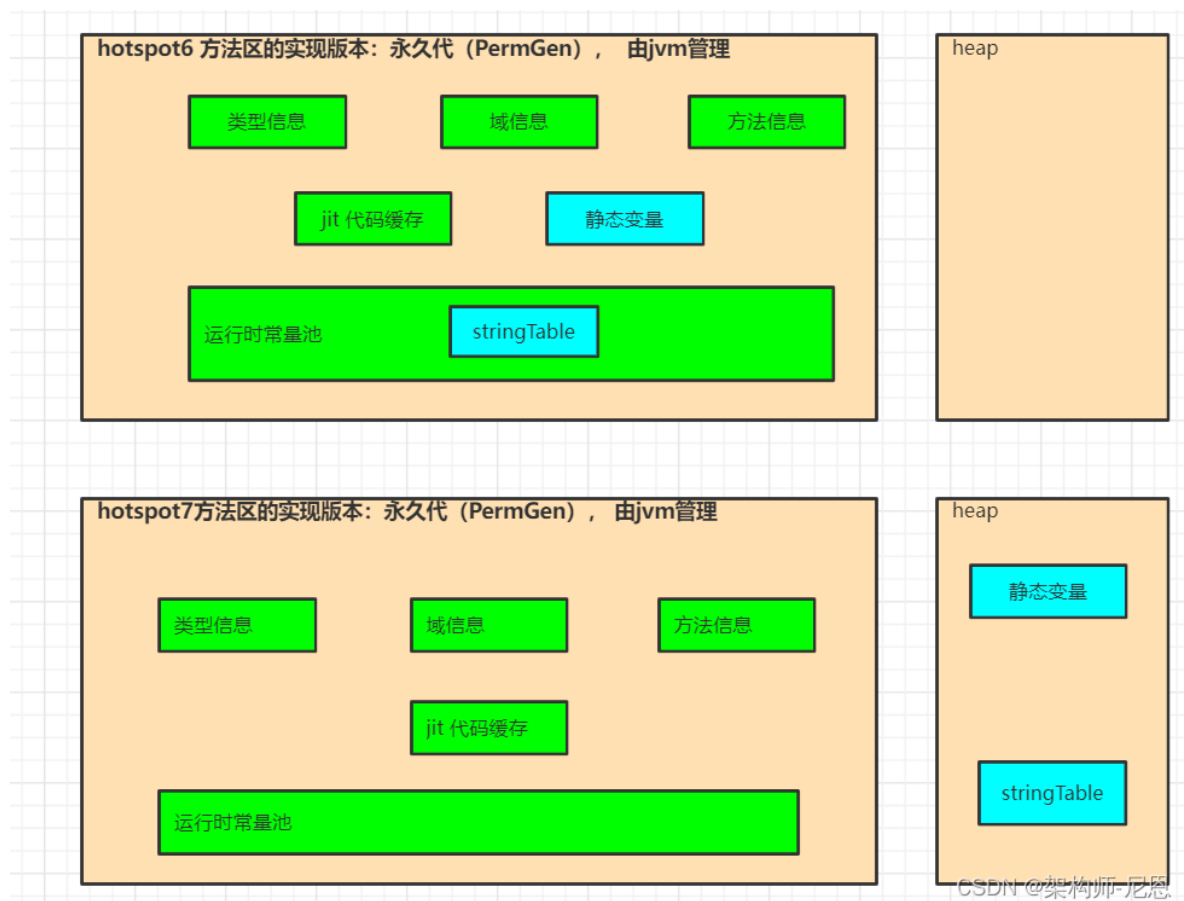
由于 GC 分代技术的影响，使之许多优秀的内存调试工具无法在 Oracle HotSpot之上运行，必须单独处理；

并且 Oracle 同时收购了 BEA 和 Sun 公司，同时拥有 JRockit 和 HotSpot，在将 JRockit 许多优秀特性移植到 HotSpot 时，由于 GC 分代技术遇到了种种困难，所以从 JDK8 开始 Oracle HotSpot 开始移除永久代。

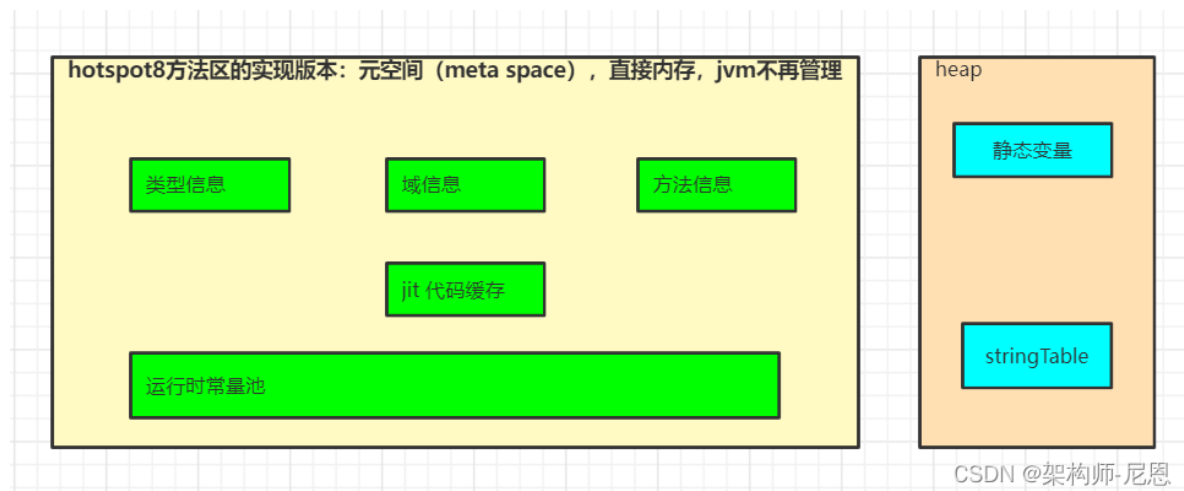
JDK7中符号表被移动到 Native Heap中， 字符串常量和类引用被移动到Java NON_HEAP中。

JDK8

在JDK6 / JDK7 中，永久代如下。



在JDK8中,永久代已完全被元空间(Metaspce)所取代。



问：为什么用元空间替换永久代？

为永久代设置最大空间大小是难以确定的。

- 在某些场景下, 如果动态加载类过多, 容易产生**Perm区的OOM**: 比如某个实际Web工程中, 因为功能点比较多, 在运行过程中, 要不断动态加载很多类, 经常出现致命错误; 而元空间和永久代之间最大的区别在于: 元空间并不在虚拟机中, 而是**使用本地内存**, 因此, 默认情况下, 元空间的最大大小仅受本地内存限制。
- 对永久代进行调优是很困难的。

问：为什么调整字符串常量池的位置？

JDK7中将字符串常量池放到了堆空间中：因为永久代的回收效率很低，在Full GC时才会触发，而Full GC在老年代的空间不足、永久代不足时才会触发，这就导致字符串常量池**回收效率不高**；

而我们开发中会有大量的字符串被创建，回收效率低会导致永久代内存不足。

将字符串常量池放到堆里，能及时回收内存。

问：几种不同的变量引用存放在哪里？

```
public class StaticObjTest {
    static class Test{
        // 静态变量
        // 一个java.lang.Class类型的对象实例引用了此变量
        static ObjectHolder staticObj = new ObjectHolder();
        // 实例变量
        ObjectHolder instanceObj = new ObjectHolder();

        void foo() {
            // 局部变量
            ObjectHolder localObj = new ObjectHolder();
            System.out.println("done");
        }
    }

    private static class ObjectHolder{
    }

    public static void main(String[] args) {
        Test test = new StaticObjTest.Test();
        test.foo();
    }
}
```

以上代码中，静态变量staticObj随着Test的类型信息存放在方法区，

实例变量instanceObj随着Test的对象实例存放在堆区，

局部变量localObj则是存放在foo()方法栈帧的局部变量表中。

三个变量引用对应的对象实体都是在堆空间。

问：什么是执行引擎？

执行引擎是 java 虚拟机的最核心组件之一，它负责执行虚拟机的字节码，有即时编译和解释执行，通常采用解释执行方式。解释执行是指解释器通过每次解释并执行一小段代码来完成.class程序的所有操作。即时编译则是将.class文件翻译成机器码在执行（比如：经常多次访问的代码可以全部编译）

垃圾回收系统是 java 虚拟机的重要组成部分，垃圾回收器可以对 栈 堆进行回收。其中，java 堆是垃圾收集器的工作重点。有三类：增量垃圾回收，分代复制垃圾回收，标记垃圾回收

和 C/C++不同，java 中所有的对象空间释放都是隐式的，也就是说，java 中没有类似 free()或者 delete()这样的函数释放指定的内存区域。对于不再使用的垃圾对象，垃圾回收系统会在后台默默工作，默默查找、标识并释放垃圾对象，完成包括 java 堆、方法区和直接内存中的全自动化管理。



转架构：通过尼恩的指导，小伙伴 **转架构** 后年薪从40W **涨到70W** @公众号 技术自由圈

问题：成员变量、局部变量、类变量分别存储在内存的什么地方？

类变量

- 类变量是用static修饰符修饰，定义在方法外的变量，随着java进程产生和销毁
- 在java8之前把静态变量存放于方法区，在java8时存放在堆中

成员变量

- 成员变量是定义在类中，但是没有static修饰符修饰的变量，随着类的实例产生和销毁，是类实例的一部分
- 由于是实例的一部分，在类初始化的时候，从运行时常量池取出直接引用或者值，与初始化的对象一起放入堆中

局部变量

- 局部变量是定义在类的方法中的变量
- 在所在方法被调用时放入虚拟机栈的栈帧中，方法执行结束后从虚拟机栈中弹出，所以存放在虚拟机栈中

问题：类常量池、运行时常量池、字符串常量池有什么关系？有什么区别？

类常量池与运行时常量池都存储在方法区，而字符串常量池在jdk7时就已经从方法区迁移到了java堆中。

在类编译过程中，会把类元信息放到方法区，类元信息的其中一部分便是类常量池，主要存放字面量和符号引用，而字面量的一部分便是文本字符，在类加载时将字面量和符号引用解析为直接引用存储在运行时常量池；

对于文本字符来说，它们会在解析时查找字符串常量池，查出这个文本字符对应的字符串对象的直接引用，将直接引用存储在运行时常量池；字符串常量池存储的是字符串对象的引用，而不是字符串本身。

使用堆外内存的优点

- 减少了垃圾回收
因为垃圾回收会暂停其他的工作。
- 加快了复制的速度
堆内在flush到远程时，会先复制到直接内存（非堆内存），然后在发送；而堆外内存相当于省略掉了这个工作。

深拷贝和浅拷贝

浅拷贝 (shallowCopy) 只是增加了一个指针指向已存在的内存地址，

深拷贝 (deepCopy) 是增加了一个指针并且申请了一个新的内存，使这个增加的指针指向这个新的内存，

使用深拷贝的情况下，释放内存的时候不会因为出现浅拷贝时释放同一个内存的错误。

浅复制：仅仅是指向被复制的内存地址，如果原地址发生改变，那么浅复制出来的对象也会相应的改变。

深复制：在计算机中开辟一块**新的内存地址**用于存放复制的对象。

说一下堆栈的区别？

物理地址

堆的物理地址分配对对象是不连续的。因此性能慢些。在GC的时候也要考虑到不连续的分配，所以有各种算法。比如，标记-消除，复制，标记-压缩，分代（即新生代使用复制算法，老年代使用标记——压缩）

栈使用的是数据结构中的栈，先进后出的原则，物理地址分配是连续的。所以性能快。

内存分别

堆因为是不连续的，所以分配的内存是在 **运行期** 确认的，因此大小不固定。一般堆大小远远大于栈。

栈是连续的，所以分配的内存大小要在 **编译期** 就确认，大小是固定的。

存放的内容

堆存放的是对象的实例和数组。因此该区更关注的是数据的存储

栈存放：局部变量，操作数栈，返回结果。该区更关注的是程序方法的执行。

PS：

1. 静态变量放在方法区
2. 静态的对象还是放在堆。

程序的可见度

堆对于整个应用程序都是共享、可见的。

栈只对于线程是可见的。所以也是线程私有。他的生命周期和线程相同。

Java 中堆和栈有什么区别？

JVM 中堆和栈属于不同的内存区域，使用目的也不同。栈常用于保存方法帧和局部变量，而对象总是在堆上分配。栈通常都比堆小，也不会多个线程之间共享，而堆被整个 JVM 的所有线程共享。

队列和栈是什么？有什么区别？

队列和栈都是被用来预存储数据的。

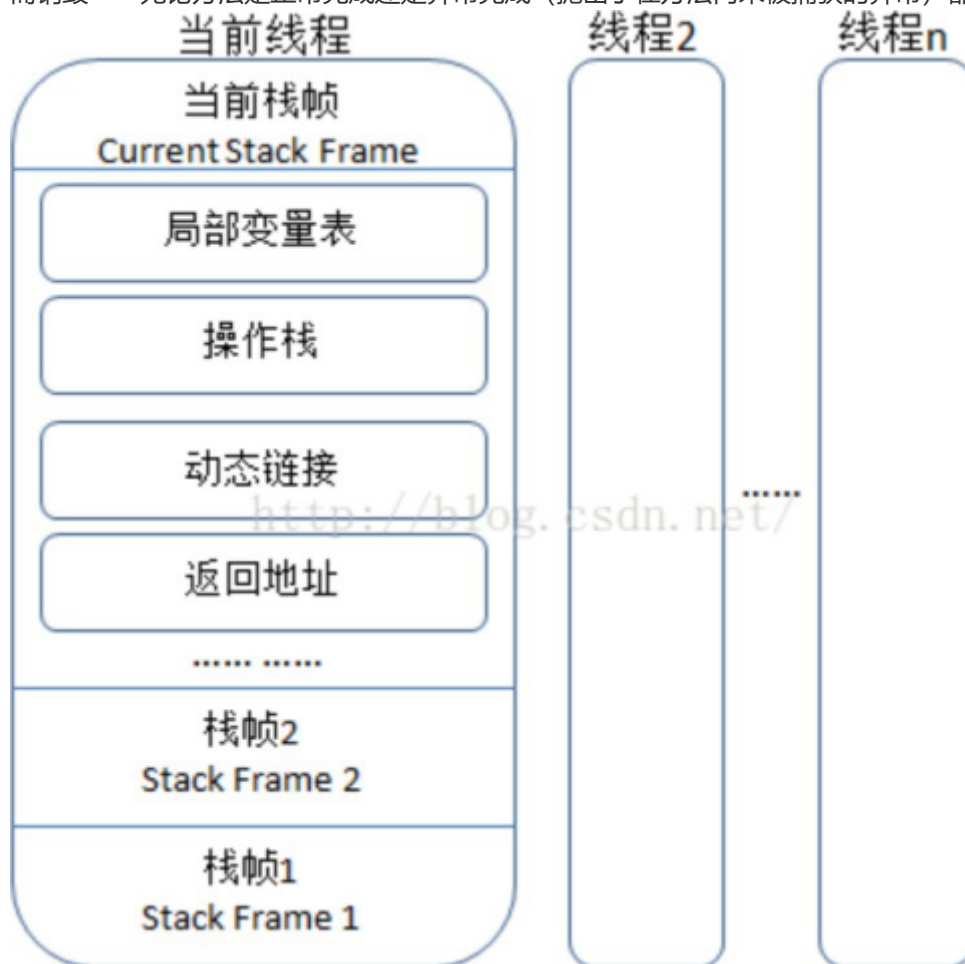
- 操作名称不同。队列的插入称为入队，队列的删除称为出队。栈的插入称为进栈，栈的删除称为出栈。

- 可操作的方式不同。队列是在队尾入队，队头出队，即两边都可操作。而栈的进栈和出栈都是在栈顶进行的，无法对栈底直接进行操作。
- 操作的方法不同。队列是先进先出（FIFO），即队列的修改是依先进先出的原则进行的。新来的成员总是加入队尾（不能从中间插入），每次离开的成员总是队列头上（不允许中途离队）。而栈为后进先出（LIFO），即每次删除（出栈）的总是当前栈中最新的元素，即最后插入（进栈）的元素，而最先插入的被放在栈的底部，要到最后才能删除。

虚拟机栈(线程私有)

是描述java方法执行的内存模型，每个方法在运行的同时都会创建一个栈帧（Stack Frame）用于存储局部变量表、操作数栈、动态链接、方法出口等信息。每一个方法从调用直至运行完成的过程，就对应着一个栈帧在虚拟机栈中入栈到出栈的过程。

栈帧（Frame）是用来存储数据和部分过程结果的数据结构，同时也被用来处理动态链接(Dynamic Linking)、方法返回值和异常分派（Dispatch Exception）。栈帧随着方法调用而创建，随着方法结束而销毁——无论方法是正常完成还是异常完成（抛出了在方法内未被捕获的异常）都算作方法结束。



程序计数器(线程私有)

一块较小的内存空间，是当前线程所执行的字节码的行号指示器，每条线程都要有一个独立的程序计数器，这类内存也称为“线程私有”的内存。

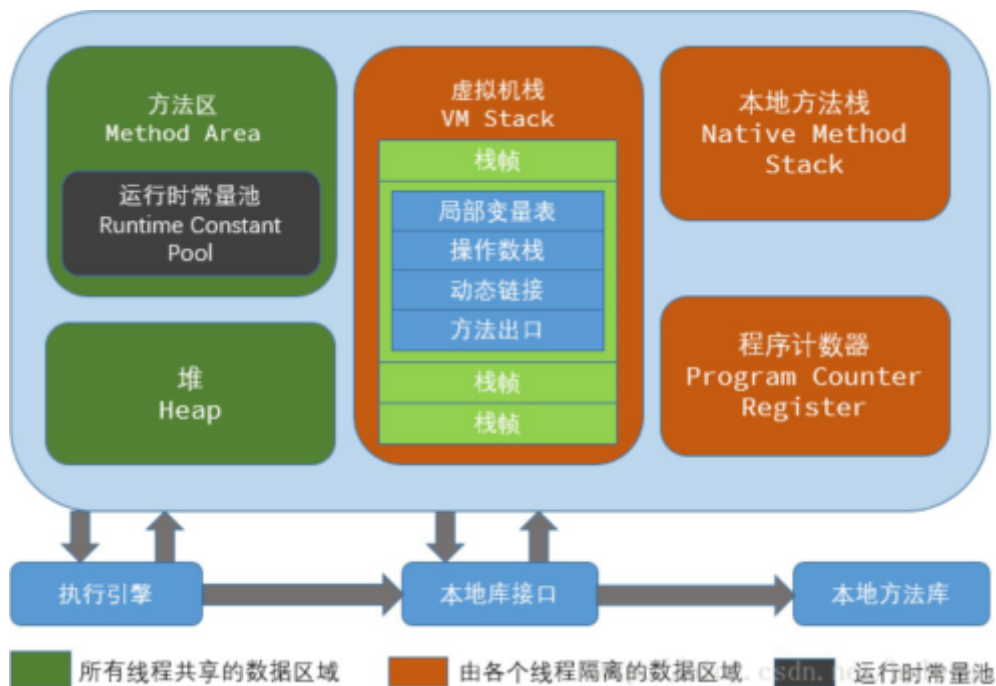
正在执行 java 方法的话，计数器记录的是虚拟机字节码指令的地址（当前指令的地址）。如果还是 Native 方法，则为空。这个内存区域是唯一一个在虚拟机中没有规定任何 OutOfMemoryError 情况的区域。



什么是直接内存？

直接内存并不是 JVM 运行时数据区的一部分，但也会被频繁的使用：

在JDK 1.4引入的NIO提供了基于Channel与Buffer的IO方式，它可以使用Native函数库直接分配堆外内存，然后使用DirectByteBuffer对象作为这块内存的引用进行操作(详见: Java I/O 扩展)，这样就避免了在Java堆和Native堆中来回复制数据，因此一些场景中可以显著提高性能。



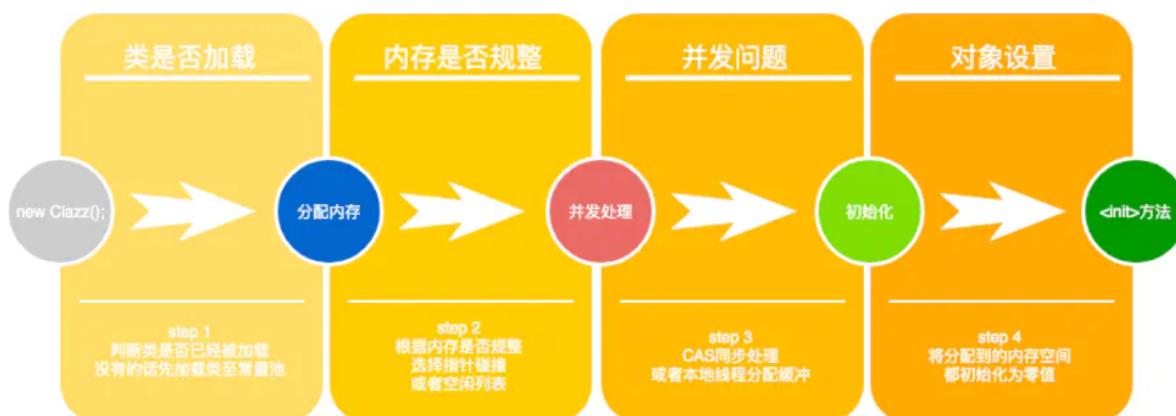
HotSpot虚拟机对象探秘

对象的创建

说到对象的创建，首先让我们看看 Java 中提供的几种对象创建方式：

Header	解释
使用new关键字	调用了构造函数
使用Class的newInstance方法	调用了构造函数
使用Constructor类的newInstance方法	调用了构造函数
使用clone方法	没有调用构造函数
使用反序列化	没有调用构造函数

下面是对象创建的主要流程:



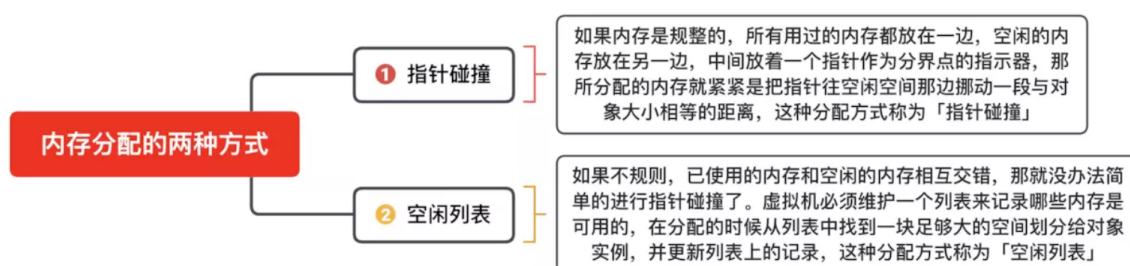
虚拟机遇到一条new指令时，先检查常量池是否已经加载相应的类，如果没有，必须先执行相应的类加载。类加载通过后，接下来分配内存。若Java堆中内存是绝对规整的，使用“指针碰撞”方式分配内存；如果不是规整的，就从空闲列表中分配，叫做“空闲列表”方式。划分内存时还需要考虑一个问题-并发，也有两种方式: CAS同步处理，或者本地线程分配缓冲(Thread Local Allocation Buffer, TLAB)。然后内存空间初始化操作，接着是做一些必要的对象设置(元信息、哈希码...)，最后执行 `<init>` 方法。

为对象分配内存

类加载完成后，接着会在Java堆中划分一块内存分配给对象。内存分配根据Java堆是否规整，有两种方式：

- 指针碰撞：如果Java堆的内存是规整的，即所有用过的内存放在一边，而空闲的放在另一边。分配内存时将位于中间的指针指示器向空闲的内存移动一段与对象大小相等的距离，这样便完成分配内存工作。
- 空闲列表：如果Java堆的内存不是规整的，则需要由虚拟机维护一个列表来记录那些内存是可用的，这样在分配的时候可以从列表中查询到足够大的内存分配给对象，并在分配后更新列表记录。

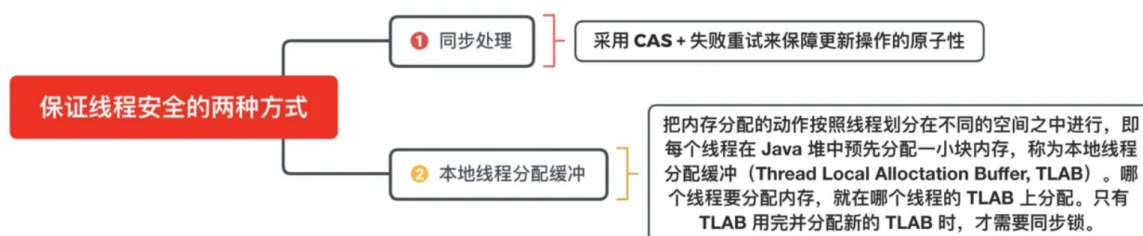
选择哪种分配方式是由Java堆是否规整来决定的，而Java堆是否规整又由所采用的垃圾收集器是否带有压缩整理功能决定。



处理并发安全问题

对象的创建在虚拟机中是一个非常频繁的行为，哪怕只是修改一个指针所指向的位置，在并发情况下也是不安全的，可能出现正在给对象 A 分配内存，指针还没来得及修改，对象 B 又同时使用了原来的指针来分配内存的情况。解决这个问题有两种方案：

- 对分配内存空间的动作进行同步处理（采用 CAS + 失败重试来保障更新操作的原子性）；
- 把内存分配的动作按照线程划分在不同的空间之中进行，即每个线程在 Java 堆中预先分配一小块内存，称为本地线程分配缓冲（Thread Local Allocation Buffer, TLAB）。哪个线程要分配内存，就在哪个线程的 TLAB 上分配。只有 TLAB 用完并分配新的 TLAB 时，才需要同步锁。通过 -XX:+/-UserTLAB 参数来设定虚拟机是否使用 TLAB。



对象结构连环炮： 给定一个具体的类，请分析对象的内存占用

```
public class MyOrder{
    private long orderId;
    private long userId;
    private byte state;
    private long createMillis;
}
```

一般来说，MyOrder 类的每个对象会占用40个字节。

对象结构连环炮： 怎么计算出来一个对象的内存占用？

计算方式为：

对象头占用12字节。

每个long类型的字段占用8字节，3个long字段占用24字节。

byte 字段占用1个字节。

以上合计 37字节，按照4字节对齐，则实际占用40个字节。

```
[InnerLockTest.showObjectStructSize]: object status:
[InnerLockTest.showObjectStructSize]: lock =
com.crazymakercircle.innerlock.InnerLockTest$MyOrder object internals:
  OFFSET  SIZE   TYPE DESCRIPTION                               VALUE
    0     4       (object header)                   05 00 00 00
(00000101 00000000 00000000 00000000) (5)
    4     4       (object header)                   00 00 00 00
(00000000 00000000 00000000 00000000) (0)
    8     4       (object header)                   e6 f2 01 f8
(11100110 11110010 00000001 11111000) (-134090010)
   12     1   byte  MyOrder.state                               0
   13     3       (alignment/padding gap)
   16     8   long  MyOrder.orderId                             0
   24     8   long  MyOrder.userId                             0
   32     8   long  MyOrder.createMillis                       0
Instance size: 40 bytes
Space losses: 3 bytes internal + 0 bytes external = 3 bytes total
```

对象结构连环炮：对象头中包含哪些部分？

对象头中一般包含两个部分：

- 标记字（mark word），占用一个机器字，也就是8字节。
- 类型指针，占用一个机器字，也就是8个字节。

这个，请阅读《Java 高并发核心编程 卷1》，非常细致

如果堆内存小于32GB，JVM默认会开启指针压缩，则只占用4个字节。

所以，前面的计算中，对象头占用12字节。

如果是数组，对象头中还会多出一个部分：

数组长度，int值，占用4字节。

对象的访问定位

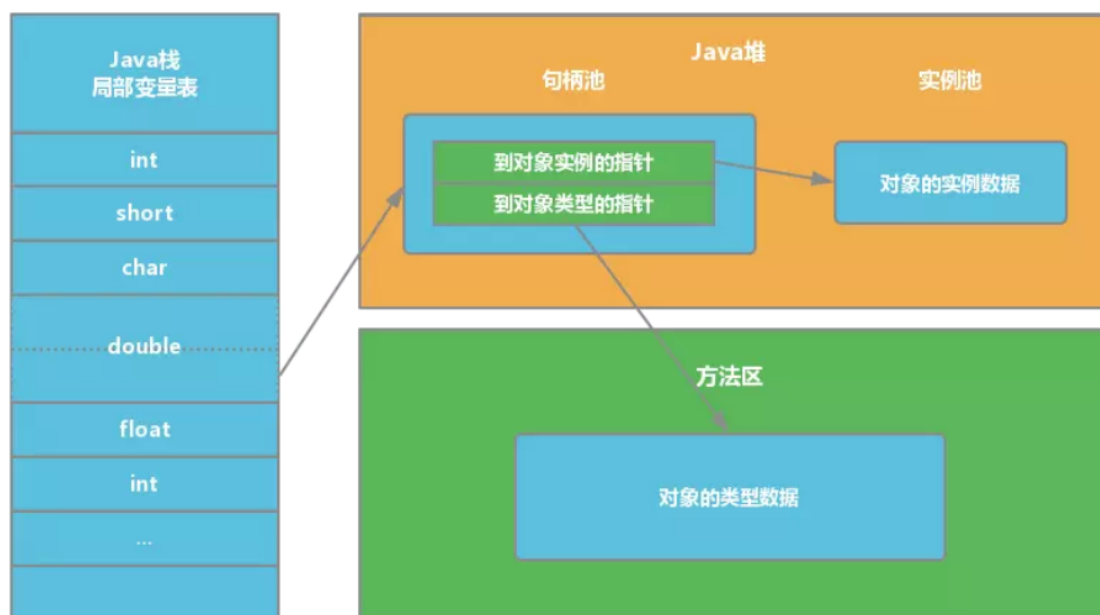
Java 程序需要通过 JVM 栈上的引用访问堆中的具体对象。对象的访问方式取决于 JVM 虚拟机的实现。目前主流的访问方式有 **句柄** 和 **直接指针** 两种方式。

指针：指向对象，代表一个对象在内存中的起始地址。

句柄：可以理解为指向指针的指针，维护着对象的指针。句柄不直接指向对象，而是指向对象的指针（句柄不发生变化，指向固定内存地址），再由对象的指针指向对象的真实内存地址。

句柄访问

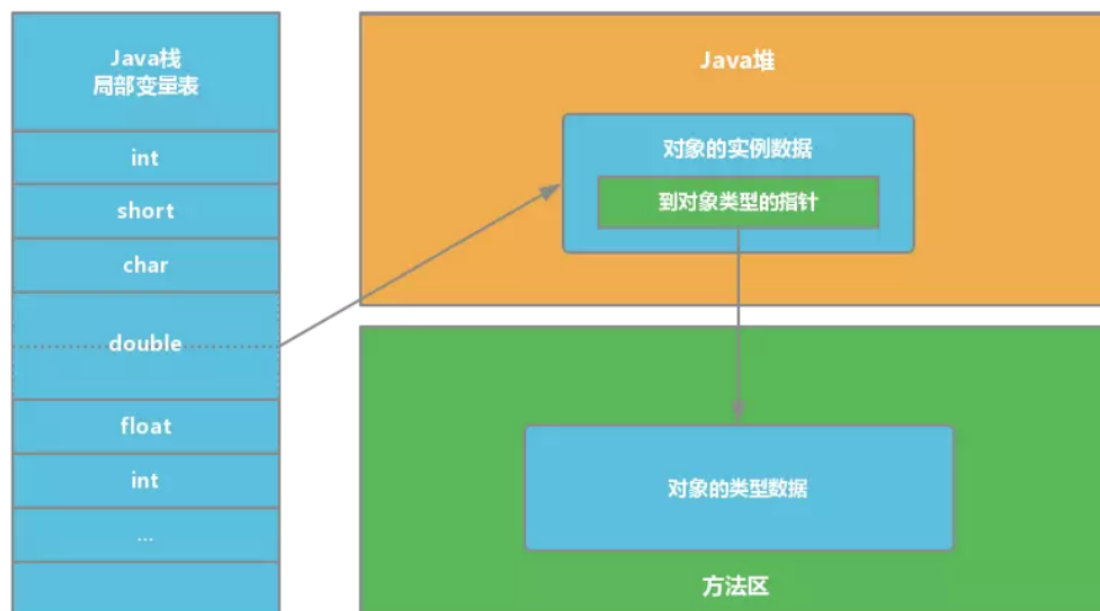
Java 堆中划分出一块内存来作为**句柄池**，引用中存储对象的**句柄地址**，而句柄中包含了**对象实例数据**与**对象类型数据**各自的具体地址信息，具体构造如下图所示：



优势：引用中存储的是**稳定**的句柄地址，在对象被移动（垃圾收集时移动对象是非常普遍的行为）时只会改变**句柄中的实例数据指针**，而**引用**本身不需要修改。

直接指针

如果使用**直接指针**访问，**引用** 中存储的直接就是**对象地址**，那么 **Java** 堆对象内部的布局中就必须考虑如何放置访问**类型数据**的相关信息。



优势：速度更快，节省了一次**指针定位**的时间开销。由于对象的访问在 **Java** 中非常频繁，因此这类开销积少成多后也是非常可观的执行成本。HotSpot 中采用的就是这种方式。

64 位 JVM 中，int 的长度是多数？

Java 中，`int` 类型变量的长度是一个固定值，与平台无关，都是 32 位。

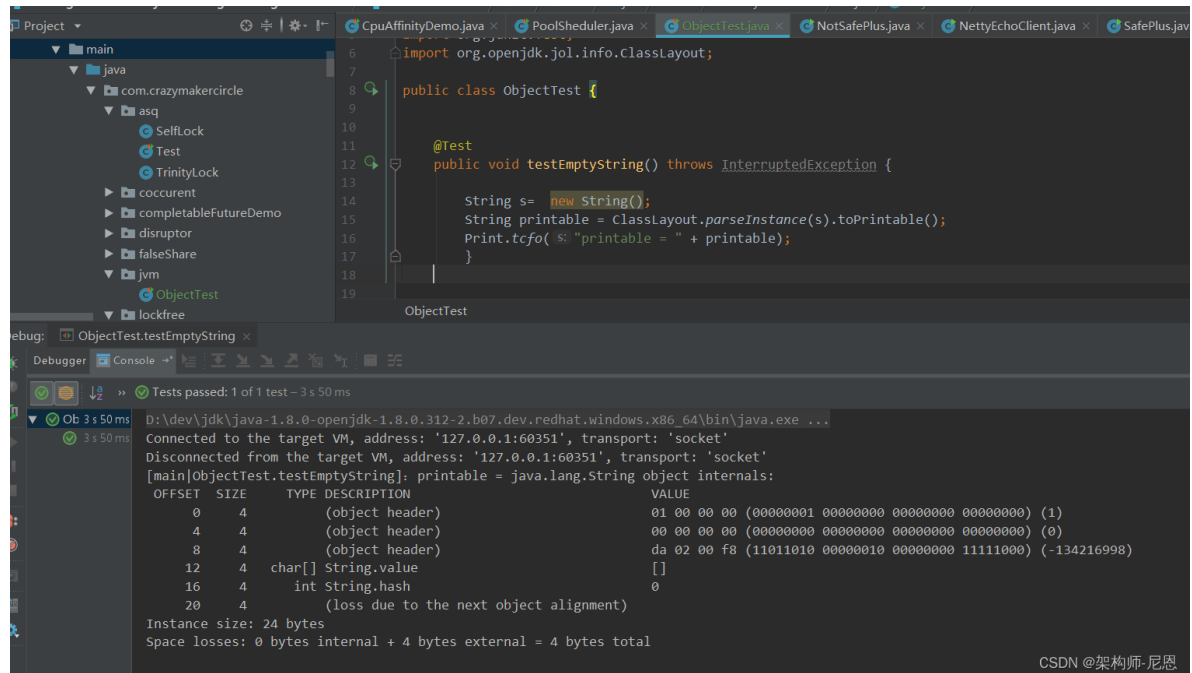
意思就是说，在 32 位 和 64 位 的 Java 虚拟机中，int 类型的长度是相同的。

32 位和 64 位的 JVM，int 类型变量的长度是多数？

32 位和 64 位的 JVM 中，int 类型变量的长度是相同的，都是 32 位或者 4 个字节。

Java中的String对象的大小

空String占用的空间



当前内存大小是在默认开启压缩指针的条件下

- 对象头 12
 - char[]数组引用 4
 - int 类型 hash数据大小 4
 - loss due to the next object alignment 对齐填充 4
- 总结：24

我们来看看String类中的成员变量。

```
/** The value is used for character storage. */
private final char value[];

/** Cache the hash code for the string */
private int hash; // Default to 0

/** use serialVersionUID from JDK 1.0.2 for interoperability */
private static final long serialVersionUID = -6849794470754667710L;
```

非空String占用的空间

内存溢出异常

Java会存在内存泄漏吗？请简单描述

内存泄漏是指不再被使用的对象或者变量一直被占据在内存中。理论上来说，Java是有GC垃圾回收机制的，也就是说，不再被使用的对象，会被GC自动回收掉，自动从内存中清除。

但是，即使这样，Java也还是存在着内存泄漏的情况，Java导致内存泄露的原因很明确：长生命周期的对象持有短生命周期对象的引用就很可能发生内存泄露，尽管短生命周期对象已经不再需要，但是因为长生命周期对象持有它的引用而导致不能被回收，这就是Java中内存泄露的发生场景。

什么情况下会发生栈内存溢出。

思路：描述栈定义，再描述为什么会溢出，再说明一下相关配置参数，OK的话可以给面试官手写是一个栈溢出的demo。

参考答案：

- 栈是线程私有的，他的生命周期与线程相同，每个方法在运行的时候都会创建一个栈帧，用来存储局部变量表，操作数栈，动态链接，方法出口等信息。局部变量表又包含基本数据类型，对象引用类型
- 如果线程请求的栈深度大于虚拟机所允许的最大深度，将抛出StackOverflowError异常，方法递归调用产生这种结果。
- 如果Java虚拟机栈可以动态扩展，并且扩展的动作已经尝试过，但是无法申请到足够的内存去完成扩展，或者在新建立线程的时候没有足够的内存去创建对应的虚拟机栈，那么Java虚拟机将抛出一个OutOfMemory 异常。(线程启动过多)
- 参数 -Xss 去调整JVM栈的大小

垃圾收集器

简述Java垃圾回收机制

在Java中，程序员是不需要显示的去释放一个对象的内存的，而是由虚拟机自行执行。在JVM中，有一个垃圾回收线程，它是低优先级的，在正常情况下是不会执行的，只有在虚拟机空闲或者当前堆内存不足时，才会触发执行，扫描那些没有被任何引用的对象，并将它们添加到要回收的集合中，进行回收。

GC是什么？为什么要GC

GC 是垃圾收集的意思 (Garbage Collection) ,内存处理是编程人员容易出现问题的地方，忘记或者错误的内存

回收会导致程序或系统的不稳定甚至崩溃，Java 提供的 GC 功能可以自动监测对象是否超过作用域从而达到自动

回收内存的目的，Java 语言没有提供释放已分配内存的显示操作方法。

垃圾回收的优点和原理。并考虑2种回收机制

java语言最显著的特点就是引入了垃圾回收机制，它使java程序员在编写程序时不再考虑内存管理的问题。

由于有这个垃圾回收机制，java中的对象不再有“作用域”的概念，只有引用的对象才有“作用域”。

垃圾回收机制有效的防止了内存泄露，可以有效的使用可使用的内存。

垃圾回收器通常作为一个单独的低级别的线程运行，在不可预知的情况下对内存堆中已经死亡的或很长时间没有用过的对象进行清除和回收。

程序员不能实时的对某个对象或所有对象调用垃圾回收器进行垃圾回收。

垃圾回收有分代复制垃圾回收、标记垃圾回收、增量垃圾回收。

垃圾回收器的基本原理是什么？垃圾回收器可以马上回收内存吗？有什么办法主动通知虚拟机进行垃圾回收？

对于GC来说，当程序员创建对象时，GC就开始监控这个对象的地址、大小以及使用情况。

通常，GC采用有向图的方式记录和管理堆(heap)中的所有对象。通过这种方式确定哪些对象是“可达的”，哪些对象是“不可达的”。当GC确定一些对象为“不可达”时，GC就有责任回收这些内存空间。

可以。程序员可以手动执行System.gc()，通知GC运行，但是Java语言规范并不保证GC一定会执行。

你能保证 GC 执行吗？

不能，虽然你可以调用 System.gc() 或者 Runtime.gc()，但是没有办法保证 GC的执行。

Java 中都有哪些引用类型？

- 强引用：发生 gc 的时候不会被回收。
- 软引用：有用但不是必须的对象，在发生内存溢出之前会被回收。
- 弱引用：有用但不是必须的对象，在下次GC时会被回收。
- 虚引用（幽灵引用/幻影引用）：无法通过虚引用获得对象，用 PhantomReference 实现虚引用，虚引用的用途是在 gc 时返回一个通知。

介绍一下强引用、软引用、弱引用、虚引用的区别？

思路：先说一下四种引用的定义，可以结合代码讲一下，也可以扩展谈到ThreadLocalMap里弱引用用处。

参考答案：

1) 强引用

我们平时new了一个对象就是强引用，例如 Object obj = new Object();即使在内存不足的情况下，JVM宁愿抛出OutOfMemory错误也不会回收这种对象。

2) 软引用

如果一个对象只具有软引用，则内存空间足够，垃圾回收器就不会回收它；如果内存空间不足了，就会回收这些对象的内存。

```
SoftReference<String> softRef=new SoftReference<String>(str); // 软引用
```

用处： 软引用在实际中有重要的应用，例如浏览器的后退按钮。按后退时，这个后退时显示的网页内容是重新进行请求还是从缓存中取出呢？这就要看具体的实现策略了。

(1) 如果一个网页在浏览结束时就进行内容的回收，则按后退查看前面浏览过的页面时，需要重新构建

(2) 如果将浏览过的网页存储到内存中会造成内存的大量浪费，甚至会造成内存溢出

如下代码：

```
Browser prev = new Browser(); // 获取页面进行浏览
SoftReference sr = new SoftReference(prev); // 浏览完毕后置为软引用
if(sr.get()!=null){
    rev = (Browser) sr.get(); // 还没有被回收器回收，直接获取
}else{
    prev = new Browser(); // 由于内存吃紧，所以对软引用的对象回收了
    sr = new SoftReference(prev); // 重新构建
}
```

3) 弱引用

具有弱引用的对象拥有更短暂的生命周期。在垃圾回收器线程扫描它所管辖的内存区域的过程中，一旦发现了只具有弱引用的对象，不管当前内存空间足够与否，都会回收它的内存。

```
String str=new String("abc");
WeakReference<String> abcWeakRef = new WeakReference<String>(str);
str=null;
等价于
str = null;
System.gc();
```

4) 虚引用

如果一个对象仅持有虚引用，那么它就和没有任何引用一样，在任何时候都可能被垃圾回收器回收。虚引用主要用来跟踪对象被垃圾回收器回收的活动。

怎么判断对象是否可以被回收？

垃圾收集器在做垃圾回收的时候，首先需要判定的就是哪些内存是需要被回收的，哪些对象是「存活」的，是不可以被回收的；哪些对象已经「死掉」了，需要被回收。

一般有两种方法来判断：

- 引用计数器法：为每个对象创建一个引用计数，有对象引用时计数器 +1，引用被释放时计数 -1，当计数器为 0 时就可以被回收。它有一个缺点不能解决循环引用的问题；
- 可达性分析算法：从 GC Roots 开始向下搜索，搜索所走过的路径称为引用链。当一个对象到 GC Roots 没有任何引用链相连时，则证明此对象是可以被回收的。

在Java中，对象什么时候可以被垃圾回收

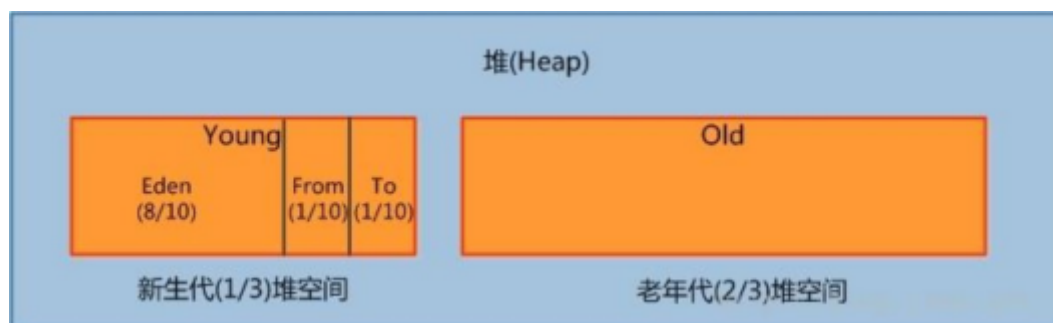
当对象对当前使用这个对象的应用程序变得不可触及的时候，这个对象就可以被回收了。

垃圾回收不会发生在永久代，如果永久代满了或者是超过了临界值，会触发完全垃圾回收(Full GC)。如果你仔细查看垃圾收集器的输出信息，就会发现永久代也是被回收的。这就是为什么正确的永久代大小对避免Full GC是非常重要的原因。

JVM 运行时堆内存如何分代？

Java 堆从 GC 的角度还可以细分为: 新生代(Eden 区、 From Survivor 区和 To Survivor 区)和老年代。

参考图1：



参考图2：



从图中可以看出：堆大小 = 新生代 + 老年代。其中，堆的大小可以通过参数 -Xms、-Xmx 来指定。

默认的，新生代 (Young) 与老年代 (Old) 的比例的值为 1:2 (该值可以通过参数 -XX:NewRatio 来指定)，

即：新生代 (Young) = 1/3 的堆空间大小。老年代 (Old) = 2/3 的堆空间大小。

其中，新生代 (Young) 被细分为 Eden 和两个 Survivor 区域，这两个 Survivor 区域分别被命名为 from 和 to，以示区分

默认的，Eden: from : to = 8 : 1 : 1 (可以通过参数-XX:SurvivorRatio 来设定)，即：Eden = 8/10 的新生代空间大小，from = to = 1/10 的新生代空间大小。

JVM 每次只会使用 Eden 和其中的一块 Survivor 区域来为对象服务，所以无论什么时候，总是有一块 Survivor 区域是空闲着的。

因此，新生代实际可用的内存空间为 9/10 (即90%)的新生代空间。

新生代

是用来存放新生的对象。一般占据堆的 1/3 空间。由于频繁创建对象，所以新生代会频繁触发MinorGC进行垃圾回收。新生代又分为 Eden区、ServivorFrom、ServivorTo 三个区。

Eden 区

Java 新对象的出生地（如果新创建的对象占用内存很大，则直接分配到老年代）。当 Eden 区内存不够的时候就会触发 MinorGC，对新生代区进行一次垃圾回收。

Servivor from 区

上一次 GC 的幸存者，作为这一次 GC 的被扫描者。

Servivor to 区

保留了一次 MinorGC 过程中的幸存者。

MinorGC 的过程（复制->清空->互换）

MinorGC 采用复制算法。

1. eden、servicorFrom 复制到 ServicorTo，年龄+1

首先，把 Eden 和 ServicorFrom 区域中存活的对象复制到 ServicorTo 区域（如果有对象的年龄以及达到了老年的标准，则赋值到老年代区），同时把这些对象的年龄+1（如果 ServicorTo 不够位置了就放到老年区）；

2. 清空 eden、servicorFrom

然后，清空 Eden 和 ServicorFrom 中的对象；

3. ServicorTo 和 ServicorFrom 互换

最后，ServicorTo 和 ServicorFrom 互换，原 ServicorTo 成为下一次 GC 时的 ServicorFrom区。

老年代

主要存放应用程序中生命周期长的内存对象。

老年代的对象比较稳定，所以 MajorGC（常常称之为 FULL GC）不会频繁执行。在进行 FULL GC前一般都先进行了一次 MinorGC，使得有新生代的对象晋身入老年代，导致空间不够用时才触发。当无法找到足够大的连续空间分配给新创建的较大对象时也会提前触发一次 MajorGC 进行垃圾回收腾出空间。

FULL GC 采用标记清除算法：首先扫描一次所有老年代，标记出存活的对象，然后回收没有标记的对象。ajorGC 的耗时比较长，因为要扫描再回收。FULL GC 会产生内存碎片，为了减少内存损耗，我们一般需要进行合并或者标记出来方便下次直接分配。当老年代也满了装不下的时候，就会抛出 OOM（Out of Memory）异常。

永久代（或者 jdk1.8元空间）

指内存的永久保存区域，主要存放 Class 和 Meta（元数据）的信息,Class 在被加载的时候被放入永久区域，它和存放实例的区域不同,GC 不会在主程序运行期对永久区域进行清理。所以这也导致了永久代的区域会随着加载的 Class 的增多而胀满，最终抛出 OOM 异常。

JVM内存为什么要分成新生代，老年代，永久代。新生代中为什么要分为Eden和Survivor。

思路：先讲一下JAVA堆，新生代的划分，再谈谈它们之间的转化，相互之间一些参数的配置（如：-XX:NewRatio，-XX:SurvivorRatio等），再解释为什么要这样划分，最好加一点自己的理解。

参考答案：

这样划分的目的是为了使 JVM 能够更好的管理堆内存中的对象，包括内存的分配以及回收。

1) 共享内存区分

- 共享内存区 = 永久代 + 堆

- 永久代 = 方法区 + 其他
- Java堆 = 老年代 + 新生代
- 新生代 = Eden + S0 + S1

2) 一些参数的配置

- 默认的, 新生代 (Young) 与老年代 (Old) 的比例的值为 1:2 , 可以通过参数 -XX:NewRatio 配置。
- 默认的, Eden : from : to = 8 : 1 : 1 (可以通过参数 -XX:SurvivorRatio 来设定)
- Survivor区中的对象被复制次数为15(对应虚拟机参数 -XX:+MaxTenuringThreshold)

3)为什么要分为Eden和Survivor?为什么要设置两个Survivor区?

- 如果没有Survivor, Eden区每进行一次Minor GC, 存活的对象就会被送到老年代。老年代很快被填满, 触发Major GC.老年代的内存空间远大于新生代, 进行一次Full GC消耗的时间比Minor GC长得多,所以需要分为Eden和Survivor。
- Survivor的存在意义, 就是减少被送到老年代的对象, 进而减少Full GC的发生, Survivor的预筛选保证, 只有经历16次Minor GC还能在新生代中存活的对象, 才会被送到老年代。
- 设置两个Survivor区最大的好处就是解决了碎片化, 刚刚新建的对象在Eden中, 经历一次Minor GC, Eden中的存活对象就会被移动到第一块survivor space S0, Eden被清空; 等Eden区再满了, 就再触发一次Minor GC, Eden和S0中的存活对象又会被复制送入第二块survivor space S1 (这个过程非常重要, 因为这种复制算法保证了S1中来自S0和Eden两部分的存活对象占用连续的内存空间, 避免了碎片化的发生)

JVM中一次完整的GC流程是怎样的, 对象如何晋升到老年代

思路: 先描述一下Java堆内存划分, 再解释Minor GC, Major GC, full GC, 描述它们之间转化流程。

我的答案:

- Java堆 = 老年代 + 新生代
- 新生代 = Eden + S0 + S1
- 当 Eden 区的空间满了, Java虚拟机会触发一次 Minor GC, 以收集新生代的垃圾, 存活下来的对象, 则会转移到 Survivor区。
- **大对象** (需要大量连续内存空间的Java对象, 如那种很长的字符串) **直接进入老年态;**
- 如果对象在Eden出生, 并经过第一次Minor GC后仍然存活, 并且被Survivor容纳的话, 年龄设为1, 每熬过一次Minor GC, 年龄+1, **若年龄超过一定限制 (15) , 则被晋升到老年态。即长期存活的对象进入老年态。**
- 老年代满了而**无法容纳更多的对象**, Minor GC 之后通常就会进行Full GC, Full GC 清理整个内存堆 - **包括年轻代和年老代。**
- Major GC **发生在老年代的GC, 清理老年区**, 经常会伴随至少一次Minor GC, **比Minor GC慢10倍以上。**

聊聊: 什么是分代, 分代的必要性

Java 虚拟机根据对象存活的周期不同, 把堆内存划分为几块, 一般分为新生代、老年代和永久代 (对 HotSpot 虚拟机而言) , 这就是

JVM 的内存分代策略。

JDK 1.7 之前, Java 虚拟机将堆内存划分为新生代、老年代和永久代 (或者元空间) ,

永久代是 HotSpot 虚拟机特有的概念 (JDK1.8 之后为 metaspace 元空间替代永久代) , 它采用永久代的方式来实现方法区, 其他的虚拟机实现没有这一概念,

而且 HotSpot 也有去永久代的趋势，在 JDK 1.7 中 HotSpot 已经开始了“去永久化”，把原本放在永久代的字符串常量池移出。永久代主要存放常量、类信息、静态变量等数据，与垃圾回收关系不大，新生代和老年代是垃圾回收的主要区域。

堆内存是虚拟机管理的内存中最大的一块，也是垃圾回收最频繁的一块区域，我们程序所有的对象实例都存放在堆内存中。

给堆内存分代 是为了**提高对象内存分配和垃圾回收的效率**。

试想一下，如果堆内存没有区域划分，所有的新创建的对象和生命周期很长的对象放在一起，随着程序的执行，堆内存需要频繁进行垃圾收集，而每次回收都要遍历所有的对象，遍历这些对象所花费的时间代价是巨大的，会严重影响我们的 GC 效率。

有了内存分代，情况就不同了，新创建的对象会在新生代中分配内存，经过多次回收仍然存活下来的对象存放在老年代中，

静态属性、类信息等存放在永久代（或者元空间）中，

新生代中的对象存活时间短，只需要在新生代区域中频繁进行 GC，老年代中对象生命周期长，内存回收的频率相对较低，不需要频繁进行回收，

永久代（或者元空间）中回收效果太差，一般不进行垃圾回收，还可以根据不同年代的特点采用合适的垃圾收集算法。

分代收集大大提升了收集效率，这些都是内存分代带来的好处。

JVM中的永久代中会发生垃圾回收吗

垃圾回收不会发生在永久代，如果永久代满了或者是超过了临界值，会触发完全垃圾回收(Full GC)。

如果你仔细查看垃圾收集器的输出信息，就会发现永久代也是被回收的。

这就是为什么正确的永久代大小对避免Full GC是非常重要的原因。

JAVA8 与元数据

在 Java8 中，永久代已经被移除，被一个称为“元数据区”（元空间）的区域所取代。

元空间的本质和永久代类似，元空间与永久代之间最大的区别在于：元空间并不在虚拟机中，而是使用本地内存。

因此，默认情况下，元空间的大小仅受本地内存限制。

类的元数据放入 native memory, 字符串池和类的静态变量放入 java 堆中，这样可以加载多少类的元数据就不再由 MaxPermSize 控制, 而由系统的实际可用空间来控制。

如何判断对象可以被回收？

判断对象是否存活一般有两种方式：

- 引用计数：

每个对象有一个引用计数属性，新增一个引用时计数加1，引用释放时计数减1，计数为0时可以回收。此方法简单，无法解决对象相互循环引用的问题。

- 可达性分析（Reachability Analysis）：

从GC Roots开始向下搜索，搜索所走过的路径称为引用链。当一个对象到GC Roots没有任何引用链相连时，则证明此对象是不可用的，不可达对象。

引用计数法

在Java中，引用和对象是有关联的。如果要操作对象则必须用引用进行。因此，很显然一个简单的办法是通过引用计数来判断一个对象是否可以回收。简单说，即一个对象如果没有任何与之关联的引用，即他们的引用计数都不为0，则说明对象不太可能再被用到，那么这个对象就是可回收对象。

可达性分析

为了解决引用计数法的循环引用问题，Java使用了可达性分析的方法。通过一系列的“GC roots”对象作为起点搜索。如果在“GC roots”和一个对象之间没有可达路径，则称该对象是不可达的。要注意的是，不可达对象不等价于可回收对象，不可达对象变为可回收对象至少要经过两次标记过程。两次标记后仍然是可回收对象，则将面临回收。

Minor GC与Full GC分别在什么时候发生？

新生代内存不够用时候发生MGC也叫YGC，JVM内存不够的时候发生FGC

垃圾收集算法有哪些类型？

GC最基础的算法有三类：标记-清除算法、复制算法、标记-压缩算法，我们常用的垃圾回收器一般都采用分代收集算法。

标记-清除算法，“标记-清除”（Mark-Sweep）算法，如它的名字一样，算法分为“标记”和“清除”两个阶段：首先标记出所有需要回收的对象，在标记完成后统一回收掉所有被标记的对象。

复制算法，“复制”（Copying）的收集算法，它将可用内存按容量划分为大小相等的两块，每次只使用其中的一块。当这一块的内存用完了，就将还存活着的对象复制到另外一块上面，然后再把已使用过的内存空间一次清理掉。

标记-压缩算法，标记过程仍然与“标记-清除”算法一样，但后续步骤不是直接对可回收对象进行清理，而是让所有存活的对象都向一端移动，然后直接清理掉端边界以外的内存

分代收集算法，“分代收集”（Generational Collection）算法，把Java堆分为新生代和老年代，这样就可以根据各个年代的特点采用最适当的收集算法



转架构：通过尼恩的指导，小伙伴 **转架构** 后年薪从40W 涨到70W @公众号 技术自由圈

说一下 JVM 有哪些垃圾回收算法？

- 标记-清除算法：标记无用对象，然后进行清除回收。缺点：效率不高，无法清除垃圾碎片。
- 复制算法：按照容量划分二个大小相等的内存区域，当一块用完的时候将活着的对象复制到另一块上，然后再把已使用的内存空间一次清理掉。缺点：内存使用率不高，只有原来的一半。
- 标记-整理算法：标记无用对象，让所有存活的对象都向一端移动，然后直接清除掉端边界以外的内存。

- 分代算法：根据对象存活周期的不同将内存划分为几块，一般是新生代和老年代，新生代基本采用复制算法，老年代采用标记整理算法。

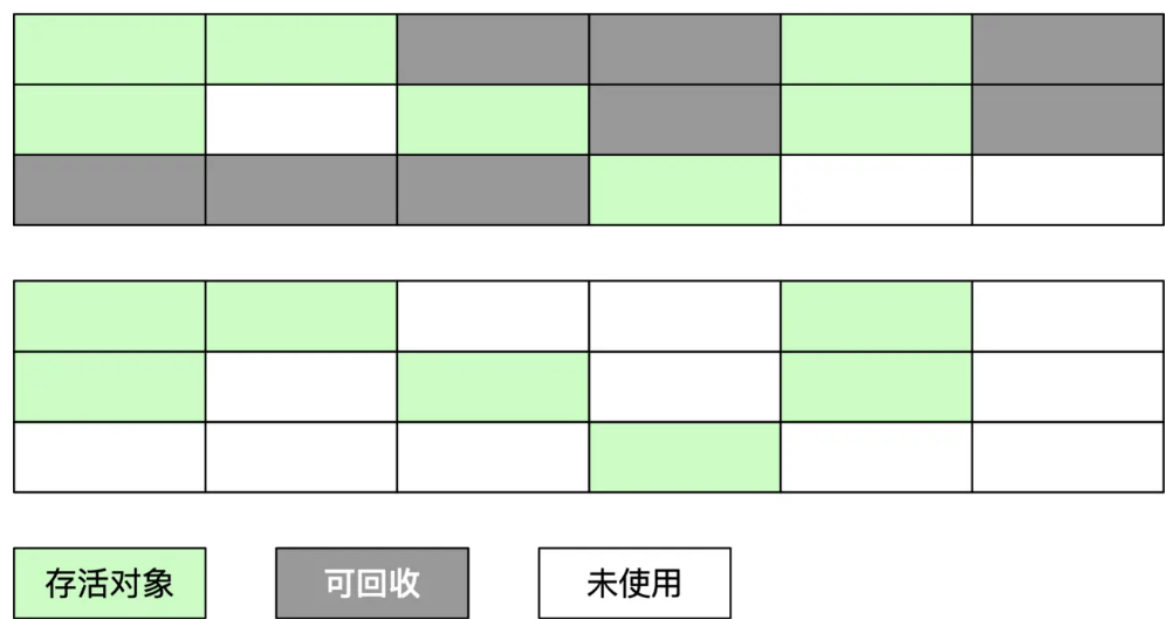
(1) 标记-清除算法

该算法分为“标记”和“清除”阶段：首先标记出所有不需要回收的对象，在标记完成后统一回收掉所有没有被标记的对象。**适用场合**：存活对象较多的情况、适用于老年代。

标记-清除算法（Mark-Sweep）是一种常见的基础垃圾收集算法，它将垃圾收集分为两个阶段：

- 标记阶段：标记出可以回收的对象。
- 清除阶段：回收被标记的对象所占用的空间。

标记-清除算法的执行的过程如下图所示



优点：实现简单，不需要对象进行移动。

缺点：

1.空间问题，易产生内存碎片，当为一个对象分配空间时可能会提前触发垃圾回收（例如，对象的大小大于空闲表中的每一块儿大小但是小于其中两块儿的和）。

2.效率问题，扫描了整个空间两次（第一次：标记存活对象；第二次：清除没有标记的对象）。标记、清除过程效率低，产生大量不连续的内存碎片，提高了垃圾回收的频率。

总之：

标记-清除算法之所以是基础的，是因为后面讲到的垃圾收集算法都是在此算法的基础上进行改进的。

(2) 标记-复制算法

为了解决标记-清除算法的效率不高的问题，产生了标记-复制算法。

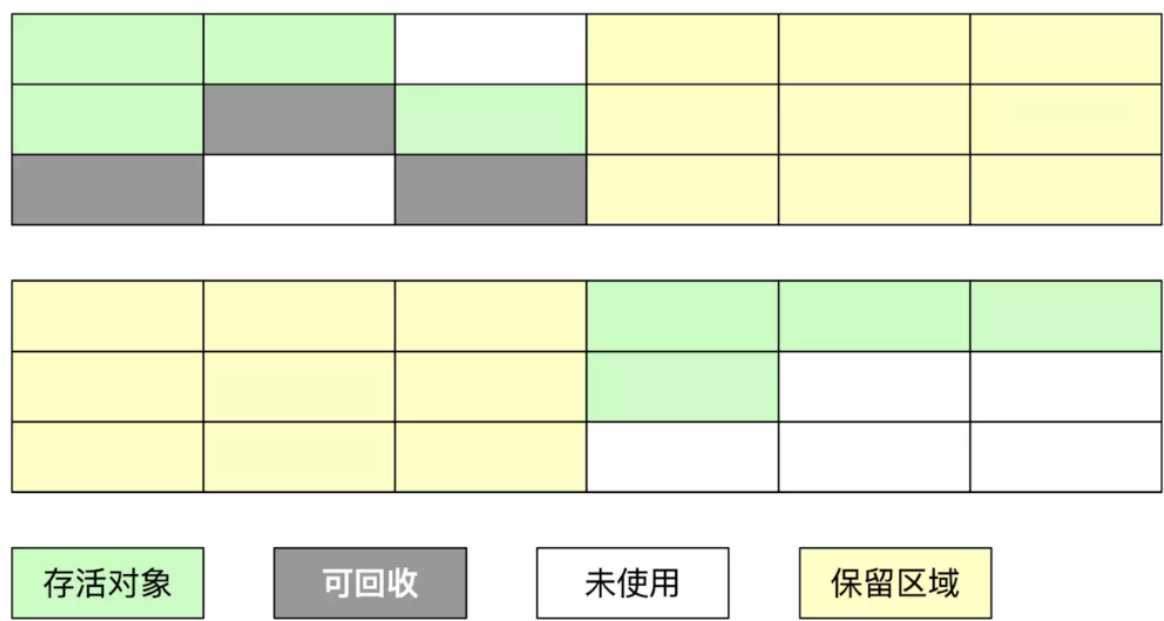
它把内存空间划为两个相等的区域，每次只使用其中一个区域。

垃圾收集时，遍历当前使用的区域，把存活对象复制到另外一个区域中，最后将当前使用的区域的可回收的对象进行回收。

优点：按顺序分配内存即可，实现简单、运行高效，不用考虑内存碎片。

缺点：可用的内存大小缩小为原来的一半，对象存活率高时会频繁进行复制。

复制算法的执行过程如下图所示



(3) 标记-整理算法

在新生代中可以使用复制算法，但是在老年代就不能选择复制算法了，因为老年代的对象存活率会较高，这样会有较多的复制操作，导致效率变低。

标记-清除算法可以应用在老年代中，但是它效率不高，在内存回收后容易产生大量内存碎片。因此就出现了一种标记-整理算法（Mark-Compact）算法。

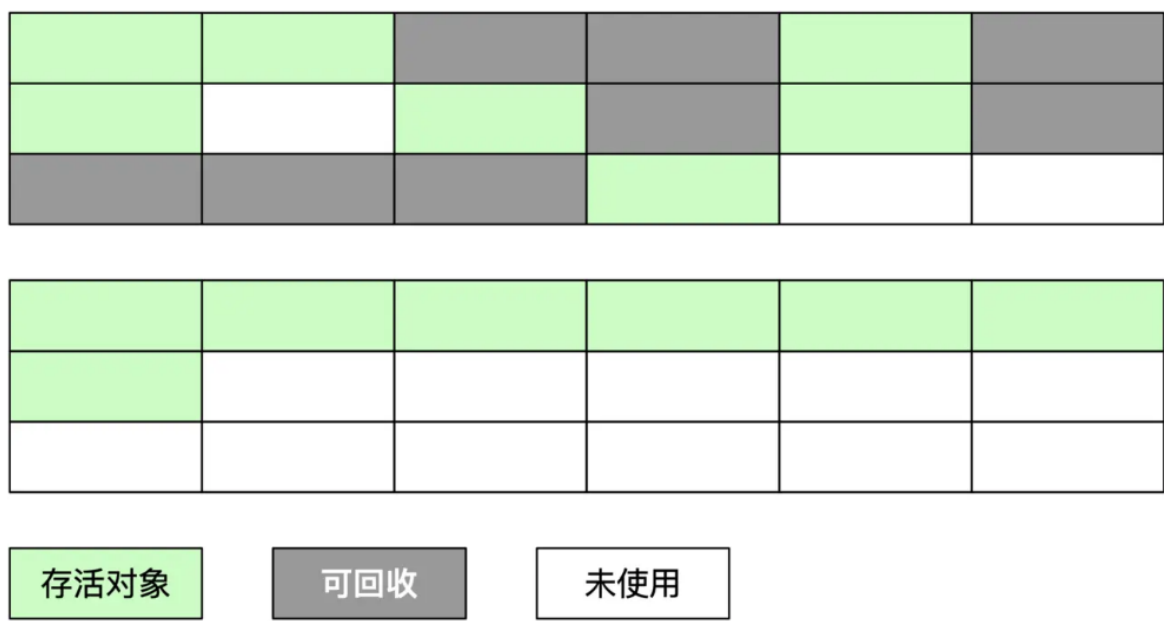
与标记-整理算法不同的是，在标记可回收的对象后将所有存活的对象压缩到内存的一端，使他们紧凑的排列在一起，然后对端边界以外的内存进行回收。

回收后，已用和未用的内存都各自一边。

优点：解决了标记-清理算法存在的内存碎片问题。

缺点：仍需要进行局部对象移动，一定程度上降低了效率。

标记-整理算法的执行过程如下图所示



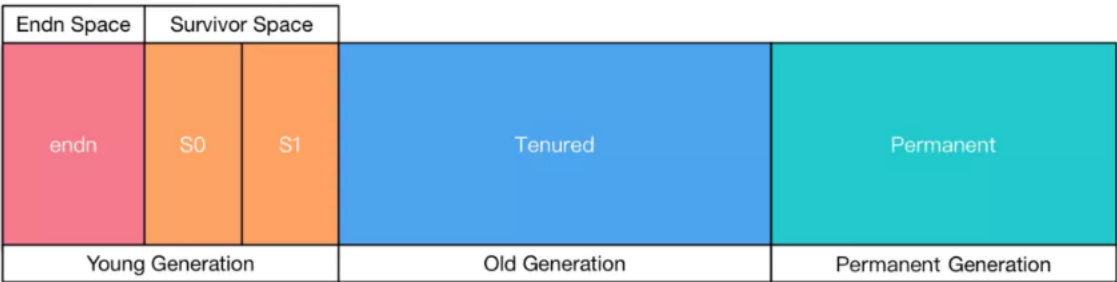
标记-整理算法适用场合： 老年代。

(4) 分代收集算法

分代收集法是目前大部分 JVM 所采用的方法，其核心思想是根据对象存活的不同生命周期将内存划分为不同的域，一般情况下将 GC 堆划分为老年代(Tenured/Old Generation)和新生代(YoungGeneration)。

老年代的特点是每次垃圾回收时只有少量对象需要被回收，新生代的特点是每次垃圾回收时都有大量垃圾需要被回收，因此可以根据不同区域选择不同的算法。

当前商业虚拟机都采用**分代收集**的垃圾收集算法。分代收集算法，顾名思义是根据对象的**存活周期**将内存划分为几块。一般包括**年轻代**、**老年代**和**永久代**，如图所示：



当前主流 VM 垃圾收集都采用“分代收集” (Generational Collection)算法, 这种算法会根据对象存活周期的不同将内存划分为几块, 如 JVM 中的 新生代、老年代、永久代，这样就可以根据各年代特点分别采用最适当的 GC 算法

新生代与复制算法

每次垃圾收集都能发现大批对象已死, 只有少量存活. 因此选用复制算法, 只需要付出少量存活对象的复制成本就可以完成收集

目前大部分 JVM 的 GC 对于新生代都采取 Copying 算法，因为新生代中每次垃圾回收都要回收大部分对象，即要复制的操作比较少，但通常并不是按照 1： 1 来划分新生代。

一般将新生代划分为一块较大的 Eden 空间和两个较小的 Survivor 空间(From Space, To Space)，每次使用Eden 空间和其中的一块 Survivor 空间，当进行回收时，将该两块空间中还存活的对象复制到另一块 Survivor 空间中。



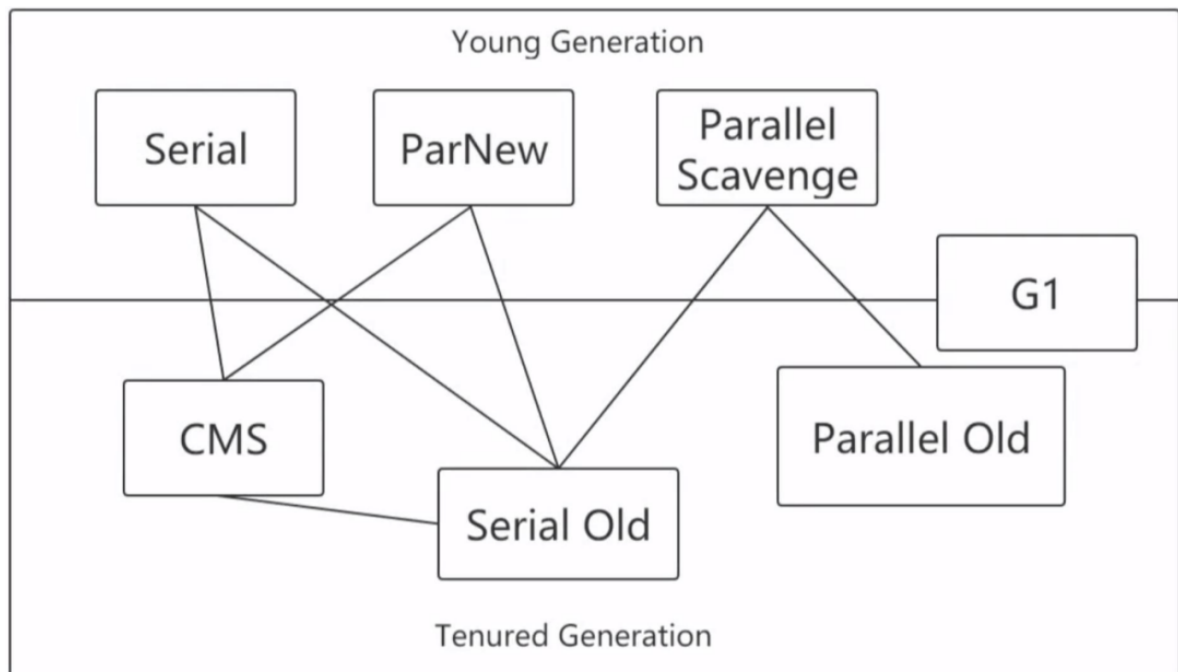
老年代与标记复制算法

因为老年代对象存活率高、没有额外空间对它进行分配担保, 就必须采用“标记—清理”或“标记—整理” 算法来进行回收, 不必进行内存复制, 且直接腾出空闲内存。因而采用 Mark-Compact 算法。

1. JAVA 虚拟机提到过的处于方法区的永生代(Permanet Generation)，它用来存储 class 类，常量，方法描述等。对永生代的回收主要包括废弃常量和无用的类。
2. 对象的内存分配主要在新生代的 Eden Space 和 Survivor Space 的 From Space(Survivor 目前存放对象的那一块)，少数情况会直接分配到老年代。
3. 当新生代的 Eden Space 和 From Space 空间不足时就会发生一次 GC，进行 GC 后，EdenSpace 和 From Space 区的存活对象会被挪到 To Space，然后将 Eden Space 和 FromSpace 进行清理。
4. 如果 To Space 无法足够存储某个对象，则将这个对象存储到老年代。
5. 在进行 GC 后，使用的便是 Eden Space 和 To Space 了，如此反复循环。
6. 当对象在 Survivor 区躲过一次 GC 后，其年龄就会+1。默认情况下年龄到达 15 的对象会被移到老年代中。

GC 垃圾收集器

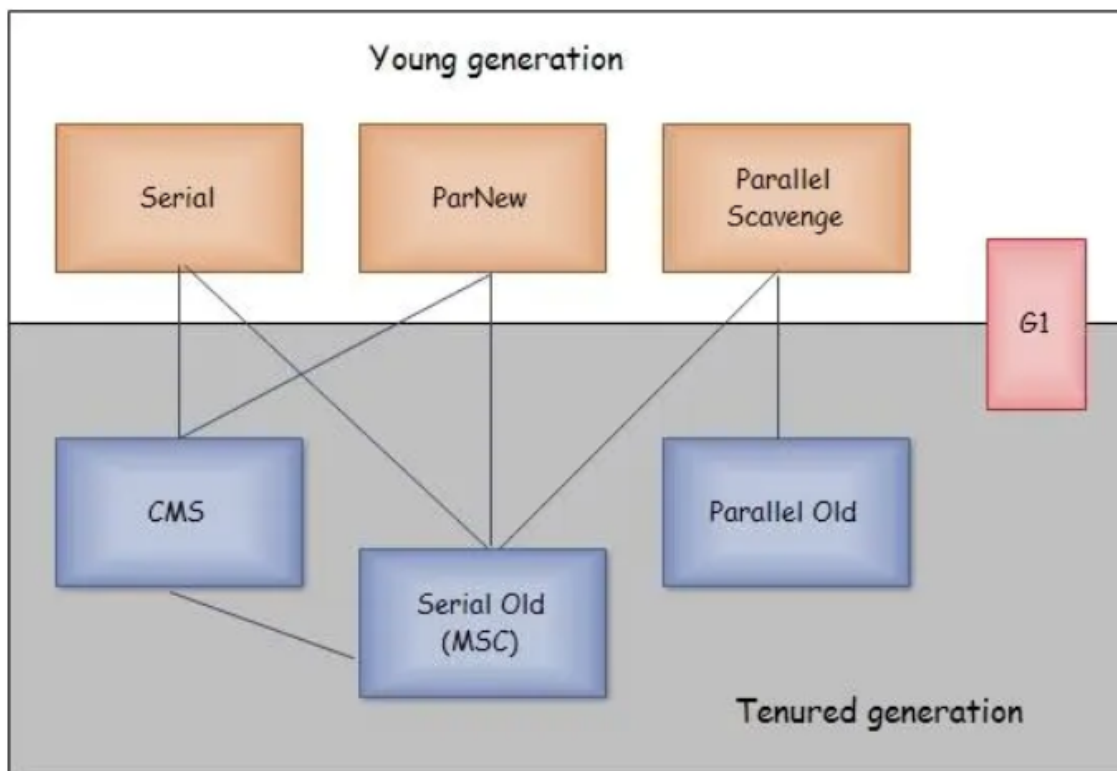
Java 堆内存被划分为新生代和年老代两部分，新生代主要使用复制和标记-清除垃圾回收算法；年老代主要使用标记-整理垃圾回收算法，因此 java 虚拟中针对新生代和年老代分别提供了多种不同的垃圾收集器，JDK1.6 中 Sun HotSpot 虚拟机的垃圾收集器如下：



转架构：通过尼恩的指导，小伙伴 **转架构** 后年薪从40W 涨到70W @公众号 技术自由圈

说一下 JVM 有哪些垃圾回收器？

如果说垃圾收集算法是内存回收的方法论，那么垃圾收集器就是内存回收的具体实现。下图展示了7种作用于不同分代的收集器，其中用于回收新生代的收集器包括Serial、PraNew、Parallel Scavenge，回收老年代的收集器包括Serial Old、Parallel Old、CMS，还有用于回收整个Java堆的G1收集器。不同收集器之间的连线表示它们可以搭配使用。



- Serial收集器 (复制算法): 新生代单线程收集器, 标记和清理都是单线程, 优点是简单高效;
- ParNew收集器 (复制算法): 新生代并行收集器, 实际上是Serial收集器的多线程版本, 在多核CPU环境下有着比Serial更好的表现;
- Parallel Scavenge收集器 (复制算法): 新生代并行收集器, 追求高吞吐量, 高效利用 CPU。吞吐量 = 用户线程时间/(用户线程时间+GC线程时间), 高吞吐量可以高效率的利用CPU时间, 尽快完成程序的运算任务, 适合后台应用等对交互相应要求不高的场景;
- Serial Old收集器 (标记-整理算法): 老年代单线程收集器, Serial收集器的老年代版本;
- Parallel Old收集器 (标记-整理算法): 老年代并行收集器, 吞吐量优先, Parallel Scavenge收集器的老年代版本;
- CMS(Concurrent Mark Sweep)收集器 (标记-清除算法): 老年代并行收集器, 以获取最短回收停顿时间目标的收集器, 具有高并发、低停顿的特点, 追求最短GC回收停顿时间。
- G1(Garbage First)收集器 (标记-整理算法): Java堆并行收集器, G1收集器是JDK1.7提供的一个新收集器, G1收集器基于“标记-整理”算法实现, 也就是说不会产生内存碎片。此外, G1收集器不同于之前的收集器的一个重要特点是: G1回收的范围是整个Java堆(包括新生代, 老年代), 而前六种收集器回收的范围仅限于新生代或老年代。

Serial 与 Parallel GC 之间的不同之处?

Serial 与 Parallel 在 GC 执行的时候都会引起 stop-the-world。它们之间主要不同 serial 收集器是默认的复制收集器, 执行 GC 的时候只有一个线程, 而parallel 收集器使用多个 GC 线程来执行。

类似的问题: 你知道哪几种垃圾收集器, 各自的优缺点, 重点讲下 cms和G1, 包括原理, 流程, 优缺点。

思路: 一定要记住典型的垃圾收集器, 尤其cms和G1, 它们的原理与区别, 涉及的垃圾回收算法。

参考答案:

1) 几种垃圾收集器:

- **Serial收集器**: 单线程的收集器, 收集垃圾时, 必须stop the world, 使用复制算法。
- **ParNew收集器**: Serial收集器的多线程版本, 也需要stop the world, 复制算法。
- **Parallel Scavenge收集器**: 新生代收集器, 复制算法的收集器, 并发的多线程收集器, 目标是达到一个可控的吞吐量。如果虚拟机总共运行100分钟, 其中垃圾花掉1分钟, 吞吐量就是99%。
- **Serial Old收集器**: 是Serial收集器的老年代版本, 单线程收集器, 使用标记整理算法。
- **Parallel Old收集器**: 是Parallel Scavenge收集器的老年代版本, 使用多线程, 标记-整理算法。
- **CMS(Concurrent Mark Sweep) 收集器**: 是一种以获得最短回收停顿时间为目标的收集器, **标记清除算法, 运作过程: 初始标记, 并发标记, 重新标记, 并发清除**, 收集结束会产生大量空间碎片。
- **G1收集器**: 标记整理算法实现, **运作流程主要包括以下: 初始标记, 并发标记, 最终标记, 筛选标记**。不会产生空间碎片, 可以精确地控制停顿。

2) CMS收集器和G1收集器的区别:

- CMS收集器是老年代的收集器, 可以配合新生代的Serial和ParNew收集器一起使用;
- G1收集器收集范围是老年代和新生代, 不需要结合其他收集器使用;
- CMS收集器以最小的停顿时间为目标的收集器;
- G1收集器可预测垃圾回收的停顿时间
- CMS收集器是使用“标记-清除”算法进行的垃圾回收, 容易产生内存碎片
- G1收集器使用的是“标记-整理”算法, 进行了空间整合, 降低了内存空间碎片。

详细介绍一下 CMS 垃圾回收器?

CMS 是英文 Concurrent Mark-Sweep 的简称, 是以牺牲吞吐量为代价来获得最短回收停顿时间的垃圾回收器。对于要求服务器响应速度的应用上, 这种垃圾回收器非常适合。在启动 JVM 的参数加上“-XX:+UseConcMarkSweepGC”来指定使用 CMS 垃圾回收器。

CMS 使用的是标记-清除的算法实现的, 所以在 gc 的时候会回产生大量的内存碎片, 当剩余内存不能满足程序运行要求时, 系统将会出现 Concurrent Mode Failure, 临时 CMS 会采用 Serial Old 回收器进行垃圾清除, 此时的性能将会被降低。

Serial 垃圾收集器 (单线程、复制算法)

Serial (英文连续) 是最基本垃圾收集器, 使用复制算法, 曾经是JDK1.3.1 之前新生代唯一的垃圾收集器。Serial 是一个单线程的收集器, 它不但只会使用一个 CPU 或一条线程去完成垃圾收集工作, 并且在垃圾收集的同时, 必须暂停其他所有的工作线程, 直到垃圾收集结束。

Serial 垃圾收集器虽然在收集垃圾过程中需要暂停所有其他的工作线程, 但是它简单高效, 对于限定单个 CPU 环境来说, 没有线程交互的开销, 可以获得最高的单线程垃圾收集效率, 因此 Serial垃圾收集器依然是 java 虚拟机运行在 Client 模式下默认的新生代垃圾收集器。

ParNew 垃圾收集器 (Serial+多线程)

ParNew 垃圾收集器其实是 Serial 收集器的多线程版本, 也使用复制算法, 除了使用多线程进行垃圾收集之外, 其余的行为和 Serial 收集器完全一样, ParNew 垃圾收集器在垃圾收集过程中同样也要暂停所有其他的工作线程。

ParNew 收集器默认开启和 CPU 数目相同的线程数, 可以通过-XX:ParallelGCThreads 参数来限制垃圾收集器的线程数。【Parallel: 平行的】

ParNew 虽然是除了多线程外和Serial 收集器几乎完全一样，但是ParNew垃圾收集器是很多 java虚拟机运行在 Server 模式下新生代的默认垃圾收集器。

Parallel Scavenge 收集器（多线程复制算法、高效）

Parallel Scavenge 收集器也是一个新生代垃圾收集器，同样使用复制算法，也是一个多线程的垃圾收集器，它重点关注的是程序达到一个可控制的吞吐量（Throughput，CPU 用于运行用户代码的时间/CPU 总消耗时间，即吞吐量=运行用户代码时间/(运行用户代码时间+垃圾收集时间))，高吞吐量可以最高效率地利用 CPU 时间，尽快地完成程序的运算任务，主要适用于在后台运算而不需要太多交互的任务。自适应调节策略也是 ParallelScavenge 收集器与 ParNew 收集器的一个重要区别。

- **新生代**：Parallel Scavenge 收集器也是一个新生代垃圾收集器；
- **复制算法**：Parallel Scavenge 收集器同样使用复制算法；
- **多线程**：Parallel Scavenge 收集器也是一个多线程的垃圾收集器；
- **高吞吐量**：Parallel Scavenge 重点关注的是程序达到一个可控制的吞吐量（Throughput，吞吐量 = 运行用户代码时间 / (运行用户代码时间 + 垃圾收集时间))，高吞吐量可以最高效率地利用 CPU 时间，尽快地完成程序的运算任务；
- **适用场景**：主要适用于在后台运算而不需要太多交互的任务。自适应调节策略也是 ParallelScavenge 收集器与 ParNew 收集器的一个重要区别。

新生代 Parallel Scavenge 收集器与 ParNew 收集器工作原理类似：

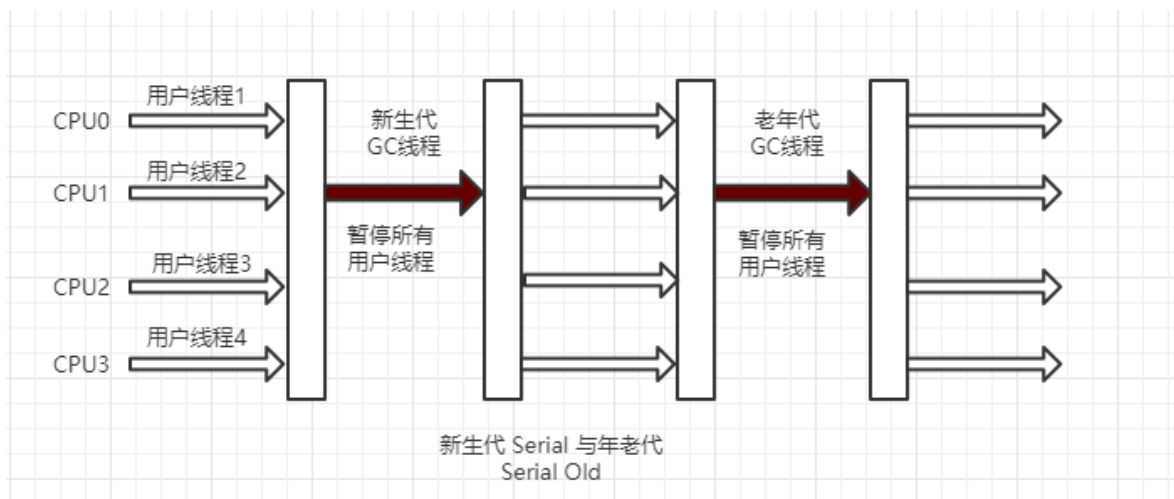
1. 都是多线程的收集器；
2. 都使用的是复制算法；
3. 在垃圾收集过程中都需要暂停所有的工作线程。

Serial Old 收集器（单线程标记整理算法）

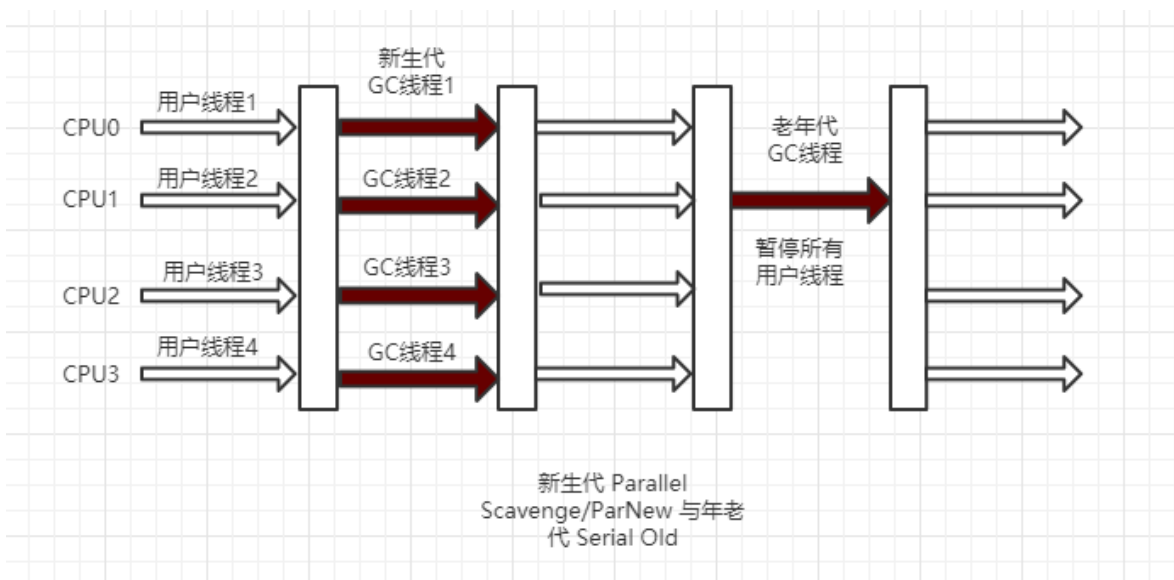
Serial Old 是 Serial 垃圾收集器年老代版本，它同样是个单线程的收集器，使用标记-整理算法，这个收集器也主要是运行在 Client 默认的

- **老年代**：Serial Old 是 Serial 垃圾收集器老年代的版本；
- **标记-整理算法**：Serial Old 使用标记-整理算法；
- **单线程**：Serial Old 与 Serial 一样是单线程收集器；
- **Client 模式**：JVM 运行在 Client 模式下，Serial Old 是默认的老年代垃圾收集器；
- **Server 模式**：JVM 运行在 Server 模式下，Serial Old 主要有两个用途：
 1. 在 JDK1.5 之前版本中与新生代的 Parallel Scavenge 收集器搭配使用；
 2. 作为年老代中使用 CMS 收集器的后备垃圾收集方案。

新生代 Serial 与年老代 Serial Old 搭配垃圾收集过程图：



新生代 ParallelScavenge/ParNew 与老年代 Serial Old 搭配垃圾收集过程图：



新生代 Parallel Scavenge 收集器与 ParNew 收集器工作原理类似，都是多线程的收集器，都使用的是复制算法，在垃圾收集过程中都需要暂停所有的工作线程。

Parallel Old 收集器（多线程标记整理算法）

Parallel Old 收集器是Parallel Scavenge的老年代版本，使用多线程的标记-整理算法，在JDK1.6才开始提供。

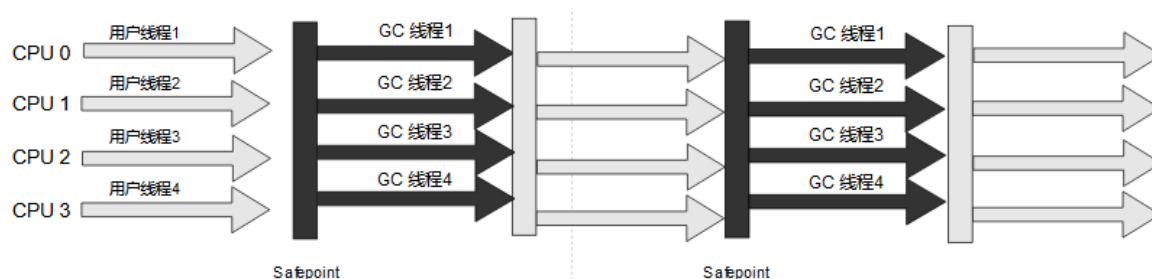
在JDK1.6之前，新生代使用ParallelScavenge收集器只能搭配年老代的Serial Old收集器，只能保证新生代的吞吐量优先，无法保证整体的吞吐量，

在JDK1.6才开始，Parallel Old 正是为了在老年代同样提供吞吐量优先的垃圾收集器，如果系统对吞吐量要求比较高，可以优先考虑新生代Parallel Scavenge和老年代Parallel Old收集器的搭配策略。

- **老年代**：Parallel Old 收集器是 Parallel Scavenge 的老年代版本；
- **标记-整理算法**：Parallel Old 收集器使用标记-整理算法；
- **多线程**：Parallel Old 收集器是多线程收集器；

- **高吞吐量**：Parallel Old 是能够为老年代提供吞吐量优先的垃圾收集器；
- **JDK1.6**：Parallel Old 是 JDK1.6 才开始提供的。

新生代 Parallel Scavenge 和老年代 Parallel Old 收集器搭配运行过程图



新生代 Parallel Scavenge 收集器，使用的是标记-复制算法，在垃圾收集过程中都需要暂停所有的工作线程。

老年代 Parallel Old 收集器，使用的是标记-整理算法，在垃圾收集过程中都需要暂停所有的工作线程。

Parallel Old 收集器出现后，“吞吐量优先”收集器终于有了比较名副其实的应用组合，在注重吞吐量以及 CPU 资源敏感的场所，都可以优先考虑 Parallel Scavenge 加 Parallel Old 收集器。

CMS 收集器（多线程标记清除算法）

Concurrent mark sweep (CMS) 收集器是一种老年代垃圾收集器，其最主要目标是获取最短垃圾回收停顿时间，和其他老年代使用标记-整理算法不同，它使用多线程的标记-清除算法。最短的垃圾收集停顿时间可以为交互比较高的程序提高用户体验。

- **老年代**：CMS (Concurrent mark sweep) 收集器是一种老年代垃圾收集器；
- **标记-清除算法**：和其他老年代使用标记-整理算法，CMS 使用标记-清除算法；
- **多线程**：CMS 采用的是多线程并发的标记-清除算法；
- **停顿时间端**：CMS 最主要目标是获取最短垃圾回收停顿时间，最短的垃圾收集停顿时间可以为交互比较高的程序提高用户体验。

CMS 工作机制相比其他的垃圾收集器来说更复杂。整个过程分为以下 4 个阶段：

初始标记

只是标记一下 GC Roots 能直接关联的对象，速度很快，仍然需要暂停所有的工作线程。

并发标记

进行 GC Roots 跟踪的过程，和用户线程一起工作，不需要暂停工作线程。

重新标记

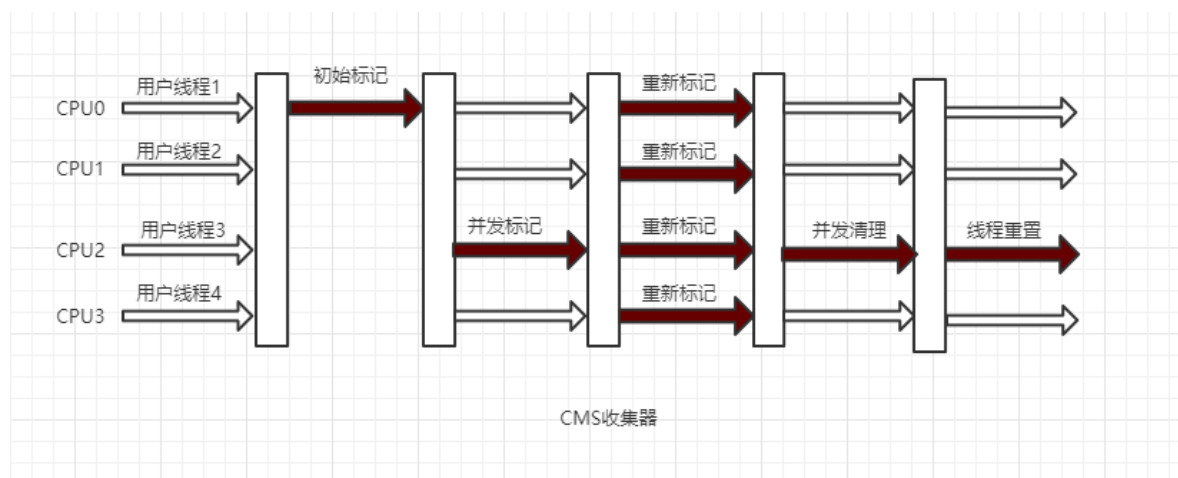
为了修正在并发标记期间，因用户程序继续运行而导致标记产生变动的那一部分对象的标记记录，仍然需要暂停所有的工作线程。

并发清除

清除 GC Roots 不可达对象，和用户线程一起工作，不需要暂停工作线程。由于耗时最长的并发标记和

并发清除过程中，垃圾收集线程可以和用户现在一起并发工作，所以总体上来看CMS 收集器的内存回收和用户线程是一起并发地执行。

CMS 收集器工作过程



G1 收集器

Garbage first 垃圾收集器是目前垃圾收集器理论发展的最前沿成果，相比与 CMS 收集器，G1 收集器两个最突出的改进是：

- 1.基于标记-整理算法，不产生内存碎片。
- 2.可以非常精确控制停顿时间，在不牺牲吞吐量前提下，实现低停顿垃圾回收。

G1 收集器避免全区域垃圾收集，它把堆内存划分为大小固定的几个独立区域，并且跟踪这些区域的垃圾收集进度，同时在后台维护一个优先级列表，每次根据所允许的收集时间，优先回收垃圾最多的区域。区域划分和优先级区域回收机制，确保 G1 收集器可以在有限时间获得最高的垃圾收集效率

G1特点

- **无分代**：G1 将新生代，老年代的物理空间划分取消了。这样我们再也不用单独的空间对每个代进行设置了，不用担心每个代内存是否足够；
- **标记-整理算法**：G1 收集器采用标记-整理算法，无内存碎片产生；
- **分区回收**：G1 虽然没有了新生代与老年代的物理限制，但是 G1 采取内存分区策略，将堆内存划分为大小固定的几个独立区域。在分区中，同时存在新生代与老年代；

分区



新生代区域： G1 收集器中新生代的垃圾收集依然采用暂停所有应用线程的方式，将存活对象拷贝到老年代或者 Survivor 空间；

老年代区域： G1 收集器通过将对象从一个区域复制到另外一个区域，以此来完成老年代的清理工作；

Humongous区域： 巨型对象区域。如果一个对象占用的空间超过了分区容量 50% 以上，G1 收集器就认为这是一个巨型对象。这些巨型对象，默认直接会被分配在老年代，但是如果它是一个短期存在的巨型对象，就会对垃圾收集器造成负面影响。为了解决这个问题，G1 划分了一个 Humongous 区，它用来专门存放巨型对象。如果一个 H 区装不下一个巨型对象，那么 G1 会寻找连续的 H 分区来存储。为了能找到连续的 H 区，有时候不得不启动 Full GC。

对象分配策略

说起大对象的分配，我们不得不谈谈对象的分配策略。

它分为3个阶段：

1. TLAB(Thread Local Allocation Buffer)线程本地分配缓冲区
2. Eden区中分配
3. Humongous区分配

TLAB为线程本地分配缓冲区，它的目的为了使对象尽可能快的分配出来。

如果对象在一个共享的空间中分配，我们需要采用一些同步机制来管理这些空间内的空闲空间指针。

在Eden空间中，每一个线程都有一个固定的分区用于分配对象，即一个TLAB。分配对象时，线程之间不再需要进行任何的同步。

对TLAB空间中无法分配的对象，JVM会尝试在Eden空间中进行分配。

如果Eden空间无法容纳该对象，就只能在老年代中进行分配空间。

最后，G1提供了两种GC模式，Young GC和Mixed GC，两种都是Stop The World(STW)的。

下面我们将分别介绍一下这2种模式。

G1 Young GC

Young GC 主要是对 Eden 区进行 GC，它在 Eden 空间耗尽时会被触发。在这种情况下，Eden 空间的数据移动到 Survivor 空间中，如果 Survivor 空间不够，Eden 空间的部分数据会直接晋升到老年代空间。Survivor 区的数据移动到新的 Survivor 区中，也有部分数据晋升到老年代空间中。最终 Eden 空间的数据为空，GC 停止工作，应用线程继续执行。

G1 Young GC 阶段：

1. **根扫描：** 静态和本地对象被扫描；

2. **更新RS**: 处理 Dirty Card 队列更新 RS (Remembered Set, 作用是跟踪指向某个 Heap 区内的对象引用) ;
3. **处理RS**: 检测从年轻代指向老年代的对象;
4. **对象拷贝**: 拷贝存活的对象到 Survivor/Old 区域;
5. **处理引用队列**: 软引用, 弱引用, 虚引用处理。

G1 Mix GC

Mix GC不仅进行正常的新生代垃圾收集, 同时也回收部分后台扫描线程标记的老年代分区。

G1 Mix GC运行步骤:

1. **全局并发标记** (global concurrent marking)
 - 1.1. **初始标记 (initial mark, STW)** : 在此阶段, G1 GC 对根进行标记。该阶段与常规的 (STW) 年轻代垃圾回收密切相关;
 - 1.2. **根区域扫描 (root region scan)** : G1 GC 在初始标记的存活区扫描对老年代的引用, 并标记被引用的对象。该阶段与应用程序 (非 STW) 同时运行, 并且只有完成该阶段后, 才能开始下一次 STW 年轻代垃圾回收;
 - 1.3. **并发标记 (Concurrent Marking)** : G1 GC 在整个堆中查找可访问的 (存活的) 对象。该阶段与应用程序同时运行, 可以被 STW 年轻代垃圾回收中断;
 - 1.4. **最终标记 (Remark, STW)** : 该阶段是 STW 回收, 帮助完成标记周期。G1 GC 清空 SATB 缓冲区, 跟踪未被访问的存活对象, 并执行引用处理;
 - 1.5. **清除垃圾 (Cleanup, STW)** : 在这个最后阶段, G1 GC 执行统计和 RSet 净化的 STW 操作。在统计期间, G1 GC 会识别完全空闲的区域和可供进行混合垃圾回收的区域。清理阶段在将空白区域重置并返回到空闲列表时为部分并发。
2. **拷贝存活对象** (evacuation)

G1 收集器与 CMS 收集器相比, G1 收集器两个最突出的改进是:

1. 基于标记-整理算法, 不产生内存碎片;
2. 可以非常精确控制停顿时间, 在不牺牲吞吐量前提下, 实现低停顿垃圾回收。

```
// G1 收集器参数设置
-XX:+UseG1GC -Xmx32g -XX:MaxGCPauseMillis=200
// -XX:+UseG1GC  -- 为开启G1垃圾收集器,
// -Xmx32g  -- 设计堆内存的最大内存为32G,
// -XX:MaxGC
```

新生代垃圾回收器和老年代垃圾回收器都有哪些? 有什么区别?

- 新生代回收器: Serial、ParNew、Parallel Scavenge
- 老年代回收器: Serial Old、Parallel Old、CMS
- 整堆回收器: G1

新生代垃圾回收器一般采用的是复制算法, 复制算法的优点是效率高, 缺点是内存利用率低; 老年代回收器一般采用的是标记-整理的算法进行垃圾回收。

简述分代垃圾回收器是怎么工作的?

分代回收器有两个分区: 老年代和新生代, 新生代默认的空间占比总空间的 1/3, 老年代的默认占比是 2/3。

新生代使用的是复制算法，新生代里有 3 个分区：Eden、To Survivor、From Survivor，它们的默认占比是 8:1:1，它的执行流程如下：

- 把 Eden + From Survivor 存活的对象放入 To Survivor 区；
- 清空 Eden 和 From Survivor 分区；
- From Survivor 和 To Survivor 分区交换，From Survivor 变 To Survivor，To Survivor 变 From Survivor。

每次在 From Survivor 到 To Survivor 移动时都存活的对象，年龄就 +1，当年龄到达 15（默认配置是 15）时，升级为老年代。大对象也会直接进入老年代。

老年代当空间占用到达某个值之后就会触发全局垃圾回收，一般使用标记整理的执行算法。以上这些循环往复就构成了整个分代垃圾回收的整体执行流程。

什么时候会触发FullGC?

除直接调用System.gc外，触发Full GC执行的情况有如下四种。

1.旧世代空间不足

老年代空间只有在新生代对象转入及创建为大对象、大数组时才会出现不足的现象，当执行Full GC后空间仍然不足，则抛出如下错误：

```
java.lang.OutOfMemoryError: Java heap space
```

为避免以上两种状况引起的FullGC，调优时应尽量做到让对象在Minor GC阶段被回收、让对象在新生代多存活一段时间及不要创建过大的对象及数组。

2.Permanet Generation空间满

PermanetGeneration中存放的为一些class的信息等，当系统中要加载的类、反射的类和调用的方法较多时，Permanet Generation

可能会被占满，在未配置为采用CMS GC的情况下会执行Full GC。如果经过Full GC仍然回收不了，那么JVM会抛出如下错误信息：

```
java.lang.OutOfMemoryError: PermGen space
```

为避免Perm Gen占满造成Full GC现象，可采用的方法为增大Perm Gen空间或转为使用CMS GC。

3.CMS GC时出现promotion failed和concurrent mode failure

对于采用CMS进行老年代GC的程序而言，尤其要注意GC日志中是否有promotion failed和concurrent mode failure两种状况，当这两种状况出现时可能会触发Full GC。

promotionfailed是在进行Minor GC时，survivor space放不下、对象只能放入老年代，而此时老年代也放不下造成的；concurrent mode failure是在执行CMS GC的过程中同时有对象要放入老年代，而此时老年代空间不足造成的。

应对措施为：增大survivorspace、老年代空间或调低触发并发GC的比率，但在JDK 5.0+、6.0+的版本中有可能由于JDK的bug29导致CMS在remark完毕后很久才触发sweeping动作。对于这种状况，可通过设置-XX:CMSMaxAbortablePrecleanTime=5（单位为ms）来避免。

4.统计得到的Minor GC晋升到旧生代的平均大小大于旧生代的剩余空间

这是一个较为复杂的触发情况，Hotspot为了避免由于新生代对象晋升到旧世代导致旧世代空间不足的现象，在进行Minor GC时，做了一个判断，如果之前统计所得到的Minor GC晋升到旧生代的平均大小大于旧生代的剩余空间，那么就直接触发Full GC。

例如程序第一次触发MinorGC后，有6MB的对象晋升到旧世代，那么当下一次Minor GC发生时，首先检查旧生代的剩余空间是否大于6MB，如果小于6MB，则执行Full GC。

当新生代采用PSGC时，方式稍有不同，PS GC是在Minor GC后也会检查，例如上面的例子中第一次Minor GC后，PS GC会检查此时旧生代的剩余空间是否大于6MB，如小于，则触发对旧生代的回收。除了以上4种状况外，对于使用RMI来进行RPC或管理的Sun JDK应用而言，默认情况下会一小时执行一次Full GC。可通过在启动时通过- java-Dsun.rmi.dgc.client.gcInterval=3600000来设置Full GC执行的间隔时间或通过-XX:+ DisableExplicitGC来禁止RMI调用System.gc



方法区的垃圾回收

《Java虚拟机规范》对方法区的约束是非常宽松的，提到过可以不要求虚拟机在方法区中实现垃圾收集；

事实上也确实有未实现或未能完整实现方法区类型卸载的收集器存在，如JDK11时期的ZGC收集器就不支持类卸载。

一般来说，方法区的回收效果比较难令人满意，尤其是**类型的卸载，条件相当苛刻**。

但是这部分区域的回收**有时又确实是必要的**。以前Sun公司的Bug列表中，曾出现过的若干个严重的Bug就是由于低版本的Hotspot虚拟机对此区域未完全回收而导致内存泄漏。

方法区的垃圾收集主要回收两部分内容：

- 运行时常量池中废弃的常量
- 不再使用的类型。

A. 运行时常量池中废弃的常量

方法区内运行时常量池之中，主要存放的两大类常量：字面量和符号引用。

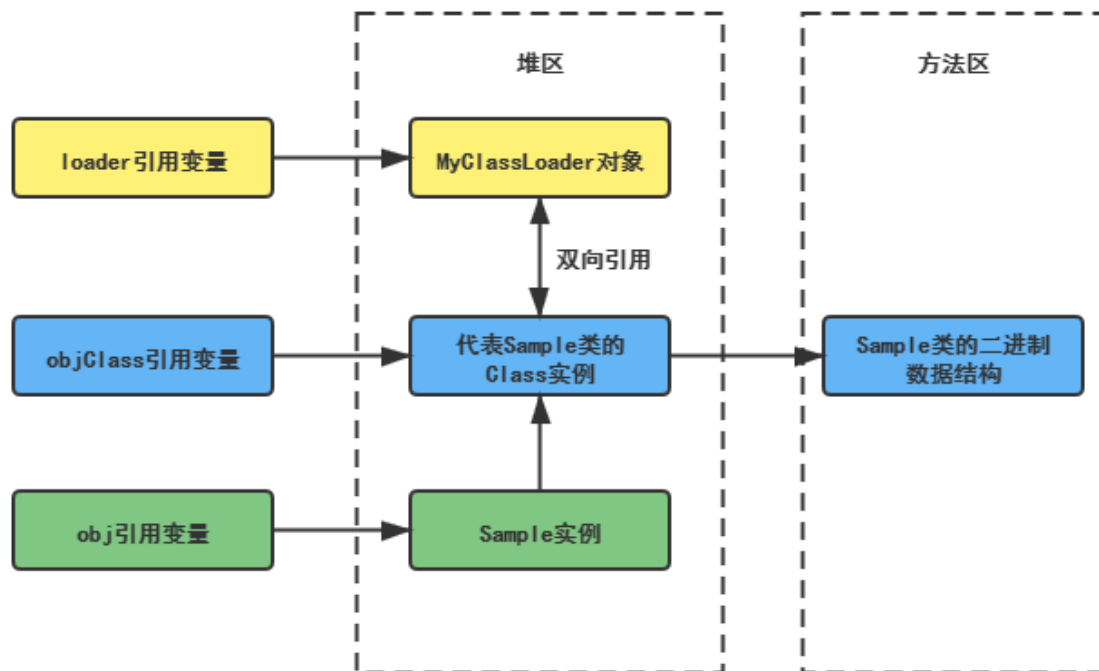
字面量比较接近Java语言层次的常量概念，如文本字符串、被声明为final的常量值等；而符号引用则属于编译原理方面的概念，包括下面三类常量：1) 类和接口的全限定名；2) 字段的名称和描述符；3) 方法的名称和描述符。

Hotspot虚拟机对运行时常量池的回收策略是很明确的，只要常量池中的常量**没有被任何地方引用，就可以被回收**。

B. 不再使用的类型

判定一个类型是否属于“不再被使用的类型”的条件比较苛刻，需要同时满足下面三个条件：

- **该类所有的实例**都已经被回收，也就是Java堆中不存在该类及其任何派生子类的实例。
- **加载该类的类加载器**已经被回收，这个条件除非是经过精心设计的可替换类加载器的场景，如OSGi、JSP的重加载等，否则通常是很难达成的。
- **该类对应的java.lang.Class对象**没有在任何地方被引用，无法在任何地方通过反射访问该类的方法。



Java虚拟机被允许对满足上述三个条件的无用类进行回收，这里说的**仅仅是“被允许”**，而并不是和对象一样，没有引用了就必然会回收。

在大量使用反射、动态代理、CGLib等字节码框架，动态生成JSP以及OSGi这类频繁自定义类加载器的场景中，通常都需要Java虚拟机具备**类型卸载**的能力，以保证不会对方法区造成过大的内存压力。

问：说是方法区和堆的区别？

逻辑上，方法区是属于堆的一部分，

但实现上，一些简单的jvm实现，可能不会对方法区选择进行垃圾收集或者进行压缩。

HotSpot虚拟机的方法区还有一个别名，叫做 **Non-Heap（非堆）**，目的就是要和堆分开。

所以，**方法区看作是一块独立于Java堆的内存空间**。

- 方法区是**各线程共享**的内存区域。
- 方法区在JVM启动的时候被创建，它的实例的**物理内存空间可以是不连续的**。
- 方法区的大小，可以选择固定大小或者可扩展。如果方法区无法满足新的内存分配需求时，将**抛出OOM异常**。
- 方法区的大小决定了系统可以保存多少个类，如果系统定义了太多的类，会导致方法区溢出，虚拟机会抛出内存溢出错误：

```
java.lang.OutOfMemoryError: PermGen space
```

或者

```
java.lang.OutOfMemoryError: Metaspace
```

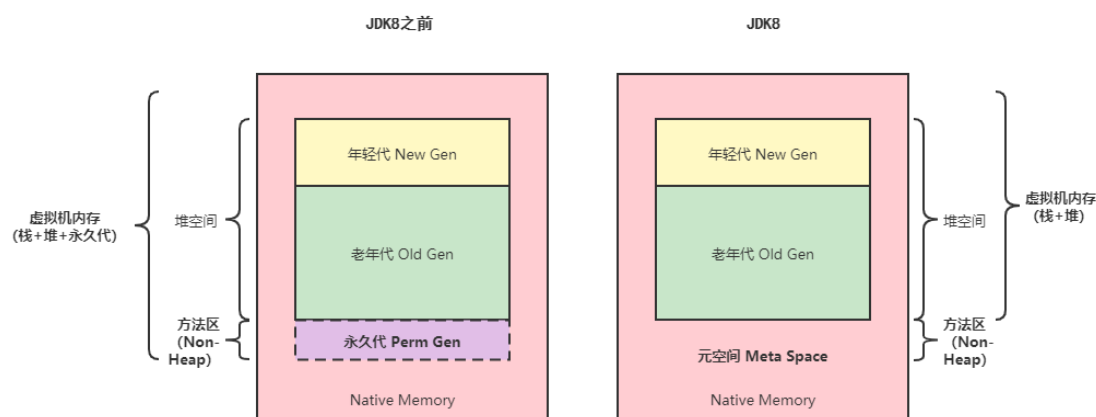
比如，以下情况：

- 加载大量的第三方的jar包；
- Tomcat部署的工程过多（30-50个）；

- 大量动态的生成反射类。
- 关闭JVM就会**释放该区域的内存**。

方法区是虚拟机规范中提出的，不同的虚拟机可以有不同的实现，在HotSpot不同版本中方法区的演进如下：

- 在JDK7及以前，习惯上把方法区称为**永久代（PermGen）**，从JDK8开始，使用**元空间（Metaspace）**作为方法区的实现。
 - In JDK 8, classes metadata is now stored in the native heap and this space is called Metaspace.
- 本质上，方法区和永久代并不等价，这两个概念仅是对 hotspot而言的。《Java虚拟机规范》对如何实现方法区，不做统一要求，例如：BEA JRockit和IBM J9中，就不存在永久代的概念。
 - 现在来看，当年使用永久代，不是好的idea，导致Java程序更容易出现OOM（超过-XX:MaxPermSize上限）。
- 元空间的本质和永久代类似，都是对JVM规范中方法区的实现。两者的最大区别是：元空间不在虚拟机设置的内存中，而是**使用本地内存**。



问：如何解决方法区的OOM？

要解决OOM异常或heap space的异常，一般手段是通过内存映像分析工具（如 Eclipse Memory Analyzer）对dump出来的堆转储快照文件进行分析，

重点是确认内存中的对象是否还都是必要存活的，即要先分清楚到底是出现了**内存泄漏（Memory Leak）**还是**内存溢出（Memory Overflow）**。

- 如果是内存泄漏，可进一步通过工具查看泄漏对象到GC Roots的引用链，就能找到泄漏对象是通过怎样的路径与 GC Roots相关联，并导致垃圾收集器无法自动回收它们的。掌握泄漏对象的类型信息，以及 GC Roots引用链的信息后，就可以比较准确地定位出泄漏代码的位置。
- 如果不存在内存泄漏，即内存中的对象确实都还必须存活着，就应当根据机器物理内存，检查虚拟机的堆参数（-Xmx与-Xms），看是否还可以调大；从代码上检查是否存在某些对象生命周期过长、持有状态时间过长的情况（其实也算是内存泄漏），尝试减少程序运行期的内存消耗。

问：如何设置方法区内存的大小

方法区的大小不是固定的，JVM可以根据应用的需要进行动态调整。

JDK7及之前：

- 通过 `-XX:Permsize` 来设置永久代的初始分配空间，默认值是20.75M。
- 通过 `-XX:MaxPermsize` 来设置永久代的最大可分配空间，32位机器默认是64M，64位机器模式是82M。
- 当JVM加载的类信息容量超过最大值，会报异常`OutOfMemoryError: PermGen space`。

JDK8及之后：

- 通过设置参数 `-XX:MetaspaceSize` 和 `-XX:MaxMetaspaceSize` 来设置元空间的初始分配空间和最大可分配空间。
- 上述参数的默认值依赖于平台，Windows系统下，`-XX:MetaspaceSize`是21M，`-XX:MaxMetaspaceSize`的值是-1，即没有限制。
- 如果不指定元空间大小，使用默认值，方法区可能耗尽所有的可用系统内存。如果元空间发生溢出，虚拟机将抛出异常`OutOfMemoryError: Metaspace`。
- 通过`-XX:MetaspaceSize`设置的初始元空间大小是一个初始的高水位线，一旦触及这个水位线，Full GC将会被触发并卸载不再使用的类，然后这个高水位线将会重置。新的高水位线的值取决于GC后释放了多少元空间，如果释放的空间不足，那么在不超过 `MaxMetaspaceSize`时，适当提高该值。如果释放空间过多，则适当降低该值。所以，如果初始化的高水位线设置过低，高水位线调整情况会发生很多次，通过垃圾回收器的日志，可以观察到Full GC多次调用。为了避免频繁Full GC，建议将`-XX:MetaspaceSize`设置为一个相对较高的值。

内存分配策略

简述java内存分配与回收策略以及Minor GC和Major GC

所谓自动内存管理，最终要解决的也就是内存分配和内存回收两个问题。前面我们介绍了内存回收，这里我们再来聊聊内存分配。

对象的内存分配通常是在Java堆上分配（随着虚拟机优化技术的诞生，某些场景下也会在栈上分配，后面会详细介绍），对象主要分配在新生代的Eden区，如果启动了本地线程缓冲，将按照线程优先在TLAB上分配。少数情况下也会直接在老年代上分配。总的来说分配规则不是百分百固定的，其细节取决于哪一种垃圾收集器组合以及虚拟机相关参数有关，但是虚拟机对于内存的分配还是会遵循以下几种「普世」规则：

对象优先在Eden区分配

多数情况，对象都在新生代Eden区分配。当Eden区分配没有足够的空间进行分配时，虚拟机将会发起一次Minor GC。如果本次GC后还是没有足够的空间，则将启用分配担保机制在老年代中分配内存。

这里我们提到Minor GC，如果你仔细观察过GC日常，通常我们还能从日志中发现Major GC/Full GC。

- **Minor GC** 是指发生在新生代的GC，因为Java对象大多都是朝生夕死，所有Minor GC非常频繁，一般回收速度也非常快；
- **Major GC/Full GC** 是指发生在老年代的GC，出现了Major GC通常会伴随至少一次Minor GC。Major GC的速度通常会比Minor GC慢10倍以上。

大对象直接进入老年代

所谓大对象是指需要大量连续内存空间的对象，频繁出现大对象是致命的，会导致在内存还有不少空间的情况下提前触发GC以获取足够的连续空间来安置新对象。

前面我们介绍过新生代使用的是标记-清除算法来处理垃圾回收的，如果大对象直接在新生代分配就会导致 Eden 区和两个 Survivor 区之间发生大量的内存复制。因此对于大对象都会直接在老年代进行分配。

长期存活对象将进入老年代

虚拟机采用分代收集的思想来管理内存，那么内存回收时就必须判断哪些对象应该放在新生代，哪些对象应该放在老年代。因此虚拟机给每个对象定义了一个对象年龄的计数器，如果对象在 Eden 区出生，并且能够被 Survivor 容纳，将被移动到 Survivor 空间中，这时设置对象年龄为 1。对象在 Survivor 区中每「熬过」一次 Minor GC 年龄就加 1，当年龄达到一定程度（默认 15）就会被晋升到老年代。

对象分配规则

1. 对象优先分配在Eden区，如果Eden区没有足够的空间时，虚拟机执行一次Minor GC。
2. 大对象直接进入老年代（大对象是指需要大量连续内存空间的对象）。这样做的目的是避免在Eden区和两个Survivor区之间发生大量的内存拷贝（新生代采用复制算法收集内存）。
3. 长期存活的对象进入老年代。虚拟机为每个对象定义了一个年龄计数器，如果对象经过了1次Minor GC那么对象会进入Survivor区，之后每经过一次Minor GC那么对象的年龄加1，知道达到阈值对象进入老年区。
4. 动态判断对象的年龄。如果Survivor区中相同年龄的所有对象大小的总和大于Survivor空间的一半，年龄大于或等于该年龄的对象可以直接进入老年代。
5. 空间分配担保。每次进行Minor GC时，JVM会计算Survivor区移至老年区的对象的平均大小，如果这个值大于老年区的剩余值大小则进行一次Full GC，如果小于检查HandlePromotionFailure设置，如果true则只进行Monitor GC,如果false则进行Full GC



不怕裁：10年小伙 12天火速上岸，反涨20%，爽爆了 @公众号 技术自由圈

虚拟机类加载机制

简述java类加载机制？

虚拟机把描述类的数据从Class文件加载到内存，并对数据进行校验，解析和初始化，最终形成可以被虚拟机直接使用的java类型。

描述一下JVM加载Class文件的原理机制

Java中的所有类，都需要由类加载器装载到JVM中才能运行。类加载器本身也是一个类，而它的工作就是把class文件从硬盘读取到内存中。在写程序的时候，我们几乎不需要关心类的加载，因为这些都是隐式装载的，除非我们有特殊的用法，像是反射，就需要显式的加载所需要的类。

类装载方式，有两种：

- 1.隐式装载，程序在运行过程中当碰到通过new 等方式生成对象时，隐式调用类装载机加载对应的类到jvm中，
- 2.显式装载，通过class.forName()等方法，显式加载需要的类

Java类的加载是动态的，它并不会一次性将所有类全部加载后再运行，而是保证程序运行的基础类(像是基类)完全加载到jvm中，至于其他类，则在需要的时候才加载。这当然就是为了节省内存开销。

描述一下 JVM 加载 class 文件的原理机制

JVM 中类的装载是由类加载器（ClassLoader）和它的子类来实现的，Java 中的类加载器是一个重要的 Java 运行时系统组件，它负责在运行时查找和装入类文件中的类。

由于 Java 的跨平台性，经过编译的 Java 源程序并不是一个可执行程序，而是一个或多个类文件。当 Java 程序需要使用某个类时，JVM 会确保这个类已经被加载、连接（验证、准备和解析）和初始化。类的加载是指把类的.class 文件中的数据读入到内存中，通常是创建一个字节数组读入.class 文件，然后产生与所加载类对应的 Class 对象。

加载完成后，Class 对象还不完整，所以此时的类还不可用。当类被加载后就进入连接阶段，这一阶段包括验证、准备（为静态变量分配内存并设置默认的初始值）和解析（将符号引用替换为直接引用）三个步骤。最后 JVM 对

类进行初始化，包括：1)如果类存在直接的父类并且这个类还没有被初始化，那么就先初始化父类；2)如果类中存在初始化语句，就依次执行这些初始化语句。

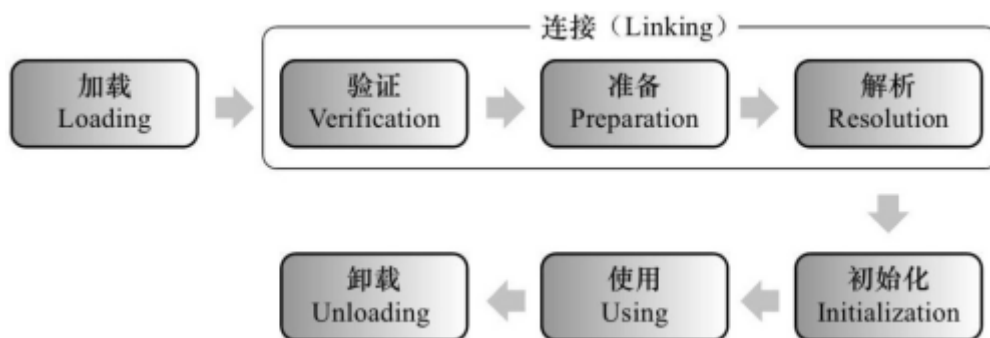
类的加载是由类加载器完成的，类加载器包括：根加载器（Bootstrap）、扩展加载器（Extension）、系统加载器（System）和用户自定义类加载器（java.lang.ClassLoader 的子类）。

从 Java 2 (JDK 1.2) 开始，类加载过程采取了父亲委托机制（PDM）。PDM 更好的保证了 Java 平台的安全性，在该机制中，JVM 自带的Bootstrap 是根加载器，其他的加载器都有且仅有一个父类加载器。类的加载首先请求父类加载器加载，父类加载器无能为力时才由其子类加载器自行加载。JVM 不会向 Java 程序提供对 Bootstrap 的引用。下面是关于几个类加载器的说明：

1. Bootstrap：一般用本地代码实现，负责加载 JVM 基础核心类库（rt.jar）；
2. Extension：从 java.ext.dirs 系统属性所指定的目录中加载类库，它的父加载器是 Bootstrap；
3. System：又叫应用类加载器，其父类是 Extension。它是应用最广泛的类加载器。它从环境变量 classpath 或者系统属性 java.class.path 所指定的目录中记载类，是用户自定义加载器的默认父加载器。

JVM 类加载机制

JVM 类加载机制分为五个部分：加载，验证，准备，解析，初始化，下面我们就分别来看一下这五个过程。



加载

加载是类加载过程中的一个阶段，这个阶段会在内存中生成一个代表这个类的 java.lang.Class 对象，作为方法区这个类的各种数据的入口。注意这里不一定非得要从一个 Class 文件获取，这里既可以从 ZIP 包中读取（比如从 jar 包和 war 包中读取），也可以在运行时计算生成（动态代理），也可以由其它文件生成（比如将 JSP 文件转换成对应的 Class 类）。

验证

这一阶段的主要目的是为了确保 Class 文件的字节流中包含的信息是否符合当前虚拟机的要求，并且不

会危害虚拟机自身的安全。

准备

准备阶段是正式为类变量分配内存并设置类变量的初始值阶段，即在方法区中分配这些变量所使用的内存空间。注意这里所说的初始值概念，比如一个类变量定义为：

实际上变量 `v` 在准备阶段过后的初始值为 0 而不是 8080，将 `v` 赋值为 8080 的 `put static` 指令是程序被编译后，存放于类构造器方法之中。

但是注意如果声明为：`public static final int v = 8080;`

在编译阶段会为 `v` 生成 `ConstantValue` 属性，在准备阶段虚拟机会根据 `ConstantValue` 属性将 `v` 赋值为 8080。

解析

解析阶段是指虚拟机将常量池中的符号引用替换为直接引用的过程。符号引用就是 class 文件中的：

```
public static int v = 8080;
```

实际上变量 `v` 在准备阶段过后的初始值为 0 而不是 8080，将 `v` 赋值为 8080 的 `put static` 指令是程序被编译后，存放于类构造器方法之中。但是注意如果声明为：

在编译阶段会为 `v` 生成 `ConstantValue` 属性，在准备阶段虚拟机会根据 `ConstantValue` 属性将 `v` 赋值为 8080。解析

解析阶段是指虚拟机将常量池中的符号引用替换为直接引用的过程。符号引用就是 class 文件中的：

```
public static final int v = 8080;
```

在编译阶段会为 `v` 生成 `ConstantValue` 属性，在准备阶段虚拟机会根据 `ConstantValue` 属性将 `v` 赋值为 8080。

解析

解析阶段是指虚拟机将常量池中的符号引用替换为直接引用的过程。符号引用就是 class 文件中的：

1. `CONSTANT_Class_info`
 2. `CONSTANT_Field_info`
 3. `CONSTANT_Method_info`
- 等类型的常量。

符号引用

符号引用与虚拟机实现的布局无关，引用的目标并不一定要已经加载到内存中。各种虚拟机实现的内存布局可以各不相同，但是它们能接受的符号引用必须是一致的，因为符号引用的字面量形式明确定义在 Java 虚拟机规范的 Class 文件格式中。

直接引用

直接引用可以是指向目标的指针，相对偏移量或是一个能间接定位到目标的句柄。如果有了直接引用，那引用的目标必定已经在内存中存在。

初始化

初始化阶段是类加载最后一个阶段，前面的类加载阶段之后，除了在加载阶段可以自定义类加载器以外，其它操作都由 JVM 主导。到了初始阶段，才开始真正执行类中定义的 Java 程序代码。

类构造器

初始化阶段是执行类构造器方法的过程。方法是由编译器自动收集类中的类变量的赋值操作和静态语句块中的语句合并而成的。虚拟机会保证子方法执行之前，父类的方法已经执行完毕，如果一个类中没有对静态变量赋值也没有静态语句块，那么编译器可以不为这个类生成 `()` 方法。注意以下几种情况不会执行类初始化：

1. 通过子类引用父类的静态字段，只会触发父类的初始化，而不会触发子类的初始化。
2. 定义对象数组，不会触发该类的初始化。
3. 常量在编译期间会存入调用类的常量池中，本质上并没有直接引用定义常量的类，不会触发定义常量所在的类。
4. 通过类名获取 `Class` 对象，不会触发类的初始化。
5. 通过 `Class.forName` 加载指定类时，如果指定参数 `initialize` 为 `false` 时，也不会触发类初始化，其实这个参数是告诉虚拟机，是否要对类进行初始化。

6. 通过 ClassLoader 默认的 loadClass 方法，也不会触发初始化动作。

什么是类加载器，类加载器有哪些？

实现通过类的权限定名获取该类的二进制字节流的代码块叫做类加载器。

主要有一下四种类加载器：

1. 启动类加载器(Bootstrap ClassLoader)用来加载java核心类库，无法被java程序直接引用。
2. 扩展类加载器(extensions class loader):它用来加载 Java 的扩展库。Java 虚拟机的实现会提供一个扩展库目录。该类加载器在此目录里面查找并加载 Java 类。
3. 系统类加载器 (system class loader)：它根据 Java 应用的类路径 (CLASSPATH) 来加载 Java 类。一般来说，Java 应用的类都是由它来完成加载的。可以通过 `ClassLoader.getSystemClassLoader()`来获取它。
4. 用户自定义类加载器，通过继承 `java.lang.ClassLoader`类的方式实现。

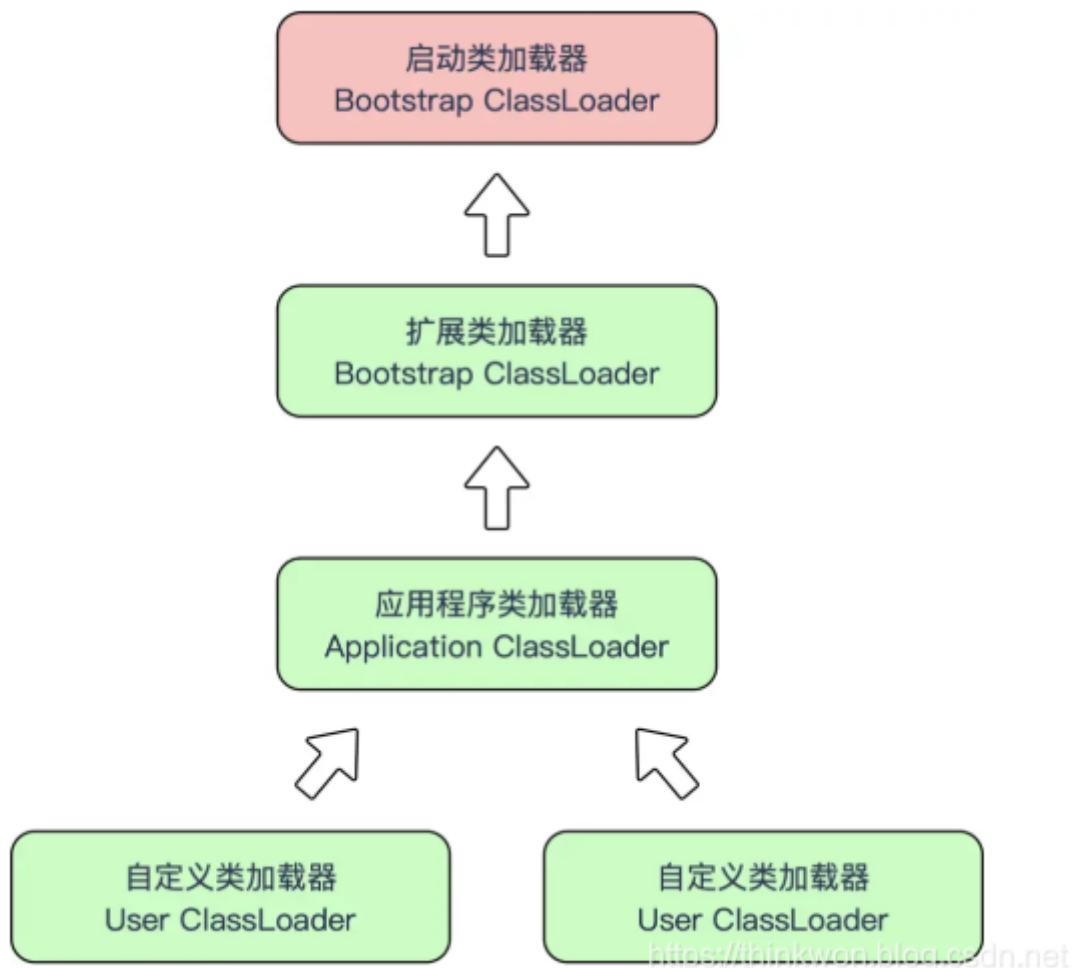
说一下类装载的执行过程？

类装载分为以下 5 个步骤：

- 加载：根据查找路径找到相应的 class 文件然后导入；
- 验证：检查加载的 class 文件的正确性；
- 准备：给类中的静态变量分配内存空间；
- 解析：虚拟机将常量池中的符号引用替换成直接引用的过程。符号引用就理解为一个标示，而在直接引用直接指向内存中的地址；
- 初始化：对静态变量和静态代码块执行初始化工作。

什么是双亲委派模型？

在介绍双亲委派模型之前先说下类加载器。对于任意一个类，都需要由加载它的类加载器和这个类本身一同确立在 JVM 中的唯一性，每一个类加载器，都有一个独立的类名称空间。类加载器就是根据指定全限定名称将 class 文件加载到 JVM 内存，然后再转化为 class 对象。



类加载器分类：

- 启动类加载器（Bootstrap ClassLoader），是虚拟机自身的一部分，用来加载Java_HOME/lib/目录中的，或者被 -Xbootclasspath 参数所指定的路径中并且被虚拟机识别的类库；
- 其他类加载器：
 - 扩展类加载器（Extension ClassLoader）：负责加载lib\ext目录或java. ext. dirs系统变量指定的路径中的所有类库；
 - 应用程序类加载器（Application ClassLoader）。负责加载用户类路径（classpath）上的指定类库，我们可以直接使用这个类加载器。一般情况，如果我们没有自定义类加载器默认就是用这个加载器。

双亲委派模型：如果一个类加载器收到了类加载的请求，它首先不会自己去加载这个类，而是把这个请求委派给父类加载器去完成，每一层的类加载器都是如此，这样所有的加载请求都会被传送到顶层的启动类加载器中，只有当父加载无法完成加载请求（它的搜索范围中没找到所需的类）时，子加载器才会尝试去加载类。

当一个类收到了类加载请求时，不会自己先去加载这个类，而是将其委派给父类，由父类去加载，如果此时父类不能加载，反馈给子类，由子类去完成类的加载。

简单说说你了解的类加载器，可以打破双亲委派么，怎么打破。

思路：先说明一下什么是类加载器，可以给面试官画个图，再说一下类加载器存在的意义，说一下双亲委派模型，最后阐述怎么打破双亲委派模型。

参考的答案：

1) 什么是类加载器？

类加载器 就是根据指定全限定名称将class文件加载到VM内存，转为Class对象。

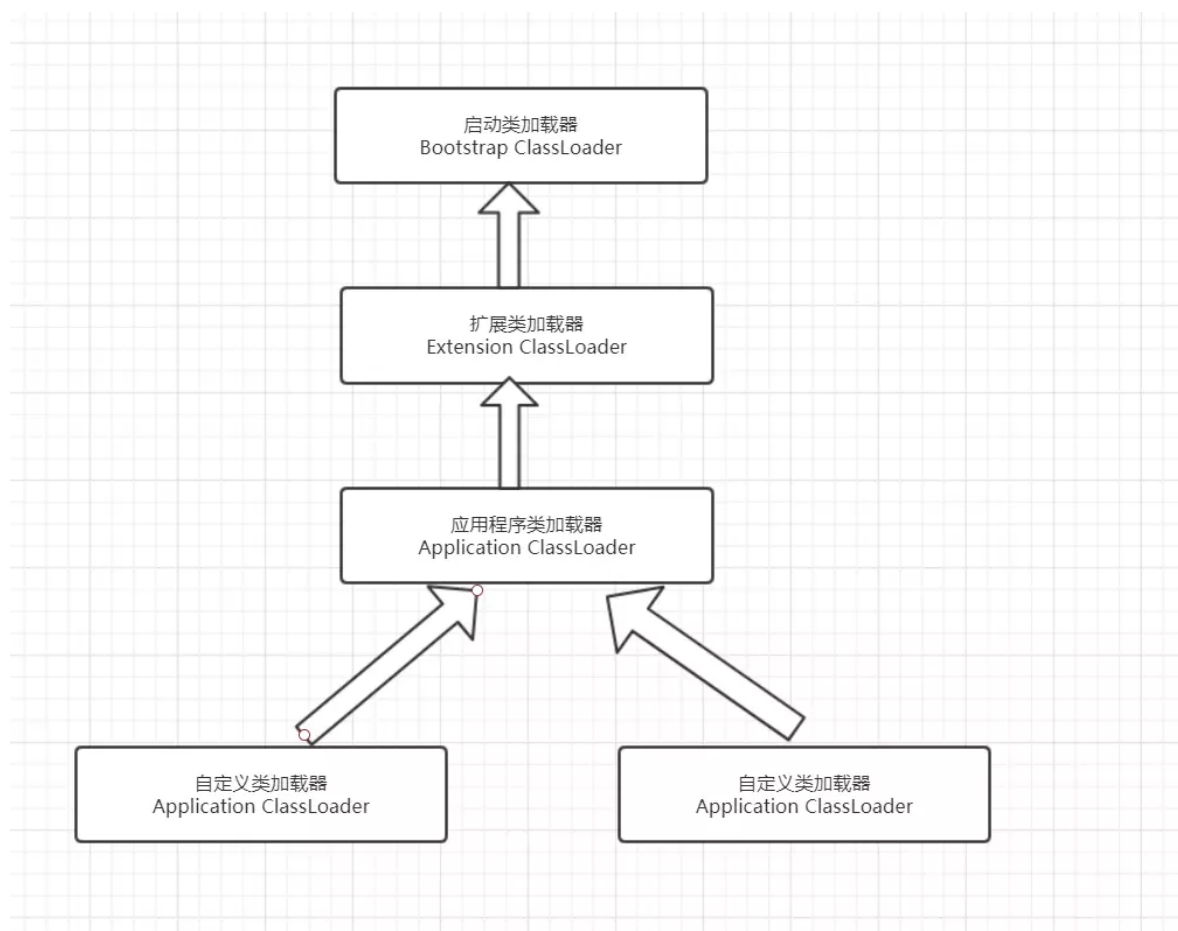
- 启动类加载器（Bootstrap ClassLoader）：由C++语言实现（针对HotSpot），负责将存放在<JAVA_HOME>\lib目录或-Xbootclasspath参数指定的路径中的类库加载到内存中。
- 其他类加载器：由Java语言实现，继承自抽象类ClassLoader。如：
 - 扩展类加载器（Extension ClassLoader）：负责加载<JAVA_HOME>\lib\ext目录或java.ext.dirs系统变量指定的路径中的所有类库。
 - 应用程序类加载器（Application ClassLoader）。负责加载用户类路径（classpath）上的指定类库，我们可以直接使用这个类加载器。一般情况，如果我们没有自定义类加载器默认就是用这个加载器。

2) 双亲委派模型

双亲委派模型工作过程是：

如果一个类加载器收到类加载的请求，它首先不会自己去尝试加载这个类，而是把这个请求委派给父类加载器完成。每个类加载器都是如此，只有当父加载器在自己的搜索范围内找不到指定的类时（即ClassNotFoundException），子加载器才会尝试自己去加载。

双亲委派模型图：



3) 为什么需要双亲委派模型？

在这里，先想一下，如果没有双亲委派，那么用户是不是可以自己定义一个java.lang.Object的同名类，java.lang.String的同名类，并把它放到ClassPath中,那么类之间的比较结果及类的唯一性将无法保证，因此，为什么需要双亲委派模型？防止内存中出现多份同样的字节码

4) 怎么打破双亲委派模型？

打破双亲委派机制则不仅要继承ClassLoader类，还要重写loadClass和findClass方法。

什么是Java虚拟机？为什么Java被称作是“平台无关的编程语言”？

Java虚拟机是一个可以执行Java字节码的虚拟机进程。Java源文件被编译成能被Java虚拟机执行的字节码文件。Java被设计成允许应用程序可以运行在任意的平台，而不需要程序员为每一个平台单独重写或者是重新编译。Java虚拟机让这个变为可能，因为它知道底层硬件平台的指令长度和其他特性。



不怕裁：被毕业3个月，11年经验小伙 0.5个月极速拿offer @公众号 技术自由圈

场景题：说说JVM相关的类的加载、链接和初始化的问题？

一个Java类从字节代码到能够在JVM中被使用，需要经过加载、链接和初始化这三个步骤。

这三个步骤中，对开发人员直接可见的是Java类的加载，通过使用Java类加载器（class loader）可以在运行时刻动态的加载一个Java类；

链接和初始化对开发人员直接可见 不直接可见，因为：链接和初始化是在使用Java类之前会发生的动作，

Java的类加载机制

我们知道一个Java类要想运行，必须由jvm将其装载到内存中才能运行，装载的目的就是把Java字节代码转换成JVM中的java.lang.Class类的对象。

这样Java就可以对该对象进行一系列操作，装载过程有两个比较重要的特征：层次组织结构和代理模式。

层次组织结构指的是每个类加载器都有一个父类加载器，通过getParent()方法可以获取到。

类加载器通过这种父亲-后代的方式组织在一起，形成树状层次结构。

代理模式则 指的是一个类加载器既可以自己完成Java类的定义工作，也可以代理给其它的类加载器来完成。

由于代理模式的存在，启动一个类的加载过程的类加载器和最终 定义这个类的类加载器可能并不是一个。

ClassLoader的加载类过程主要使用loadClass方法，该方法中封装了中加载机制：双亲委派模式。

一般来说，父类优先的策略就足够好了。

在某些情况下，可能需要采取相反的策略，即先尝试自己加载，找不到的时候再代理给父类加载器。

这种做法在Java的Web容器中比较常见，也是Servlet规范推荐的做法。比如，Apache Tomcat为每个Web应用都提供一个独立的类加载器，使用的就是自己优先加载的策略。

IBM WebSphere Application Server则允许Web应用选择类加载器使用的策略。

类加载器的一个重要用途是在JVM中为相同名称的Java类创建隔离空间。

在JVM中，判断两个类是否相同，不仅是根据该类的二进制名称，还需要根据两个类的定义类加载器。

只有两者完全一样，才认为两个类的是相同的。因此，即便是同样的Java字节代码，被两个不同的类加载器定义之后，所得到的Java类也是不同的。

如果试图在两个类的对象之间进行赋值操作，会抛出`java.lang.ClassCastException`。这个特性为同样名称的Java类在JVM中共存创造了条件。

在实际的应用中，可能会要求同一名称的Java类的不同版本在JVM中可以同时存在。通过类加载器就可以满足这种需求。这种技术在OSGi中得到了广泛的应用

Java类的加载过程：

- 1.通过类的全名产生对应类的二进制数据流。（如果没找到对应类文件，只有在类实际使用时才抛出错误。）
- 2.分析并将这些二进制数据流转换为方法区特定的数据结构（这些数据结构是实现有关的，不同JVM有不同实现）。这里处理了部分检验，比如类文件的魔数的验证，检查文件是否过长或者过短，确定是否有父类（除了Object类）。
- 3.创建对应类的`java.lang.Class`实例（注意，有了对应的Class实例，并不意味着这个类已经完成了加载链接！）。

Java类的链接

Java类的链接指的是将Java类的二进制代码合并到JVM的运行状态之中的过程。

在链接之前，这个类必须被成功加载。

链接的过程比加载过程要复杂很多，这是实现Java的动态性的重要一步！

分为三部分：verification（检测），preparation（准备）和resolution（解析）

1.verification（检测）：

验证是用来确保Java类的二进制表示在结构上是完全正确的。如果验证过程出现错误的话，会抛出`java.lang.VerifyError`错误。

linking的resolve会把类中成员方法、成员变量、类和接口的符号引用替换为直接引用，而在这之前，需要检测被引用的类型正确性和接入属性是否正确（就是public,private的问题）诸如，检查final class 没有被继承，检查静态变量的正确性等等。

验证是用来确保Java类的二进制表示在结构上是完全正确的。如果验证过程出现错误的话，会抛出`java.lang.VerifyError`错误。

2.preparation（准备）：

准备过程则是创建Java类中的静态域，并将这些域的值设为默认值。'准备过程并不会执行代码。

在一个Java类中会包含对其它类或接口的形式引用，包括它的父类、所实现的接口、方法的形式参数和返回值的Java类等。

对类的成员变量分配空间。

虽然有初始值，但这个时候不会对他们进行初始化（因为这里不会执行任何Java代码）。

具体如下：

所有原始类型的值都为0。如 float: 0f, int: 0, boolean: 0(注意 boolean 底层实现大多使用 int)，

引用类型则为 null。

值得注意的是，JVM 可能会在这个时期给一些有助于程序运行效率提高的数据结构分配空间。

3.resolution（解析）：

解析的过程就是确保这些被引用的类能被正确的找到。解析的过程可能会导致其它的Java类被加载。

为类、接口、方法、成员变量的**符号引用定位直接引用**，完成内存结构的布局。

这一步是可选的。可以在符号引用第一次被使用时完成，即所谓的**延迟解析**(late resolution)。

但对用户而言，这一步永远是延迟解析的，即使运行时执行 early resolution，但程序不会显示的在第一次判断出错误时抛出错误，而会在对应的类第一次主动使用的时候抛出错误！

另外，这一步与之后的类初始化是不冲突的，并非一定要所有的解析结束以后才执行类的初始化。

不同的 JVM 实现不同。

看下面一段代码：

```
public class LinkTest {
    public static void main(String[] args) {
        ToBeLinked toBeLinked = null;
        System.out.println("Test link.");
    }
}
```

类 LinkTest引用了类ToBeLinked，但是并没有真正使用它，只是声明了一个变量，并没有创建该类的实例或是访问其中的静态域。

如果把编译好的ToBeLinked的Java字节代码删除之后，再运行LinkTest，程序不会抛出错误。

这是因为ToBeLinked类没有被真正用到。

链接策略使得ToBeLinked类不会被加载，因此也不会发现ToBeLinked的Java字节代码实际上是不存在的。

如果把代码改成ToBeLinked toBeLinked = new ToBeLinked();之后，再按照相同的方法运行，就会抛出异常了。

因为这个时候ToBeLinked这个类被真正使用到了，会需要加载这个类。

Java类的初始化

开发 Java 时，接触最多的是对象的初始化。实际上类也是有初始化的。相比对象初始化，类的初始化机制要简单不少。

类的初始化也是延迟的，直到类第一次被主动使用(active use)，JVM 才会初始化类。

当一个Java类第一次被真正使用到的时候，JVM会进行该类的初始化操作。初始化过程的主要操作是执行静态代码块和初始化静态域。在一个类被初始化之前，它的直接父类也需要被初始化。

但是，一个接口的初始化，不会引起其父接口的初始化。在初始化的时候，会按照源代码中从上到下的顺序依次执行静态代码块 和初始化静态域。

```
public class StaticTest {
    public static int x = 10;
    public static void main(String[] args) {
        System.out.println(Y); //输出60
    }
    static {
        x = 30;
    }
    public static int Y = x * 2;
}
```

在上面的代码中，在初始化的时候，静态域的初始化和静态代码块的执行会从上到下依次执行。因此变量X的值首先初始化成10，后来又被赋值成30；而变量Y的值则被初始化成60。

类的初始化分两步：

- 1.如果基类没有被初始化，初始化基类。
- 2.有类构造函数，则执行类构造函数。

类构造函数是由Java编译器完成的。

它把类成员变量的初始化和 static 区间的代码提取出，放到一个方法中。

这个方法不能被一般的方法访问（注意，static final 成员变量不会在此执行初始化，它一般被编译器生成 constant 值）。

同时，中是不会显示的调用基类的的，因为 1 中已经执行了基类的初始化。该初始化过程是由 jvm 保证线程安全的。。

Java类和接口的初始化只有在特定的时机才会发生，这些时机包括：

创建一个Java类的实例。如

```
MyClass obj = new MyClass()
```

调用一个Java类中的静态方法。如

```
MyClass.sayHello()
```

给Java类或接口中声明的静态域赋值。如

```
MyClass.value = 10
```

访问Java类或接口中声明的静态域，并且该域不是常值变量。如

```
int value = MyClass.value
```

在顶层Java类中执行assert语句。

通过Java反射API也可能造成类和接口的初始化。需要注意的是，当访问一个Java类或接口中的静态域的时候，只有真正声明这个域类或接口才会被初始化。考虑下面的代码：

```
class B {
    static int value = 100;
    static {
        System.out.println("Class B is initialized."); //输出
    }
}
class A extends B {
    static {
        System.out.println("Class A is initialized."); //不会输出
    }
}
public class InitTest {
    public static void main(String[] args) {
        System.out.println(A.value); //输出100
    }
}
```

```
}
```

在上述代码中，类InitTest通过A.value引用了类B中声明的静态域value。由于value是在类B中声明的，只有类B会被初始化，而类A则不会被初始化。



不怕裁：经过指导后，9年小伙伴被毕业 没offer，年薪 近100W @公众号 技术自由圈

JVM调优

JVM调优连环炮：常用的JVM启动参数有哪些？

JVM可配置参数已经达到1000多个，其中GC和内存配置相关的JVM参数就有600多个。

但在绝大部分业务场景下，常用的JVM配置参数也就10来个。

```
# JVM启动参数不换行
# 设置堆内存
-Xmx4g -Xms4g
# 指定GC算法
-XX:+UseG1GC -XX:MaxGCPauseMillis=50
# 指定GC并行线程数
-XX:ParallelGCThreads=4
# 打印GC日志
-XX:+PrintGCDetails -XX:+PrintGCDateStamps
# 指定GC日志文件
-Xloggc:gc.log
# 指定Meta区的最大值
-XX:MaxMetaspaceSize=2g
# 设置单个线程栈的大小
-Xss1m
# 指定堆内存溢出时自动进行Dump
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/usr/local/

# 指定默认的连接超时时间
-Dsun.net.client.defaultConnectTimeout=2000

-Dsun.net.client.defaultReadTimeout=2000
# 指定时区
-Duser.timezone=GMT+08
# 设置默认的文件编码为UTF-8
-Dfile.encoding=UTF-8
# 指定随机数熵源(Entropy source)
-Djava.security.egd=file:/dev/./urandom
```

附带：JVM上的随机数与熵池策略

在apache-tomcat官方文档提到了一些启动时的优化项，其中一项是关于随机数生成时，采用的“熵源”(entropy source)的策略。

他提到tomcat7的session id的生成主要通过 `java.security.SecureRandom` 生成随机数来实现，随机数算法使用的是“SHA1PRNG”

```
private String secureRandomAlgorithm = "SHA1PRNG";
```

在sun/oracle的jdk里，这个算法的提供者在底层依赖到操作系统提供的随机数据，

在linux上，与之相关的是 `/dev/random` 和 `/dev/urandom`，对于这两个设备块的描述以前也见过讨论随机数的文章，wiki中有比较详细的描述，摘抄过来，先看 `/dev/random`：

在读取时，`/dev/random`设备会返回小于熵池噪声总数的随机字节。

`/dev/random`可生成高随机性的公钥或一次性密码本。

若熵池空了，对`/dev/random`的读操作将会被阻塞，直到收集到了足够的环境噪声为止

而 `/dev/urandom` 则是一个非阻塞的发生器：

`dev/random`的一个副本是`/dev/urandom`（“unlocked”，非阻塞的随机数发生器），

它会重复使用熵池中的数据以产生伪随机数据。

这表示对`/dev/urandom`的读取操作不会产生阻塞，但其输出的熵可能小于`/dev/random`的。

它可以作为生成较低强度密码的伪随机数生成器，不建议用于生成高强度长期密码。

另外wiki里也提到了为什么linux内核里的随机数生成器采用SHA1散列算法而非加密算法，是为了避开法律风险(密码出口限制)。

回到tomcat文档里的建议，采用非阻塞的熵源(entropy source)，通过java系统属性来设置：

```
-Djava.security.egd=file:/dev/./urandom
```

这个系统属性egd表示熵收集守护进程(entropy gathering daemon)，但这里值为何要在 `dev` 和 `random` 之间加一个点呢？

是因为一个[jdk的bug](#)，在这个bug的连接里有人反馈及时对 `securerandom.source` 设置为 `/dev/urandom`

它也仍然使用的 `/dev/random`，有人提供了变通的解决方法，其中一个变通的做法是对 `securerandom.source`设置为 `/dev/./urandom` 才行。

也有人评论说这个不是bug，是有意为之。

举例：常用的 JVM 调优的参数都有哪些？

- `-Xms2g`：初始化堆大小为 2g；
- `-Xmx2g`：堆最大内存为 2g；
- `-XX:NewRatio=4`：设置年轻的和老年代的内存比例为 1:4；
- `-XX:SurvivorRatio=8`：设置新生代 Eden 和 Survivor 比例为 8:2；
- `-XX:+UseParNewGC`：指定使用 ParNew + Serial Old 垃圾回收器组合；
- `-XX:+UseParallelOldGC`：指定使用 ParNew + ParNew Old 垃圾回收器组合；
- `-XX:+UseConcMarkSweepGC`：指定使用 CMS + Serial Old 垃圾回收器组合；

- -XX:+PrintGC: 开启打印 gc 信息;
- -XX:+PrintGCDetails: 打印 gc 详细信息。

说说：你知道的几种主要的JVM参数

思路：可以说一下堆栈配置相关的，垃圾收集器相关的，还有一下辅助信息相关的。

参考答案：

1) 堆栈配置相关

```
java -Xmx3550m -Xms3550m -Xmn2g -Xss128k -XX:MaxPermSize=16m -XX:NewRatio=4 -XX:SurvivorRatio=4 -XX:MaxTenuringThreshold=0
```

-Xmx3550m： 最大堆大小为3550m。

-Xms3550m： 设置初始堆大小为3550m。

-Xmn2g： 设置年轻代大小为2g。

-Xss128k： 每个线程的堆栈大小为128k。

-XX:MaxPermSize： 设置永久代大小为16m

-XX:NewRatio=4: 设置年轻代（包括Eden和两个Survivor区）与年老代的比值（除去永久代）。

-XX:SurvivorRatio=4： 设置年轻代中Eden区与Survivor区的大小比值。设置为4，则两个Survivor区与一个Eden区的比值为2:4，一个Survivor区占整个年轻代的1/6

-XX:MaxTenuringThreshold=0： 设置垃圾最大年龄。如果设置为0的话，则年轻代对象不经过Survivor区，直接进入年老代。

2) 垃圾收集器相关

```
-XX:+UseParallelGC-XX:ParallelGCThreads=20-XX:+UseConcMarkSweepGC -XX:CMSFullGCsBeforeCompaction=5-XX:+UseCMSCompactAtFullCollection:
```

-XX:+UseParallelGC： 选择垃圾收集器为并行收集器。

-XX:ParallelGCThreads=20： 配置并行收集器的线程数

-XX:+UseConcMarkSweepGC： 设置年老代为并发收集。

-XX:CMSFullGCsBeforeCompaction： 由于并发收集器不对内存空间进行压缩、整理，所以运行一段时间以后会产生“碎片”，使得运行效率降低。此值设置运行多少次GC以后对内存空间进行压缩、整理。

-XX:+UseCMSCompactAtFullCollection： 打开对年老代的压缩。可能会影响性能，但是可以消除碎片

3) 辅助信息相关

```
-XX:+PrintGC-XX:+PrintGCDetails
```

-XX:+PrintGC 输出形式：

[GC 118250K->113543K(130112K), 0.0094143 secs] [Full GC 121376K->10414K(130112K), 0.0650971 secs]

-XX:+PrintGCDetails 输出形式:

[GC [DefNew: 8614K->781K(9088K), 0.0123035 secs] 118250K->113543K(130112K), 0.0124633 secs] [GC [DefNew: 8614K->8614K(9088K), 0.0000665 secs][Tenured: 112761K->10414K(121024K), 0.0433488 secs] 121376K->10414K(130112K), 0.0436268 secs

说一下 JVM 调优的工具?

JDK 自带了很多监控工具，都位于 JDK 的 bin 目录下，其中最常用的是 jconsole 和 jvisualvm 这两款视图监控工具。

- jconsole：用于对 JVM 中的内存、线程和类等进行监控；
- jvisualvm：JDK 自带的全能分析工具，可以分析：内存快照、线程快照、程序死锁、监控内存的变化、gc 变化等。

JVM调优连环炮：调优命令有哪些？

Sun JDK监控和故障处理命令有jps jstat jmap jhat jstack jinfo

1. jps, JVM Process Status Tool,显示指定系统内所有的HotSpot虚拟机进程。
2. jstat, JVM statistics Monitoring是用于监视虚拟机运行时状态信息的命令，它可以显示出虚拟机进程中的类装载、内存、垃圾收集、JIT编译等运行数据。
3. jmap, JVM Memory Map命令用于生成heap dump文件
4. jhat, JVM Heap Analysis Tool命令是与jmap搭配使用，用来分析jmap生成的dump，jhat内置了一个微型的HTTP/HTML服务器，生成dump的分析结果后，可以在浏览器中查看
5. jstack, 用于生成java虚拟机当前时刻的线程快照。
6. jinfo, JVM Configuration info 这个命令作用是实时查看和调整虚拟机运行参数

调优工具

常用调优工具分为两类,jdk自带监控工具：jconsole和jvisualvm，第三方有：MAT(Memory AnalyzerTool)、GChisto。

1. jconsole, Java Monitoring and Management Console是从java5开始，在JDK中自带的java监控和管理控制台，用于对JVM中内存，线程和类等的监控
2. jvisualvm, jdk自带全能工具，可以分析内存快照、线程快照；监控内存变化、GC变化等。
3. MAT, Memory Analyzer Tool, 一个基于Eclipse的内存分析工具，是一个快速、功能丰富的Javaheap分析工具，它可以帮助我们查找内存泄漏和减少内存消耗
4. GChisto, 一款专业分析gc日志的工具

JVM调优连环炮：设置堆内存XMX应该考虑哪些因素？

需要根据系统的配置来确定，要给操作系统和JVM本身留下一定的剩余空间。
推荐配置系统或容器里可用内存的 70-80% 最好。

JVM调优连环炮：假设物理内存是8G，设置多大堆内存比较合适？

比如说系统有 8G 物理内存，系统自己可能会用掉一点，大概还有 7.5G 可以用，那么建议配置 `-Xmx6`
说明：

$7.5G * 0.8 = 6G$

如果知道系统里有明确使用堆外内存的地方，还需要进一步降低这个值。

XM设置的值与JVM进程所占用的内存有什么关系？

JVM总内存=栈+堆+非堆+直接内存+Native

整个堆大小=年轻代大小 + 年老代大小 + 永久代大小.

设置:

`java -Xmx3550m -Xms3550m -Xmn2g -Xss128k`

- ①、**-Xmx3550m**:设置JVM最大可用内存为3550M.
- ②、**-Xms3550m**:设置JVM初始内存为3550m.此值可以设置与-Xmx相同,以避免每次垃圾回收完成后JVM重新分配内存.
- ③、**-Xmn2g**:设置年轻代大小为2G.永久代一般固定大小为64m,所以增大年轻代后,将会减小年老代大小.此值对系统性能影响较大,Sun官方推荐配置为整个堆的3/8.
- ④、**-Xss128k**:设置每个线程的堆栈大小.JDK5.0以后每个线程堆栈大小为1M,以前每个线程堆栈大小为256K.更具应用的线程所需内存大小进行调整.在相同物理内存下,减小这个值能生成更多的线程.但是操作系统对一个进程内的线程数还是有限制的,不能无限生成,经验值在3000~5000左右.

`java -Xmx3550m -Xms3550m -Xss128k -XX:NewRatio=4 -XX:SurvivorRatio=4 -`

`XX:MaxPermSize=16m -XX:MaxTenuringThreshold=0`

- ①、**-XX:NewRatio=4**:设置年轻代(包括Eden和两个Survivor区)与年老代的比值(除去永久代).设置为4,则年轻代与年老代所占比值为1:4,年轻代占整个堆栈的1/5
- ②、**-XX:SurvivorRatio=4**:设置年轻代中Eden区与Survivor区的大小比值.设置为4,则两个Survivor区与一个Eden区的比值为2:4,一个Survivor区占整个年轻代的1/6
- ③、**-XX:MaxPermSize=16m**:设置永久代大小为16m.
- ④、**-XX:MaxTenuringThreshold=0**:设置垃圾最大年龄.如果设置为0的话,则年轻代对象不经过Survivor区,直接进入年老代.对于年老代比较多的应用,可以提高效率.如果将此值设置为一个较大值,则年轻代对象会在Survivor区进行多次复制,这样可以增加对象再年轻代的存活时间,增加在年轻代即被回收的概率.

JVM调优连环炮：怎样开启GC日志？

一般来说，JDK8及以下版本通过以下参数来开启GC日志：

```
-XX:+PrintGCDetails  
-XX:+PrintGCDateStamps  
-Xloggc:gc.log
```

如果是在JDK9及以上的版本，则格式略有不同：

```
-# JDK 11 环境，输出 info 级别的 GC 日志  
java -Xms512m -Xmx512m  
-Xlog:gc*=info:file=gc.log:time:filecount=0  
demo.jvm0204.GCLogAnalysis
```

JVM调优连环炮：怎么打出线程栈信息。

思路：可以说一下jps, top, jstack这几个命令，再配合一次排查线上问题进行解答。

参考答案：

- 输入jps，获得进程号。
- top -Hp pid 获取本进程中所有线程的CPU耗时性能
- jstack pid命令查看当前java进程的堆栈状态
- 或者 jstack -l > /tmp/output.txt 把堆栈信息打到一个txt文件。
- 可以使用fastthread 堆栈定位, fastthread.io/

JVM调优连环炮：请指定使用G1垃圾收集器来启动 程序

```
java -XX:+UseG1GC  
-Xms4g  
-Xmx4g  
-Xloggc:gc.log  
-XX:+PrintGCDetails  
-XX:+PrintGCDateStamps Hello
```

Java8默认使用的垃圾收集器是什么？

Java8版本的Hotspot JVM，默认情况下使用的是并行垃圾收集器（Parallel GC）。

其 他厂商提供的JDK8基本上也默认使用并行垃圾收集器。

Java11的默认垃圾收集器是什么？

Java9之后，官方JDK默认使用的垃圾收集器是G1。

常见的垃圾收集器有哪些？

常见的垃圾收集器包括：

串行垃圾收集器： -XX:+UseSerialGC

并行垃圾收集器：-XX:+UseParallelGC

CMS垃圾收集器：-XX:+UseConcMarkSweepG

G1垃圾收集器：-XX:+UseG1GC

什么是串行垃圾收集？

就是只有单个worker线程来执行GC工作。

什么是并行垃圾收集？

并行垃圾收集，是指使用多个GC worker 线程并行地执行垃圾收集，能充分利用多核CPU的能力，缩短垃圾收集的暂停时间。

除了单线程的GC，其他的垃圾收集器，比如 PS，CMS， G1等新的垃圾收集器都使

用了多个线程来并行执行GC工作。

什么是并发垃圾收集器？

并发垃圾收集器，是指在应用程序在正常执行时，有一部分GC任务，由GC线程在应用线程一起并发执行。

例如 CMS/G1的各种并发阶段。

什么是增量式垃圾收集？

首先， G1的堆内存不再单纯划分为年轻代和老年代，而是划分为多个（通常是 2048个）可以存放对象的小块堆区域（smaller heap regions）。

每个小块，可能一会定义成 Eden 区，一会指定为 Survivor 区或者 Old 区。

这样划分之后，使得 G1 不必每次都去回收整个堆空间，而是以增量的方式来进行处理：每次只处理一部分内存块，称为此次 GC 的回收集（collection set）。

下一次GC时在本次的基础上，再选定一定的区域来进行回收。

增量式垃圾收集的好处是大大降低了单次GC暂停的时间。

什么是年轻代？

年轻代是分来垃圾收集算法中的一个概念，相对于老年代而言，年轻代一般包括：

新生代，Eden区。

存活区，执行年轻代GC时，用存活区来保存活下来的对象。

存活区也是年轻代的一部分，但一般有2个存活区，所以可以来回倒腾。

什么是GC停顿(GC pause)?

因为GC过程中，有一部分操作需要等所有应用线程都到达安全点，暂停之后才能执行，这时候就叫做GC停顿，或者叫做GC暂停。

GC停顿与STW停顿有什么区别？

这两者一般可以认为就是同一个意思。

如果CPU使用率突然飙升，你会怎么排查？

缺乏经验的话，针对当前问题，往往需要使用不同的工具来收集信息，例如：收集不同的指标（CPU，内存，磁盘IO，网络等等）

然后进行专项分析：

- 分析应用日志
- 分析GC日志
- 获取线程转储并分析
- 获取堆转储来进行分析



涨薪200%：经过指导后，3年经验小伙 喜提外企offer，涨了200% @公众号 技术自由圈

聊聊：什么是逃逸分析

和下面的题目一致

聊聊：Java中的对象一定是在堆上分配的吗？

对于JVM“逃逸分析”特性，也是近年来大厂面试、高薪面试的常见面试题。

和逃逸分析有关的常见面试题：

- Java中的对象一定是在堆上分配的吗？

JVM的内存分配策略

首先回顾一下JVM的内存分配策略。

JVM的内存包括方法区、堆、虚拟机栈、本地方法栈、程序计数器。

一般情况下JVM运行时的数据都是存在栈和堆上的。

- 栈用来存放一些基本变量和对象的引用，
- 堆用来存放数组和对象，也就是说new出来的实例。

但是：凡事都有例外

随着JIT编译器的发展和逃逸分析的技术成熟，栈上分配、标量替换等优化技术，使对象不一定全都分配在堆中。

so，重点来了：对象不一定全都分配在堆中

在JVM的实现中，为了提高JVM的性能和节省内存空间，JVM提供了一种叫做“逃逸分析”的特性，逃逸分析是目前Java虚拟机中比较前沿的优化技术，也是JIT中一个很重要的优化技术。

“逃逸分析”的直观认知

直接说这些概念，确实有点晕啊，那我们就来“逃逸分析”的直观认知。

三连问：“逃逸分析”的本质是什么呢？为什么叫逃逸？逃逸的范围是什么呢？

“逃逸分析”的本质：

主要就是分析对象的动态作用域，分析一个对象的动态作用域是否会逃逸出方法范围、或者线程范围。

简单的说：

如果一个对象在一个方法内定义，如果被方法外部的引用所指向，那认为它逃逸了。

否者，这个对象，没有发生逃逸。

尼恩提示：

看起来一个 很牛逼轰轰的概念，其实就是这么简单。

逃逸分析的概念

先以官方的形式来说下什么是逃逸分析。

逃逸分析就是：一种确定指针动态范围的静态分析，它可以分析在程序的哪些地方可以访问到指针。

在JVM的即时编译语境下，逃逸分析将判断新建的对象是否逃逸。

即时编译判断对象是否逃逸的依据：

一种是对象是否被存入堆中（静态字段或者堆中对象的实例字段），另一种就是对象是否被传入未知代码。

逃逸分析的类型

逃逸分析的类型有两种：

- 方法逃逸
- 线程逃逸

什么是：方法逃逸(对象逃出当前方法)：

当一个对象在方法里面被定义后，它可能被外部方法所引用，例如作为调用参数传递到其它方法中。

什么是：线程逃逸((对象逃出当前线程)：

这个对象甚至可能被其它线程访问到，例如赋值给类变量或可以在其它线程中访问的实例变量

方法逃逸

当一个对象在方法里面被定义后，它可能被外部方法所引用，这种称为方法逃逸

方法逃逸包括：

- 通过调用参数，将对象地址传递到其他方法中，
- 对象通过return语句将对象指针，返回给其他方法
- 等等

我们可以用下面的代码来表示这个现象。

```
//StringBuffer对象发生了方法逃逸
public static StringBuffer createStringBuffer(String s1, String s2) {
    StringBuffer sb = new StringBuffer();
    sb.append(s1);
    sb.append(s2);
    return sb;
}
```

上面的例子中，StringBuffer 对象通过return语句返回。

StringBuffer sb是一个方法内部变量，上述代码中直接将sb返回，这样这个StringBuffer有可能被其他方法所改变，这样它的作用域就不只是在方法内部，虽然它是一个局部变量，称其逃逸到了方法外部。

甚至还有可能被外部线程访问到，譬如赋值给类变量或可以在其他线程中访问的实例变量，称为线程逃逸。

不直接返回 StringBuffer，那么StringBuffer将不会逃逸出方法。

具体的代码如下：

```
// 非方法逃逸
public static String createString(String s1, String s2) {
    StringBuffer sb = new StringBuffer();
    sb.append(s1);
    sb.append(s2);
    return sb.toString();
}
```

可以看出，想要逃逸方法的话，需要让对象本身被外部调用，或者说，对象的指针，传递到了 **方法之外**。

线程逃逸

当一个对象可能被外部线程访问到，这种称为线程逃逸。

例如赋值给类变量或可以在其它线程中访问的实例变量

逃逸分析后的代码优化

从不逃逸、方法逃逸到线程逃逸，称为对象由低到高的不同逃逸程度。

如果能证明一个对象不会逃逸到方法或线程之外（换句话说别的方法或线程无法通过任何途径访问到这个对象），或者逃逸程度比较低（只逃逸出方法而不会逃逸出线程），则可能为这个对象实例采取不同程度的优化。

通过逃逸分析，编译器会对代码进行优化。

如果能够证明一个对象不会逃逸到方法外或者线程外，或者说逃逸程度比较低，则可以对这个对象采用不同程度的优化：

- 栈上分配
- 标量替换
- 消除同步锁

栈上分配

对象不分配在堆上，而是分配在栈内存上。

前提是：对象没有逃逸，很老实的话。

具体而言：

完全不会逃逸的局部变量和不会逃逸出的线程对象，采用栈上分配，

对于发生逃逸的、不老实的对象，才使用 堆上分配。

栈上分配可以快速地在栈帧上创建和销毁对象，不用再将对象分配到堆空间，可以有效地减少 JVM 垃圾回收的压力。

标量替换

一个对象可能不需要作为一个连续的存储空间，也能被访问到，那么对象的部分可以不存储的在连续的内存，而是存可以打散存储，甚至部分存储或者打散在CPU寄存器中。

通过逃逸分析确定该对象不会被外部访问后，JVM判断对象是否可以被进一步分解，如果对象可以打散为 变量，则JVM不会创建该对象，而是化整为零， 将该对象成员变量分解若干个被这个方法使用的成员变量，

JVM将一个大的对象打散成若干变量的过程，叫做**标量替换**，也称之为 **分离对象**。

比如如下代码：

```
public static void main(String[] args) {
    alloc();
}

private static void alloc() {
    Point point = new Point (1,2) ;
    System.out.println("point.x="+point.x+"; point.y="+point.y);
}

class Point{
    private int x;
    private int y;
}
```

从以上代码可以看出，Point对象并没有逃逸出alloc方法，并且Point对象是可以拆解成标量的。

此时，JIT就会不会直接创建Point对象，而是直接使用两个标量int x,int y来替代Point对象

为啥要 **化整为零** 呢？

因为 栈空间是非常有限的，很多的场景下，一个线程的栈空间就是1M的大小。

标量替换之后的成员变量，可以选择在栈帧分配，也可以就近在寄存器上分配空间，这样就不会因为没有一大块连续空间导致对象内存不够分配。

开启标量替换参数-XX:+EliminateAllocations，JDK7之后默认开启。

总之：

当JVM通过逃逸分析，确定要将对象分配到栈上时，即时编译可以将对象打散，将对象替换为一个个很小的局部变量，我们将这个打散的过程叫做标量替换。

将对象替换为一个个局部变量后，就可以非常方便的在栈上进行分配了。

同步锁消除

如果JVM通过逃逸分析，发现一个对象只能从一个线程被访问到，则访问这个对象时，可以不加同步锁。

具体来说：

如果同步块所使用的锁对象通过这种分析后，发现只能够被一个线程访问，根本用不着同步，

那么，JIT编译器在编译这个同步块的时候就会取消对这部分代码的同步锁。

所以：如果程序中使用了synchronized内置锁，则JVM会将synchronized内置锁消除。

注意：

这种情况针对的是synchronized锁，而对于非内置锁，比如 Lock 显示锁、CAS乐观锁等等，则JVM并不能消除。

要开启同步消除，需要加加上两个JVM启动选项：

```
-XX:+EliminateLocks  
-XX:+DoEscapeAnalysis
```

-XX:+EliminateLocks启动选项，表示启动同步锁消除。

-XX:+DoEscapeAnalysis 选项，表示启动逃逸分析。

因为同步锁消除依赖逃逸分析，所以同时要打开 -XX:+DoEscapeAnalysis 选项。

逃逸分析相关JVM参数

```
-XX:+DoEscapeAnalysis 开启逃逸分析  
-XX:+PrintEscapeAnalysis 开启逃逸分析后，可通过此参数查看分析结果。  
-XX:+EliminateAllocations 开启标量替换  
-XX:+EliminateLocks 开启同步消除  
-XX:+PrintEliminateAllocations 开启标量替换后，查看标量替换情况。
```

逃逸分析的底层原理是什么呢？

在Java的编译体系中，一个Java的源代码文件变成计算机可执行的机器指令的过程中，需要经过两段编译：

第一段编译，指前端编译器把**.java文件**转换成**.class文件**(字节码文件)。

前端编译器产品可以是JDK的javac、Eclipse JDT中的增量式编译器。

第二编译阶段，JVM通过解释字节码将其翻译成对应的机器指令，逐条读入字节码，逐条解释翻译成机器码。

很显然，由于有一个解释的中间过程，其执行速度必然会比可执行的二进制字节码程序慢很多。

这就是传统的JVM的解释器（Interpreter）的功能。

如何去掉中间商，提升效率？

为了解决这种效率问题，引入了JIT（即时编译器，Just In Time Compiler）技术。

引入了JIT技术后，Java程序还是通过解释器进行解释执行，也就是说，主体还是解释执行，只是局部去掉中间环节。

怎么做局部去掉中间环节呢？

当JVM发现某个方法或代码块运行特别频繁的时候，就会认为这是“热点代码”（Hot Spot Code）。

然后JIT会把部分“热点代码”翻译成本地机器相关的机器码，并进行优化，然后再把翻译后的机器码缓存起来，以备下次使用。

把翻译后的机器码缓存在哪里呢？这个缓存，叫做Code Cache。可见，JVM和WEB应用实现高并发的手段是类似的，还是使用了缓存架构。

当JVM下次遇到相同的热点代码时，跳过解释的中间环节，直接从Code Cache加载机器码，直接执行，无需再编译。

所以，JIT总的目标是发现热点代码，热点代码变成了提升性能的关键，hotspot JVM的名字，就是这么来的，把识别热点代码，写在名字上，作为毕生的追求。

所以，JVM总的策略为：

- 对于占据大部分的不常用的代码，我们无需耗费时间将其编译成机器码，而是采取解释执行的方式运行；
- 另一方面，对于仅占据小部分的热点代码，我们则可以将其编译成机器码，以达到理想的运行速度。

JIT(即时编译)的出现与 解释器的区别

- (1) 解释器是将字节码解释为机器码，下次即使遇到相同的字节码，仍会执行重复的解释。
- (2) JIT 是将一些字节码编译为机器码，并存入Code Cache，下次遇到相同的代码，直接执行，无需再编译。
- (3) 解释器是将字节码解释为针对所有平台都通用的机器码。
- (4) JIT 会根据平台类型，生成平台特定的机器码。

JVM包含多个即时编译器，主要有C1和C2，还有个Graal (实验性的)。

多个即时编译器，都会对字节码进行优化并生成机器码

C1会对字节码进行简单可靠的优化，包括方法内联、去虚拟化、冗余消除等，编译速度较快，可以通过-client强制指定C1编译

C2会对字节码进行激进优化，包括分支频率预测、同步擦除等，

可以通过-server强制指定C2编译

JVM 将执行状态分成了 5 个层次：

- 0 层，解释执行 (Interpreter)
- 1 层，使用 C1 即时编译器编译执行 (不带 profiling)
- 2 层，使用 C1 即时编译器编译执行 (带基本的 profiling)
- 3 层，使用 C1 即时编译器编译执行 (带完全的 profiling)
- 4 层，使用 C2 即时编译器编译执行

JVM不会直接启用C2，而是先通过C1编译收集程序的运行状态，再根据分析结果判断是否启用C2。

分层编译模式下，虚拟机执行状态由简到繁、由快到慢分为5层

在编译期间，JIT 除了对 热点代码做缓存提速，会对代码做很多优化。

其中有一部分优化的目的就是**减少内存堆分配压力**，其中JIT优化中一种重要的技术叫做逃逸分析。根据逃逸分析，即时编译器会在编译过程中对代码做如下优化：

- 锁消除：当一个锁对象只被一个线程加锁时，即时编译器会把锁去掉
- 栈上分配：当一个对象没有逃逸时，会将对象直接分配在栈上，随着线程回收，由于JVM的大量代码都是堆分配，所以目前JVM不支持栈上分配，而是采用标量替换
- 标量替换：当一个对象没有逃逸时，会将当前对象打散成若干局部变量，并分配在虚拟机栈的局部变量表中

关于面试题答案：Java中的对象一定是在堆上分配的吗？

答：不一定。

如果满足了逃逸分析的条件，一个对象，完全可以在栈上分配。**减少堆内存的分配和GC压力。**

由于栈内存有限，所以，如果对象符合标量替换的条件，**进一步为对象来一次化整为零的手术**

标量替换具体的做法是：

JVM会将对象进一步打散，将对象分解为若干个被这个方法使用的成员变量，**从而，达到更好的利用栈内存和寄存器的目标**



转架构：通过尼恩的指导，小伙伴 **转架构** 后年薪从40W **涨到70W** @公众号 技术自由圈

聊聊：什么是即时编译？

1、Java与编译相关的三个概念

首先了解三个概念

- 前端编译
- 解释执行
- 编译执行

1.1、前端编译

编译器（javac）将源文件（.java）编译成java字节码文件（.class）的步骤是前端编译。

1.2、解释执行

在JVM加载字节码后，每次执行方法调用时，JVM都会将字节码翻译成机器码，然后执行机器码，这个过程叫**解释执行**。

解释执行为了提升启动效率，并没有在启动时将字节码全部翻译成机器码，所以**启动效率较高**

但是，

由于字节码不行执行，要机器码才能执行，所以，执行时要进行字节码翻译，所以**执行效率相对较低**

1.3、编译执行

什么是编译执行？

与解释执行相反，JVM加载字节码的时候，直接将字节码转换为机器码，在执行方法调用时直接执行机器码，不需要做翻译工作，这样的过程叫**编译执行**。

编译执行的问题是什么呢？

和解释执行相反，编译执行在启动时将字节码全部翻译成机器码，所以启动效率较低

但是

编译执行时省去了翻译的步骤，所以执行效率相对较高

2、即时编译

解释执行的特点是：启动效率高、执行效率低

编译执行的特点是：启动效率低、执行效率高

JVM如何实现双高呢？实现，启动效率高、执行效率也高。

2.1、如何平衡启动速度和执行的速度

为了平衡启动和执行的效率，JVM结合解释执行和编译执行的特点，进行综合和平衡，形成了一种折中的性能优化策略。

JVM以解释执行，编译执行为辅，达到启动速度和执行速度的最优化。

那些代码需要编译执行呢？热点代码。

JVM 并不对全部代码进行编译执行，仅仅对热点代码进行编译优化，这样的执行过程叫即时编译

2.2、什么是“热点代码”（Hot Spot Code)?

当JVM发现某个方法或代码块运行特别频繁的时候，就会认为这是“热点代码” (Hot Spot Code)。

然后JIT会把部分“热点代码”翻译成本地机器相关的机器码，并进行优化，然后再把翻译后的机器码缓存起来，以备下次使用。

把翻译后的机器码缓存在哪里呢？这个 机器码缓存，叫做 Code Cache。

当JVM下次遇到相同的热点代码时，跳过解释的中间环节，直接从 Code Cache加载机器码，直接执行，无需再编译。

所以，JIT总的目标是发现热点代码，热点代码变成了提升性能的关键，

Java官方给自家开源的JVM取名为hotspot JVM，也就是这么来的。

说得更大白话一点，Java官方把识别“热点代码” (Hot Spot Code)这个任务，写在名字上，作为毕生的追求。

所以，JVM总的策略为：

- 对于占据大部分的不常用的代码，我们无需耗费时间将其编译成机器码，而是采取解释执行的方式运行；
- 另一方面，对于仅占据小部分的热点代码，我们则可以将其编译成机器码，并且缓存起来，后面省略编译的过程，直接从缓存当中取得机器码，从而达到理想的运行速度。

2.2 架构师视角解读：JVM中的缓存架构

“热点代码” (Hot Spot Code)编译后，放入到 Code Cache中，当JVM下次遇到相同的热点代码时，直接从 Code Cache加载机器码，跳过中间的编译环节，无需再编译。

从架构的角度来说，这里应用到了尼恩常说的3高架构的三板斧之一：**缓存架构**。

可见，JVM和WEB应用实现高并发的手段，是非常类似的。JVM为了实现高性能和高并发，也会使用缓存架构。

在《Java高并发核心编程 卷2 加强版中》，大家知道，CPU内部，为了提升性能，也用了缓存架构，并且是多级缓存架构。

同时，在缓存架构中，热点数据非常重要。

在WEB应用的缓存架构中，识别热点数据 (HotKey)，是提升三级缓存命中率的 核心环节。我们会通过有效的 识别组件，识别其中的HotKey。

这种HotKey的理论和思想，在JVM中的缓存架构也是想通的。

JVM中的缓存架构中，不能什么都缓存，需要缓存的同样是他的 HotKey，这里叫做 Hot Spot Code，仅仅换了一个名字而已。

所以，正如尼恩 三高架构的思想所云，缓存架构 都是相通的：

- 高并发WEB三级缓存架构里边，有hotkey和本地高速缓存
- JVM架构里边有Hot Spot和 Code Cache

3.1、即时编译器

JVM包含多个即时编译器，主要有C1和C2，还有个Graal (实验性的)。

多个即时编译器，都会对字节码进行优化并生成机器码。

但是不同的即时编译器，优化的程度不同：C1会对字节码进行简单可靠的优化，C2会对字节码进行激进优化。

- C1会对字节码进行简单可靠的优化，包括方法内联、去虚拟化、冗余消除等，编译速度较快，可以通过-client强制指定C1编译
- C2会对字节码进行激进优化，包括分支频率预测、同步擦除等，可以通过-server强制指定C2编译

如果没有强制指定，JVM默认会使用分层编译模式。

3.2、分层编译模式

JVM不会直接启用C2，而是先通过C1编译收集程序的运行状态，再根据分析结果判断是否启用C2。

分层编译模式下，虚拟机执行状态由简到繁、由快到慢分为5层

- 0 层，解释执行 (Interpreter)
- 1 层，使用 C1 即时编译器编译执行，无profiling
- 2 层，使用 C1 即时编译器编译执行，带基本的 profiling（仅方法调用次数及循环回边执行次数的 profiling）
- 3 层，使用 C1 即时编译器编译执行，带完全的 profiling
- 4 层，使用 C2 即时编译器编译执行

什么是profiling？

profiling是C0/C1在编译过程中收集程序执行状态的过程

收集的执行状态记录为profile (概述/ 印象)，包括分支跳转频率、是否出现过空值和异常等，主要用于触发C2编译。

如何实现呢？

profiling 是指在程序执行过程中，JVM织入的一些协助收集数据的辅助代码，这些织入的辅助代码，收集能够反映程序执行状态的数据。这里所收集的数据我们称之为程序的 profile。

profiling 在思想上，非常类似于 Java Agent 的字节码增强。只是 Java Agent 的字节码增强发生在字节码 层面，profiling 的指令织入，发生在 机器码的层面。

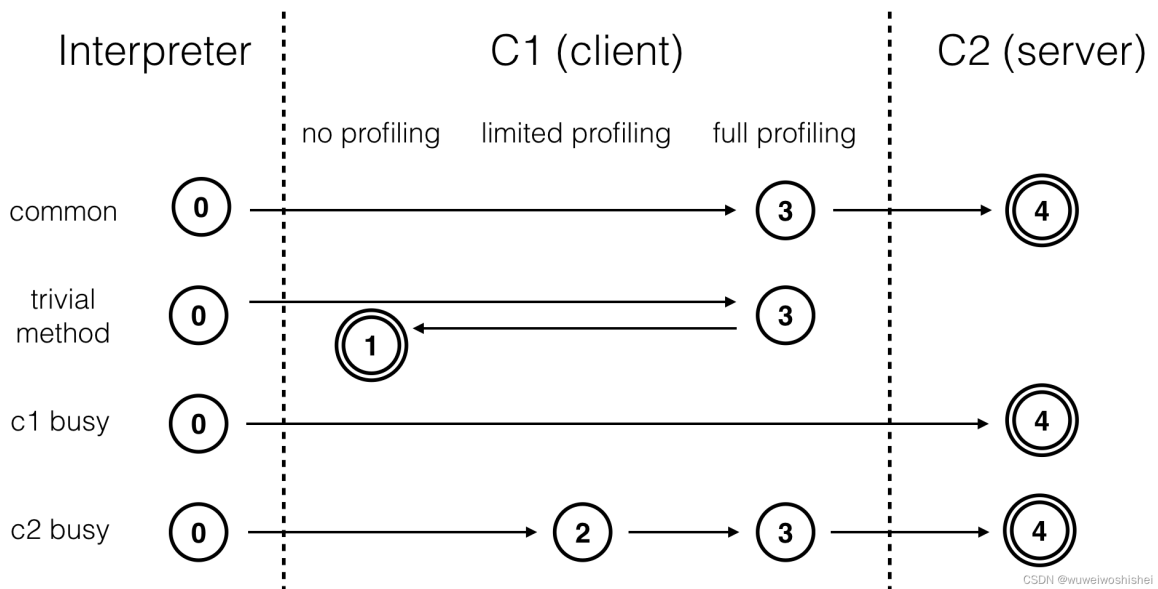
至于profiling 的指令织入，织入了啥？咱们不需要深究，除非不想再从事Java开发，而是去用C++ 开发JVM。否则，不用关系 织入了啥，了解这个思想就行。

什么是完全的 profiling？

除了 基本的 profiling（仅方法调用次数及循环回边执行次数的profiling）外，

还包括分支 profile（针对分支跳转字节码，包括跳转次数和不跳转次数）以及receiver type（针对成员方法调用或类检测，如checkcast，instanceof，aastore字节码）的类型profile

分层编译5层执行状态之间的关系，具体如下图：



第一条执行路径，指的是通常情况下，一个方法先被解释执行（level 0），然后被C1编译（level 3），再然后被得到profile数据的C2编译（level 4）

第二条执行路径，指的是编译对象非常简单的情况下，如getter和setter，虚拟机认为通过C1编译或通过C2编译并无区别，就会在3层编译后，直接由C1编译且不插入profiling代码（level 1）。

第三条执行路径，指的是C1繁忙时，JVM会在解释执行时收集profiling，而后直接由4层的C2编译。

第四条执行路径，指的是C2繁忙时，先由2层的C1编译再由3层的C1编译，这样可以减少方法在3层的执行时间，最终再交给C2执行。

注意：这个收集的动作，叫做**profiling**，这个收集的结果，叫做**profile**。

分层编译中的0层、2层和3层都会进行profiling，织入的一些协助收集数据的辅助代码，收集能够反映程序执行状态的数据。

其中，最为基础的便是2层进行的profiling，它只需要统计方法的调用次数以及循环回边的执行次数，当统计之和超过阈值就会触发**即时编译**。

所以，方法的调用次数以及循环回边的执行次数达到阈值，这部分的代码，就会被识别成为——热点代码。

0层和3层相较于2层复杂一些，需要收集用于4层C2编译的数据。

比如说分支跳转字节码的分支profile（branch profile），包括跳转次数和不跳转次数，

比如receiver类型profile（receiver type profile）：非私有实例方法调用指令、强制类型转换checkcast指令、类型测试instanceof指令，和引用类型的数组存储aastore指令。

上述数据分为两大类：分支profile和类型profile。

根据图片中的编译途径可知，分层编译下，无论何种情况，大概率都要进行分支profile和类型profile的收集。

但是，需要注意的是，有利必有弊：分支profile和类型profile的收集，将给应用程序带来不少的性能开销。

据统计，正是因为这部分额外的profiling，导致的一个结果是：

3层C1代码（带完全的profile）的性能比2层C1代码（带基本的profiling）低30%。

那么这些耗费巨大代价收集而来的 profile 具体有什么作用呢？

答案是：

C2 可以根据收集得到的数据进行猜测，从而作出比较激进的优化。

当然：4层的C2代码有一个前提，就是 假设接下来的执行，同样会C1 代码按照所收集的 profile 进行。

3.3、profiling

3.4、触发C2的时机

当方法调用次数profile或循环次数profile达到阈值时，会触发即时编译

阈值不仅需要通过-XX:TierXInvocationThreshold、-XX:TierXMINInvocationThreshold和-XX:TierXCompileThreshold设置，还跟待编译方法的数目和编译线程的总数有关。

编译线程的数量是处理器动态指定的，参数为-XX:+CICompilerCountPerCPU默认开启，

可以通过-XX:+CICompilerCount=N强制指定编译线程总数。

JVM会将这些线程以1:2的比例分配给C1和C2

3.5、去优化

去优化是当C2编译的机器码假设失败时，将即时编译切换回解释执行的过程

在C2编译生成机器码时，会在假设失败的一端设置一条指令。

当假设失败时，调用指令让JVM将栈帧的方法返回地址从机器码所在的本地内存地址改回运行时常量池中的方法地址，并进行解释执行

4、方法内联

在即时编译方法时，将目标方法的方法体取代方法调用的过程叫方法内联，增加了编译的代码量，但是降低了方法调用带来的入栈出栈的成本

4.1、静态方法内联

即时编译器会根据方法调用层数，目标方法的调用次数及字节码大小等决定该方法是否允许被内联

- -XX:CompileCommand配置中的inline指令指定的方法会被强制内联，dontinline和exclude指定的方法始终不会被内联
- @ForceInline注解的jdk内部方法会被强制内联，@DontInline注解jdk内部方法始终不会被内联
- 方法的符号引用未被解析、目标方法所在类未被初始化、目标方法是native方法，都会导致方法无法内联
- C2默认不支持9层以上的方法调用（-XX:MaxInlineLevel），以及1层的直接递归调用（-XX:MaxRecursiveInlineLevel）
- 自动拆箱总会被内联，Throwable类的方法不能被其他类内联等

4.2、动态方法内联

即时编译器需要将动态绑定的虚方法转化为直接调用，才能进行方法内联，这样的过程叫虚方法的去虚化

- 根据字节码生成的IR图确定调用者类型的过程叫基于类型推导的完全去虚化

- 根据JVM中已加载的类找到接口的唯一实现的过程叫基于类层次分析的完全去虚化
- 根据编译时收集的类型profile，依次匹配方法调用者的动态类型与profile中的类型

5、逃逸分析

当方法内部定义的对象被外部代码引用时，称为该对象逃逸，JVM对对象的分析过程叫逃逸分析

根据逃逸分析，即时编译器会在编译过程中对代码做如下优化：

- 锁消除：当一个锁对象只被一个线程加锁时，即时编译器会把锁去掉
- 栈上分配：当一个对象没有逃逸时，会将对象直接分配在栈上，随着线程回收，由于JVM的大量代码都是堆分配，所以目前JVM不支持栈上分配，而是采用标量替换
- 标量替换：当一个对象没有逃逸时，会将当前对象打散成若干局部变量，并分配在虚拟机栈的局部变量表中

6、即时编译的其他优化

字段读取优化：缓存多次读取的数据，减少出入栈次数

```
public String register(User user,String username,String password){
    user.username = username;
    return user.username + password;
}

class User{
    private String username;
}
```

```
public String register(User user,String username){
    String s = user.username; //user.username被缓存成了s
    s = username;
    return s + password;
}
```

字段存储优化：将被覆盖的赋值操作优化掉，减少无用的入栈

```
private void test(){
    int a = 1;
    a = 2;
}

private void test(){
    int a = 2; //a=1被优化掉了
}
```

循环无关代码外提：避免重复执行表达式，减少出入栈次数

```
private void test(String s){
    String password;
    for (int i=0;i<10;i++){
        password = s.replaceAll("/", "");
        System.out.println(i);
    }
}
```

```

    }
}

private void test(String s){
    String password = s.replaceAll("/", ""); //与循环无关的代码被编译器外提了
    for (int i=0;i<10;i++){
        System.out.println(i);
    }
}

```

循环展开：将相同的循环逻辑多次重复在一次迭代中，以减少循环次数

```

private void test(int[] arr){
    int sum=0;
    for (int i=0;i<8;i++){
        sum +=arr[i];
    }
}

private void test(int[] arr){
    int sum=0;
    for (int i=0;i<8;i+=2){ //循环次数减少
        sum +=arr[i];
        sum +=arr[i+1]; //重复循环体内相同逻辑
    }
}

```

循环的自动向量化：对循环中地址连续的数组操作，会按顺序批量出入栈（这段是伪代码）

```

private void test(int[] arr1,int[] arr2){
    for (int i=0;i<arr1.length;i++){
        arr1[i] = arr2[i];
    }
}

private void test(int[] arr1,int[] arr2){
    for (int i=0;i<arr1.length;i+=4){
        arr1[i:i+4] = arr2[i:i+4]; //可以看成是在循环展开的基础上，将多个数组一块出入
栈
    }
}

```



美团一面：OOM后，JVM一定会退出吗？为什么？

说在前面

在40岁老架构师 尼恩的[读者交流群](#)(50+)中，最近有小伙伴拿到了一线互联网企业如美团、拼多多、极兔、有赞、希音的面试资格，遇到一几个很重要的面试题：

- OOM后，JVM一定会退出吗？为什么？

生产环境，很多小伙伴，也遇到过OOM后，JVM会退出的问题。

这里尼恩给大家做一下系统化、体系化的梳理，使得大家可以充分展示一下大家雄厚的“技术肌肉”，让面试官爱到“不能自己、口水直流”。

也一并把这个题目以及参考答案，收入咱们的《[尼恩Java面试宝典](#)》V70版本，供后面的小伙伴参考，提升大家的 3高 架构、设计、开发水平。

最新《尼恩 架构笔记》《尼恩高并发三部曲》、《尼恩Java面试宝典》的PDF文件，请通过公众号（技术自由圈）获取。

问题背景

问题是，咱们常常说：发生OOM，程序就会挂。

很多情况是：发生OOM了，JVM没有挂。

回顾一下OOM与异常

来看一下 OutOfMemoryError，说到底，OutOfMemoryError 也只是一个java中的异常而已，

OutOfMemoryError 属于Error一系非检查异常，其继承关系如下

```
Object
  Throwable
    Error
      VirtualMachineError
        OutOfMemoryError
```

再看看，堆内存不够与 OutOfMemoryError 异常的关系

线程发生OutOfMemoryError，首先是堆空间不够了，然后再由jvm在申请分配空间的的时候，在调用上抛出OOM异常。

申请内存的线程，会像处理普通的其他异常一样，处理OutOfMemoryError。

线程是资源调度的基本单位，Java在设计线程时充分考虑了线程的独立性。

在异常方面，线程也保持了线程异常的独立性。

在线程执行中，如果发生的异常，都由线程进行独立的处理，而不是也不会抛出到其它的线程。这就是保证了这种线程的独立性。

从线程的实现维度，也可以看到异常处理的策略。

线程Thread里边，最终会执行内部target对象的run方法，也就是java.lang.Runnable接口实现方法，线程通过其run方法运行，方法签名如下：

```
public abstract void run();
```

注意这个方法，run方法不能声明抛出任何检查异常（checked exception）。因此在线程方法执行中发生的任何检查异常，必须在线程中处理。

线程拿到异常，有两种处理方式：

- 捕获并且处理异常，线程继续执行
- 线程停止执行

默认异常处理器

如果没有被捕获

除了检查异常，java中还有非检查异常（unchecked exception），这种异常无需显式声明也能沿着方法调用链向上抛出。

线程对于这种未处理的异常，提供了默认异常处理器：

```
/**
 * Dispatch an uncaught exception to the handler. This method is
 * intended to be called only by the JVM. (将未被捕获的异常分发给处理器。这个方法只被JVM调用)
 */
private void dispatchUncaughtException(Throwable e) {
    getUncaughtExceptionHandler().uncaughtException(this, e);
}
```

Thread的init()方法线程至少有一个默认异常处理器，兜底的异常处理器是当前线程父线程的线程组ThreadGroup，可以看到线程组是有能力处理异常的：

```
public class ThreadGroup implements Thread.UncaughtExceptionHandler {}
```

线程通过这两种机制，保证内部发生的异常，在线程内解决，而不会抛出给启动线程的外部线程。

JVM退出条件

java虚拟机退出的条件是：JVM 不存在非守护线程（前台线程），JVM就会退出。

线程发生未处理的异常（未处理异常由默认异常处理器处理）会导致线程结束，而线程结束了，如果还有非守护线程（前台线程），JVM也不会退出。

OOM也是一种异常，它的发生也不会导致JVM退出。

所以，OOM 与JVM的退出，没有很强的关系。

以下实例说明：

实例一：线程OOM，JVM不一定退出

实例二：线程池OOM，JVM不一定退出

```
emoos> src > main > java > com > crazymakercircle > error > OOMObject > main
OOMObject.java x
2 usages
class OOMObject {
    1 usage
    private final Byte[] toLeak;
    1 usage
    public OOMObject() {
        toLeak = new Byte[1024 * 1024];
    }
    // 为快速发生oom, 设置堆大小; VM args: -Xms20m -Xmx20m
    public static void main(String[] args) throws InterruptedException {
        Thread thread = new Thread(() -> {
            List<OOMObject> list = new ArrayList<>();
            while (true) {
                list.add(new OOMObject());
            }
        });
        thread.setUncaughtExceptionHandler(new Thread.UncaughtExceptionHandler() {
            @Override
            public void uncaughtException(Thread t, Throwable e) {
                e.printStackTrace();
                System.out.println("这里是没有捕获的处理 =====> " + t.getId() + " ==> " + e.getLocalizedMessage());
            }
        });
        thread.start();
        while (true) {
            System.out.println(Thread.currentThread().getName() + " 我还行...");
            Thread.sleep( millis: 1000L);
            System.out.println(thread.getName() + " 的状态: " + thread.getState());
        }
    }
}
```

Debugger: OOMObject ×

Debugger Console

main 我还行...

Thread-0 的状态: RUNNABLE

main 我还行...

Thread-0 的状态: RUNNABLE

main 我还行...

Thread-0 的状态: RUNNABLE

main 我还行...

Thread-0 的状态: RUNNABLE

main 我还行...

这里是没捕获的处理 ===== 22=== Java heap space

java.lang.OutOfMemoryError: Create breakpoint : Java heap space

at com.crazymakercircle.error.OOMObject.<init>(OOMObject.java:8)

at com.crazymakercircle.error.OOMObject.lambda\$main\$0(OOMObject.java:15)

at com.crazymakercircle.error.OOMObject\$\$Lambda\$1/1406718218.run(Unknown Source) <1 internal line>

Thread-0 的状态: TERMINATED

main 我还行...

Thread-0 的状态: TERMINATED

main 我还行...

Thread-0 的状态: TERMINATED

main 我还行...

Thread-0 的状态: TERMINATED

main 我还行...

Thread-0 的状态: TERMINATED

main 我还行...

Disconnected from the target VM, address: '127.0.0.1:57653', transport: 'socket'

领取《4000 页尼恩Java 面试宝典》

公众号: 技术自由圈

CSDN @40岁资深老架构师尼恩

线程中发生OOM异常，和发生其他异常一样，只是那个线程终止了，但是不影响其他线程，thread-0 线程线程OOM，也不会导致JVM退出。

实例二：线程池OOM，JVM不一定退出

```
class OOMThreadPool {
    private final Byte[] toLeak;

    public OOMThreadPool() {
        toLeak = new Byte[1024 * 1024];
    }

    static final Thread[] t = new Thread[1];
    static ThreadPoolExecutor threadPool = new ThreadPoolExecutor(1, 1, 5,
        TimeUnit.SECONDS, new ArrayBlockingQueue<>(9),

        new ThreadFactory() {
            public Thread newThread(Runnable r) {
                t[0] = new Thread(r);
                t[0].setDaemon(false);
                t[0].setPriority(Thread.NORM_PRIORITY);
                t[0].setUncaughtExceptionHandler(new
Thread.UncaughtExceptionHandler() {
                    @Override
                    public void uncaughtException(Thread t, Throwable e) {
                        e.printStackTrace();
                        System.out.println(t.getName() + " 的状态: " +
t.getState());
                        System.out.println("这里是没有捕获的处理 ==> " +
t.getId() + " ==> " + e.getLocalizedMessage());
                    }
                });
                return t[0];
            }
        },
        new ThreadPoolExecutor.DiscardOldestPolicy()) {

        @Override
        protected void afterExecute(Runnable r, Throwable t) {
            System.out.println(Thread.currentThread().getName() + " 任务执行完成，
但是线程不会结束");
            if (null != t) {
                System.out.println(Thread.currentThread().getName() + "任务异常
了");
                t.printStackTrace();
            }
        }
    };

    // 为快速发生oom，设置堆大小；VM args: -Xms10m -Xmx10m
    public static void main(String[] args) throws InterruptedException {
        List<OOMThreadPool> list = new LinkedList<>();
```

```
Runnable target = () -> {
    System.out.println(Thread.currentThread().getName() + " 开始了");
    try {
        while (true) {
            list.add(new OOMThreadPool());
        }
    } catch (Throwable throwable) {
        throwable.printStackTrace();
    }
};

threadPool.submit(target);
while (true) {
    System.out.println(Thread.currentThread().getName() + " 我还行...");
    System.out.println(t[0].getName() + " 的状态: " + t[0].getState());
    Thread.sleep(1000L);
}
}
```

在线程池中，thread-0 线程抛出OOM 后线程的task任务结束，thread-0 线程并没有结束，而是可以接收新的任务。当然，main线程依旧会循环打印"我还行....."。

啰嗦一下：在线程池中，thread-0 线程 当前的task任务结束了，但是 线程没有结束，还是可以执行新任务的。

The screenshot shows an IDE with a Java project named 'thread-demos'. The main file is 'ThreadPoolExecutor.java'. The console output shows the following sequence of events:

- The thread 'Thread-0' starts in a 'RUNNABLE' state.
- The thread 'Thread-0' is repeatedly shown in a 'RUNNABLE' state.
- The thread 'Thread-0' is shown in a 'WAITING' state.
- A 'java.lang.OutOfMemoryError: Java heap space' exception is thrown.
- The exception message indicates that the thread 'Thread-0' is in a 'WAITING' state.

The exception message is: `java.lang.OutOfMemoryError: Java heap space`

The stack trace shows the exception was thrown from the `main` method of the `ThreadPoolExecutor` class, specifically at line 12 and line 59.

The console output also shows the thread 'Thread-0' in a 'WAITING' state.

OOM与JVM退出的关系

什么时候发生OOM、JVM才退出呢？

- 场景1：所有的非守护线程由于申请不到内存而OOM，所有非守护线程退出，JVM退出，这个属于主动退出

OOM的发生表示了此刻JVM堆内存告罄，不能分配出更多的资源，或者GC回收效率不可观。

一个线程的OOM，在一定程度的并发下，若此时其他线程（含非守护线程）也需要申请堆内存，那么其他线程也会因为申请不到内存而OOM，甚至连锁反应导致整个JVM的退出。

- 场景1：OOM溢出，说明内存耗尽，如果操作系统内存耗尽，就会发生OOM killer(Out Of Memory killer)，干掉JVM进程，导致被动退出

Linux 内核有个机制叫OOM killer(Out Of Memory killer)，该机制会监控那些占用内存过大，尤其是瞬间占用内存很快的进程，然后防止内存耗尽而自动把该进程杀掉。内核检测到系统内存不足、挑选并杀掉某个进程的过程可以参考内核源代码linux/mm/oom_kill.c，当系统内存不足的时候，out_of_memory()被触发，然后调用select_bad_process()选择一个“bad”进程杀掉。如何判断和选择一个“bad”进程呢？linux选择“bad”进程是通过调用oom_badness()，挑选的算法和想法都很简单很朴实：最bad的那个进程就是那个最占用内存的进程。

说在最后

OOM相关面试题，是非常常见的面试题。

以上的答案，如果大家能对答如流，如数家珍，基本上面试官会被你震惊到、吸引到。

最终，让面试官爱到“不能自己、口水直流”。offer，也就来了。

学习过程中，如果有啥问题，大家可以来找 40岁老架构师尼恩交流。



转架构：通过尼恩的指导，小伙伴 转架构 后年薪从40W 涨到70W @公众号 技术自由圈

美团面试：Oracle JDK那么好，为何要用Open JDK？

说在前面

在40岁老架构师 尼恩的**读者交流群**(50+)中，最近有小伙伴拿到了一线互联网企业如阿里、滴滴、极兔、有赞、希音、百度、网易、美团的面试资格，遇到很多很重要的面试题：

- 既然 Oracle JDK 这么好，那为什么还要有 OpenJDK？

最近有小伙伴在面试美团，又遇到了相关的面试题。小伙伴懵了，因为JDK都是架构师定的，他从来没有全面去了解过，支支吾吾的说了几句，面试官不满意，面试挂了。

所以，尼恩给大家做一下系统化、体系化的梳理，使得大家内力猛增，可以充分展示一下大家雄厚的“技术肌肉”，**让面试官爱到“不能自己、口水直流”**，然后实现“offer直提”。

完整的答案 参见这个链接：

[美团面试：Oracle JDK那么好，为何要用Open JDK？](#)

面试真题：你们用什么版本的jdk？聊一聊不同版本的jdk的新特性？

说在前面

在40岁老架构师 尼恩的**读者交流群**(50+)中，最近有小伙伴拿到了一线互联网企业如美团、拼多多、极兔、有赞、希音的面试资格，遇到一几个很重要的面试题：

- 你们用什么版本的jdk？
- 为啥你们还在用java8？
- 聊一聊不同版本的jdk的新特性？
- Oracle JDK 和 OpenJDK 的对比
-

这里尼恩给大家做一下系统化、体系化的梳理，使得大家可以充分展示一下大家雄厚的“技术肌肉”，**让面试官爱到“不能自己、口水直流”**。

也一并把这个题目以及参考答案，收入咱们的《[尼恩Java面试宝典](#)》V87版本，供后面的小伙伴参考，提升大家的 3高 架构、设计、开发水平。

最新《尼恩 架构笔记》《尼恩高并发三部曲》《尼恩Java面试宝典》的PDF文件，请关注公众号【技术自由圈】领取，暗号：领电子书

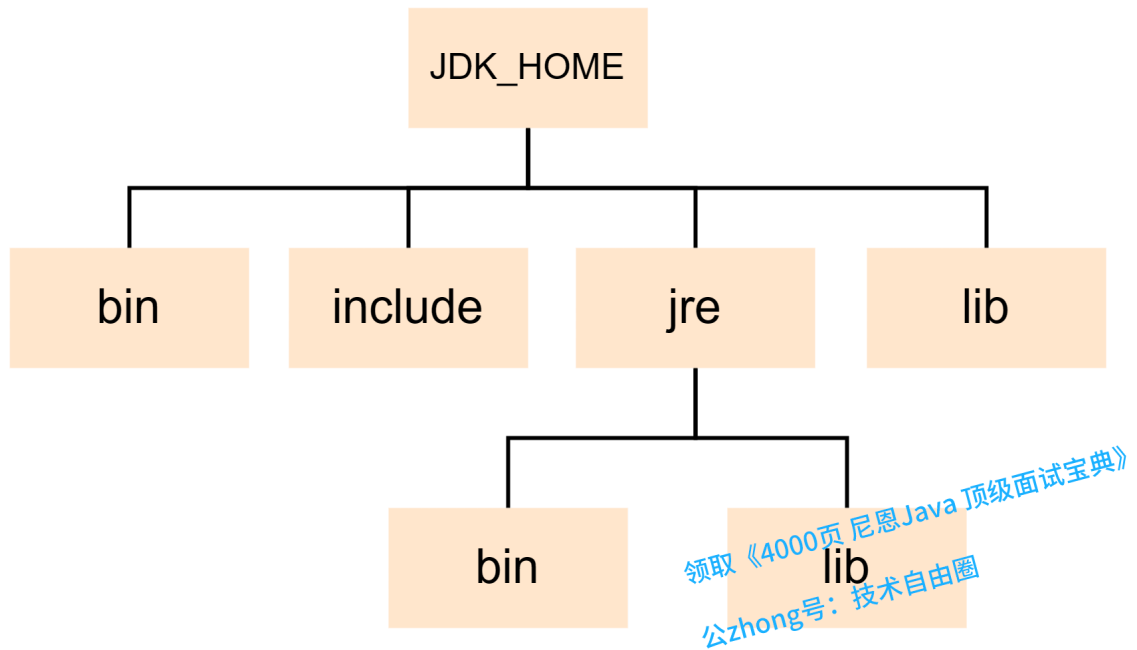
完整的答案 参见这个链接：

[美团面试：你们用什么版本的jdk？聊一聊不同版本的jdk的新特性？](#)

jdk 和jre的改变

在jdk SE 9 之前，jdk构建系统用于生成两种类型的运行时映射**JRE**(java运行时环境)和**JDK**(java开发工具包)。

JRE 是java SE 平台的完整实现，JDK包含了JRE 和开发工具和类库。

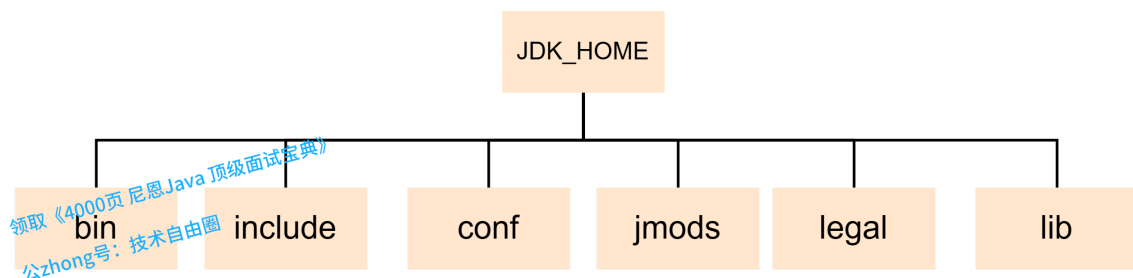


jdk9之前的目录

目录说明:

- **bin目录**用于包含命令行开发和调试工具，如javac, jar和javadoc。它还用于包含Java命令来启动Java应用程序。
- **include目录**包含在编译本地代码时使用的C/C++头文件。
- **lib目录**包含JDK工具的几个JAR和其他类型的文件。它有一个tools.jar文件，其中包含javac编译器的Java类。
- **jre\bin目录**包含基本命令，如Java命令。在Windows平台上，它包含系统的运行时动态链接库(DLL)
- **jre\lib目录**包含用户可编辑的配置文件，如.properties和.policy文件。包含几个JAR。rt.jar文件包含运行时的Java类和资源文件。

Java SE 9调整了JDK的目录层次结构如下图所示:



jdk9以后目录

最大的区别: 从jdk9开始删除了JDK和JRE之间的区别, 在jdk9中, 没有名为jre的目录。

目录说明:

- `bin`目录包含所有命令。在Windows平台上，它继续包含系统的运行时动态链接库。
- `conf`目录包含用户可编辑的配置文件，例如以前位于jre\lib目录中的.properties和.policy文件。
- `include`目录包含要在以前编译本地代码时使用的C/C++头文件。它只存在于JDK中。
- `jmods`目录包含JMOD格式的平台模块。创建自定义运行时映像时需要它。它只存在于JDK中。
- `legal`目录包含法律声明。

- lib目录包含非Windows平台上的动态链接本地库。其子目录和文件不应由开发人员直接编辑或使用。

阿里面试：说说 jvm 锁的 膨胀过程？锁内存怎么变化的？

尼恩说在前面

在40岁老架构师 尼恩的**读者交流群**(50+)中，很多小伙伴拿到了一线互联网企业如得物、阿里、滴滴、极兔、有赞、希音、百度、网易、美团的面试机会，遇到很多很重要的面试题：

- 1.请解释 JVM 偏向锁、轻量级锁、自旋锁、重量级锁什么？
- 2.请介绍一下什么是synchronized的自旋锁、偏向锁、轻量级锁、重量级锁？
- 3.请介绍一下 jvm 内置锁 的膨胀过程？
- 4.请介绍一下 jvm 内置锁 的膨胀过程中锁内存怎么变化的，？
- 5.请介绍一下 jvm 内置锁 的 从轻量级锁升级重量级锁内存怎么变化的？
- 6.请介绍一下 jvm 锁的 膨胀过程？锁内存怎么变化的？

最近有小伙伴在面试阿里，又遇到了 jvm 内置锁 膨胀相关的面试题。小伙伴 支支吾吾的说了几句，没说清楚，面试挂了。

所以，尼恩给大家做一下系统化、体系化的梳理，使得大家内力猛增，可以充分展示一下大家雄厚的“技术肌肉”，**让面试官爱到“不能自己、口水直流”**，然后实现“offer直提”。

完整的答案 参见这个链接：

[阿里面试：说说 jvm 锁的 膨胀过程？锁内存怎么变化的？](#)

美团面试：G1 垃圾回收 底层原理是什么？说说你的调优过程？

尼恩说在前面

在40岁老架构师 尼恩的**读者交流群**(50+)中，最近有小伙伴拿到了一线互联网企业如得物、阿里、滴滴、极兔、有赞、希音、百度、网易、美团的面试资格，遇到很多很重要的面试题：

- 听说你是高手，说说，你的JVM调优方法论？
- 说说，何时进行JVM调优？JVM调优的基本原则？
- 说说，G1 垃圾回收器的底层原理、基本流程、调优过程？
- 说说，JVM调优量化目标？JVM调优的步骤？

最近有小伙伴在面试 美团，又遇到了相关的面试题。小伙伴懵了，因为没有遇到过，所以支支吾吾的说了几句，面试官不满意，面试挂了。

所以，尼恩给大家做一下系统化、体系化的梳理，使得大家内力猛增，可以充分展示一下大家雄厚的“技术肌肉”，让面试官爱到“不能自己、口水直流”，然后实现“offer直提”。

完整的答案 参见这个链接：

[美团面试：G1 垃圾回收 底层原理是什么？说说你的调优过程？](#)

微博一面：RPC怎么做零呼损？

说在前面

在40岁老架构师 尼恩的[读者交流群](#)(50+)中，最近有小伙伴拿到了一线互联网企业如微博、阿里、汽车之家、极兔、有赞、希音、百度、网易、滴滴的面试资格，遇到一几个很重要的面试题：

- RPC怎么做无损升级？
- 微服务发布的时候，RPC怎么做零呼损？

与之类似的、其他小伙伴遇到过的问题还有：

- 微服务升级时，RPC里怎么避免调用方业务受损呢？

所以，这里尼恩给大家做一下系统化、体系化的梳理，使得大家可以充分展示一下大家雄厚的“技术肌肉”，让面试官爱到“不能自己、口水直流”。

也一并把这个题目以及参考答案，收入咱们的《[尼恩Java面试宝典](#)》V108版本，供后面的小伙伴参考，提升大家的 3高 架构、设计、开发水平。

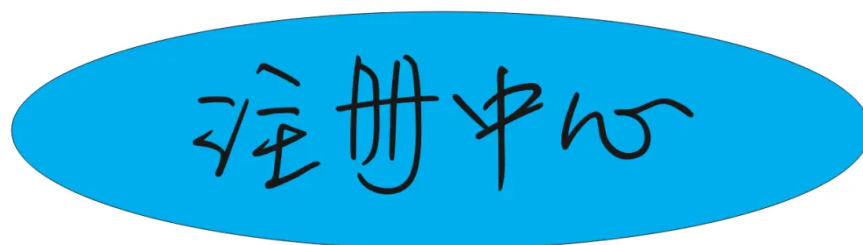
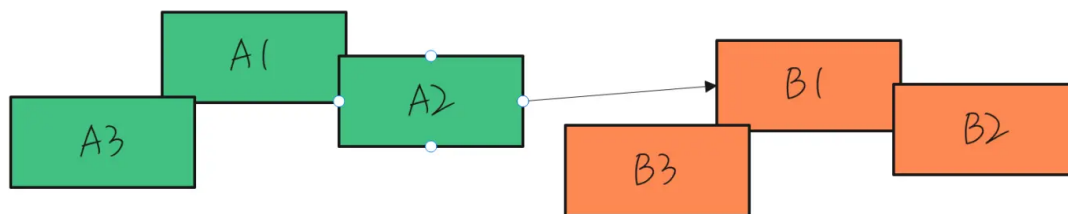
注：本文以 PDF 持续更新，最新尼恩 架构笔记、面试题 的PDF文件，请从公众号【技术自由圈】获取。

场景分析

微服务架构中，线上跑着几十个，甚至上百的微服务。

服务实例之间，有着错综复杂的RPC调用关系。

这些服务实例，归属到不同的小分队进行开发、设计、维护。



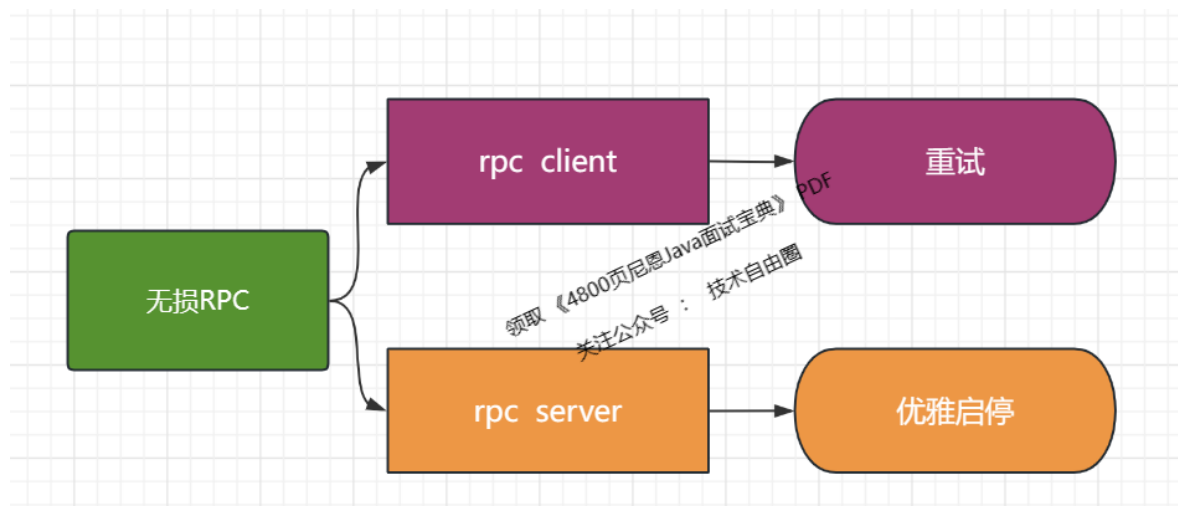
问题是：

在其中部分服务实例**重启、升级的过程中**，怎么做到让微服务RPC调用方系统不出问题呢？

确保RPC零呼损的维度

如果要确保RPC零呼损，至少可以从以下两个维度进行规避：

- 维度一：rpc client 维度
- 维度一：rpc server 维度



rpc client 维度零呼损的有效措施是：重试

rpc server 维度零呼损的有效措施是：优雅启停

rpc client 维度的零呼损：重试

rpc client 在发生错误的时候，可以进行 服务实例的 重试。

常见的rpc框架如 OpenFeign 框架就有重试几次、重试间隔这样的参数。

当然，如果希望通过重试的机制，让RPC零呼损，那么要保证升级的时候，还有健康的实例可用。

不能所有的微服务实例都同时升级，从而同时不可以使用，这样的话，重试也没有意义。

所以，重试措施最好配套对应的线上滚动升级/滚动发布、或灰度升级/灰度发布等类似的策略。

rpc server 维度的零呼损：优雅启停

优雅启停包括：

- 优雅启动
- 优雅停止/优雅下线

优雅启动

优雅启动，当微服务实例真正完成启动，甚至完成预热之后，真正具备处理 rpc 请求能力的时候，再将实例自己注册到注册中心。

为啥要优雅启动呢？核心原因是：避免了RPC请求发进来，没有完成启动流程的微服务实例却无法处理。

优雅启动的流程，还包含JVM预热的流程，有关JVM预热的方案，请查看尼恩的上一篇文章：

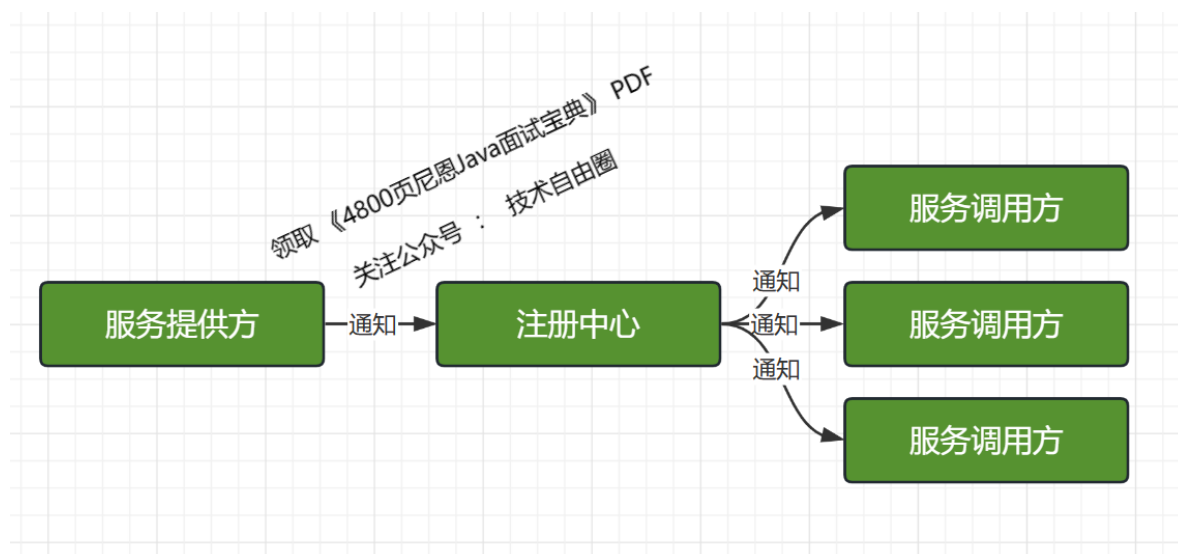
[微博一面：JVM预热，你的方案是啥？](#)

优雅停止/优雅下线

在服务下线前，先通过“某种方式”把要下线的实例从调用方维护的“健康列表”里面删除就可以了，这样负载均衡就选不到这个节点

这个操作，一般来说，都是依赖注册中心完成的。

当服务提供方关闭前，先通知注册中心进行下线，然后通过注册中心告诉调用方进行节点摘除。



如上图所示，整个关闭过程中依赖了两次 RPC 调用：

- 一次是服务提供方通知注册中心下线操作
- 一次是注册中心通知服务调用方下线节点操作。

需要注意的是：两次通知都是异步的，只保证最终一致性，并不保证强一致性。

这个中间，有一定的时间周期，所以，如果要做到应用无损升级，需要在发出通知之后，隔一段时间，再把服务实例正式关闭。

优雅停止/优雅下线的简单流程

1. 下线实例在注册中心进行注销，注销该实例元数据信息；
2. 注册中心节点元数据更新周期为15s，调用方在感知注册中心实例变更后，更新本地缓存服务地址，不再将流量路由到下线实例，期间保障业务无中断；
3. 下线实例等待30s（2个心跳周期）后，进行实际下线操作；

优雅停止的总结：优雅停止是当微服务快要下线的时候，先从注册中心进行去注销，然后把接收到的RPC调用消息，处理完毕后，再彻底关闭。通过优雅停机，可以有效地防止升级期间，发送到老节点的呼损。需要注意发送下线通知，到正式下线之间的时间间隔。

确保RPC零呼损的维度

如果要确保RPC零呼损，至少可以从以下两个维度进行规避：

- 维度一：rpc client 维度
- 维度一：rpc server 维度

rpc client 维度的核心策略是重试

rpc server 维度的核心策略是优雅启停

两个维度都不可或缺，都需要实现。

说在最后

RPC 相关面试题，是非常常见的面试题。

以上的内容，如果大家能对答如流，如数家珍，基本上面试官会被你震惊到、吸引到。

在面试之前，建议大家系统化的刷一波 5000页《尼恩Java面试宝典》，并且在刷题过程中，如果有啥问题，大家可以来找 40岁老架构师尼恩交流。

最终，让面试官爱到“不能自己、口水直流”。offer，也就来了。

参考文献

<https://blog.csdn.net/Weixiaohuai/article/details/125391957>

<https://www.cnblogs.com/daoqidelv/p/7043696.html>

清华大学出版社《Java高并发核心编程 卷2 加强版》

参考文献

虚拟机规范 <https://docs.oracle.com/javase/specs/index.html>
<https://tin.js.org/2021/02/14/openjdk-constant-pool/>
<http://ifeve.com/jvm-random-and-entropy-source/>
https://blog.csdn.net/m0_49095721/article/details/120710937
<https://blog.csdn.net/u012060033/article/details/100604722>
https://blog.csdn.net/weixin_43215305/article/details/120028825
<http://www.hollischuang.com/archives/201>
<https://www.cnblogs.com/knqiufan/p/16345488.html>
<https://www.bbsmax.com/A/WpdKp1ymzV/>
https://blog.csdn.net/weixin_39841589/article/details/112597193
<https://blog.csdn.net/Herishwater/article/details/123698633>

推荐阅读

《[百亿级访问量，如何做缓存架构设计](#)》
《[多级缓存 架构设计](#)》
《[消息推送 架构设计](#)》
《[阿里2面：你们部署多少节点？1000W并发，当如何部署？](#)》
《[美团2面：5个9高可用99.999%，如何实现？](#)》
《[网易一面：单节点2000Wtps，Kafka怎么做的？](#)》
《[字节一面：事务补偿和事务重试，关系是什么？](#)》
《[网易一面：25Wqps高吞吐写Mysql，100W数据4秒写完，如何实现？](#)》
《[亿级短视频，如何架构？](#)》
《[炸裂，靠“吹牛”过京东一面，月薪40K](#)》
《[太猛了，靠“吹牛”过顺丰一面，月薪30K](#)》
《[炸裂了...京东一面索命40问，过了就50W+](#)》
《[问麻了...阿里一面索命27问，过了就60W+](#)》
《[百度狂问3小时，大厂offer到手，小伙真狠！](#)》
《[饿了么太狠：面个高级Java，抖这多硬活、狠活](#)》
《[字节狂问一小时，小伙offer到手，太狠了！](#)》
《[收个滴滴Offer：从小伙三面经历，看看需要学点啥？](#)》

技术自由圈

实现架构转型，再无中年危机



关注**技术自由圈**公众号，获取每天技术干货
一起成为牛逼的**未来超级架构师**

几十篇架构笔记、5000页面试宝典、20个技术圣经

请加尼恩个人微信 免费拿走

暗号，请在 公众号后台 发送消息：**领电子书**